PROJECT DOCUMENTATION

Library Management System

Complete Project Documentation

A four-role PHP and MySQL web application, explained line by line for technical and non-technical readers alike.

System	Library Management System (LibSys)
Roles	Administrator, Librarian, Student, Visitor
Backend	PHP 8 with procedural MySQLi and prepared statements
Database	MySQL / MariaDB — 7 tables
Frontend	HTML, CSS and vanilla JavaScript with AJAX / JSON
Architecture	MVC with a single front controller
Runs on	XAMPP (Apache + PHP + MySQL)
Size	26 files, approximately 4,000 lines
Testing	60 automated end-to-end checks — 60 passed, 0 failed
Document date	31 August 2026

Contents

The table below is generated by Word. If it appears empty, click anywhere inside it and press F9, or answer "Yes" when Word offers to update fields on opening.

Contents at a glance

Part 1 — Introduction

This chapter explains what the software is and what problem it solves, in ordinary language. No programming knowledge is needed to read it.

1.1 What this project is

This project is a **Library Management System**: a small website that a school, college or public library can run on its own computer to keep track of who works there, who borrows books, which books exist, and who has walked in for the day.

Before software like this, a library kept the same information in paper registers. A register is slow to search, easy to lose, and only one person can hold it at a time. This system keeps exactly the same information in a database, which means many people can use it at once, searching takes a fraction of a second, and nothing is lost when a page tears.

The system is deliberately small. It was written to be **read**, not just to be run. Every file is short, every function does one job, and the comments in the code explain the reasoning rather than repeating what the code already says. A student who has finished one term of PHP should be able to open any file in this project and follow it from top to bottom.

1.2 Who uses it

Four kinds of people sign in, and each one sees a different screen. The screen a person sees is decided by their **role**, which is stored next to their name in the database.

Role	Who this is in real life	What they see when they sign in
Admin	The person in charge of the library system — usually one member of staff or the IT teacher.	A screen for creating and managing every other account, approving membership requests, and reading a log of everything that has happened.
Librarian	Staff behind the desk who handle the books.	A screen for adding books to the catalogue, handing books out to students, taking them back, and watching which books are running low.
Student	A member who may take books home.	A screen for browsing the catalogue, asking to borrow a book, and checking when each book is due back and what the fine is if it is late.
Visitor	Someone who is not a member — a guest, a researcher, a parent.	A screen for booking a day pass to sit in the reading room, suggesting books the library should buy, and applying to become a full member.

1.3 What the system can do

Every one of the four roles can do the same five basic things with the records it owns. In software these five are so common that they have a nickname, **CRUD**, plus search:

Letter	Meaning	Example in this system
C	Create	A librarian adds a new book to the catalogue.
R	Read	The librarian sees the list of all books on screen.
U	Update	The librarian corrects a misspelled author name.

Letter	Meaning	Example in this system
D	Delete	The librarian removes a book that was lost.
S	Search	The librarian types "Cormen" and the table narrows down as they type.

On top of CRUD, each role has **three special features that no other role has**. These are listed in full in Part 7. In short: the admin can suspend accounts and read the activity log; the librarian runs the issue desk and gets low-stock warnings; the student gets a due-date and fine tracker and a printable library card; the visitor gets a printable day pass and a book suggestion box.

1.4 The ideas behind the design

1. **Separate the parts.** The code that talks to the database, the code that makes decisions, and the code that draws the screen are kept in three different folders. This is called MVC and is explained in Part 4.
2. **Never trust the browser.** Anything typed by a person is checked twice: once in the browser for speed, and again on the server, because the browser check can be switched off by anyone.
3. **Assume someone will try to break it.** Passwords are scrambled before they are stored, database queries are built in a way that makes injection impossible, and every form carries a secret token that a foreign website cannot guess.
4. **Explain, do not impress.** Where a clever one-line trick and a plain five-line version both work, the project uses the plain version.

1.5 How to read this document

If you are…	Read these parts	You can skip
A non-technical reader (a manager, an examiner, a client)	Parts 1, 2, 7 and the glossary in Part 13	Parts 5 and 6, which are pure code
A student who must present this project	All of it, but especially Parts 4, 5, 8 and the viva questions in Part 14	Nothing
A developer who has to change something	Parts 3, 4, 5, 6 and 12	Part 2 if the system already runs

In Parts 5 and 6 every source file appears twice: first as a **complete numbered listing**, then as a **table that explains each line by its number**. The line numbers in the tables were generated automatically from the real files, so they always match.

Part 2 — The system in plain language

A walk through what actually happens, with no code. If you only read one chapter, read this one.

2.1 What a "web application" is

When you type an address into a browser and press Enter, the browser sends a short message across the network asking for a page. A program on the other computer — the **server** — reads that request, works out what to send back, and returns a page made of HTML. The browser draws it.

That is all this project is: a program that receives requests and returns pages. The program is written in **PHP**. The information it works with is stored in **MySQL**. The pages it returns are styled with **CSS** and made interactive with **JavaScript**.

The important thing to understand is **where each piece runs**. PHP and MySQL run on the server, where the user cannot see or change them. HTML, CSS and JavaScript run inside the user's own browser, where a determined user can read and change anything. That single fact explains most of the security decisions in this project.

2.2 A day in the life of the system

Here is a complete story that touches every role. Follow it once and the whole system will make sense.

1. **Morning.** Nabila is not a library member. She opens the site, clicks "Create an account", and chooses **Visitor**. The system stores her details and scrambles her password so that even the administrator cannot read it.
2. Nabila signs in and books a **day pass** for Thursday. The system generates a code such as `LIB-4821-KQ` and shows her a pass she can print. She also types two books she wishes the library would buy into the **suggestion box**.
3. She decides she would rather borrow books than only read them in the building, so she fills in the **membership application** and explains why. The application is now waiting.
4. **Mid-morning.** Kamal, a librarian, signs in. His screen shows the catalogue. He adds a newly delivered book: title, author, category, ISBN, three copies, price. The book is now visible to every student.
5. **Lunchtime.** Sadia, a student, signs in. She searches the catalogue for "algorithms", sees that two copies are on the shelf, and clicks **Request**. She picks a collection date and adds a note explaining it is for her final-year project. Her request is now waiting for a librarian.
6. **Afternoon.** Kamal sees Sadia's request on his **issue desk**. He clicks **Issue**. Three things happen in one step: the request is marked as issued, a due date fourteen days from today is calculated and stored, and the number of copies on the shelf drops from two to one.
7. **Later.** The admin signs in. On the dashboard is Nabila's membership application. The admin clicks **Approve**. Nabila's role changes from visitor to student, so the next time she signs in she sees the student screen instead of the visitor screen.
8. **Three weeks later.** Sadia has not returned the book. Her dashboard shows the book in red, "4 days late", and a fine that grows by five every day. When she finally brings it back, Kamal clicks **Return**. The system works out the fine, stores it against the record, and puts the copy back on the shelf.
9. **Any time.** The admin opens the **activity log** and sees every one of those steps listed with the username, the role, the time and the network address of the computer that did it.

2.3 Why the roles are kept apart

A student must not be able to delete books. A visitor must not be able to see other people's accounts. The system enforces this in two places, not one.

The first place is the **screen**: a student is never shown a "Delete book" button. But hiding a button is not security, because anyone can type an address by hand. So there is a second place: before any page runs, the server checks the role stored in the session and turns away anybody who does not belong. If a student types the admin address into the browser, the server sends them straight back to their own screen. The same check is repeated on every background data request.

> **NOTE**
> This is worth stressing to a non-technical reader: **hiding something on screen is not protecting it.** The protection is the server-side check that happens before the page is even built.

2.4 What the system deliberately does not do

Being honest about the boundaries of a project is part of documenting it. This system does not do the following, and none of them are bugs:

- It does not send email or SMS reminders. Overdue books are shown on screen when the student signs in.
- It does not take payments. A fine is calculated and recorded, but money changes hands at the desk.
- It does not handle reservations or a waiting list. If every copy is out, the student must try again later.
- It does not produce printed barcode labels. Books are identified by their ISBN, typed by the librarian.
- It is designed for a single library on a single server, not for many branches sharing one system.

Part 3 — Technology and installation

What each piece of technology is, why it was chosen, and exactly how to get the project running.

3.1 The technology stack, explained

Technology	What it is	What it does in this project
PHP 8	A programming language that runs on the server.	Everything that thinks: reading the form a user submitted, checking it, talking to the database, and building the HTML that goes back to the browser.
MySQL / MariaDB	A database: an organised, permanent store of information.	Holds the seven tables — users, books, borrow requests, visit passes, membership applications, suggestions, and the activity log.
mysqli (procedural)	The set of PHP functions used to talk to MySQL.	Every database call in this project goes through `mysqli_prepare` and friends. The procedural style was chosen over objects because it is what most first courses teach.
HTML	The language that describes the structure of a page.	Produced by the files in the `views/` folder — headings, tables, forms.
CSS	The language that describes how a page looks.	One file, `assets/css/style.css`, controls the colours, spacing and layout of every screen.
JavaScript	A programming language that runs inside the browser.	Two jobs: checking a form before it is sent, and fetching fresh data in the background so a table can update without reloading the page.
AJAX / JSON	A technique for fetching data in the background, and the simple text format that data arrives in.	Powers every search box in the project and the live statistics on the admin screen.
XAMPP	A single installer that bundles the Apache web server, PHP and MySQL.	The environment the project is meant to run in. Nothing else needs to be installed.

NOTE
There is no framework, no Composer, no `npm install` and no build step. The files you copy are the files that run. This is intentional: a beginner can change one line, refresh the browser, and immediately see the effect.

3.2 Installing on XAMPP, step by step

1. Install XAMPP if it is not already installed, accepting the default location `C:\xampp`.
2. Unzip the project and copy the whole `library_management` folder into `C:\xampp\htdocs\`. When you are finished, the file `index.php` should sit at `C:\xampp\htdocs\library_management\index.php`.
3. Open the XAMPP Control Panel and press **Start** next to **Apache** and again next to **MySQL**. Both should turn green.
4. Open a browser and go to `http://localhost/phpmyadmin`.
5. Click the **Import** tab, press **Choose File**, select `database.sql` from the project folder, scroll down and press **Go**. A green success message means the seven tables were created.

6. Go to `http://localhost/library_management/`. You should see the purple sign-in screen.
7. Sign in with username `admin` and password `admin123`.

That is the entire installation. There is no configuration file to edit unless your MySQL has a password — see the next section.

3.3 The one setting you may need to change

XAMPP normally ships with a MySQL user called `root` and no password, which is what the project expects. If your MySQL does have a password, open `config/config.php` and put it on the `DB_PASS` line:

config/config.php — the four database settings

```
define('DB_HOST', 'localhost');
define('DB_USER', 'root');
define('DB_PASS', '');
define('DB_NAME', 'library_db');
```

3.4 The default administrator

A brand-new database has no users at all, which would leave nobody able to sign in. To avoid that trap the project creates one administrator automatically the first time any page is loaded. The code sits at the bottom of `config/config.php`:

```
/* ---------- 5. Create the default admin the first time the app runs ---------- */
// Runs only when there is no admin yet.  Login: admin / admin123
$check = mysqli_query($conn, "SELECT id FROM users WHERE role = 'admin' LIMIT 1");
if ($check && mysqli_num_rows($check) === 0) {
    $hash = password_hash('admin123', PASSWORD_DEFAULT);
    $stmt = mysqli_prepare($conn,
        "INSERT INTO users (name, email, contact, username, password, role)
         VALUES ('Administrator', 'admin@libsys.test', '0000000000', 'admin', ?, 'admin')");
    mysqli_stmt_bind_param($stmt, 's', $hash);
    mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
```

It first asks the database whether any user with the role `admin` already exists. Only if the answer is "none" does it create one. That means the block does its job once and then quietly does nothing forever after, no matter how many times pages are loaded.

NOTE
In a real library the very first thing you should do after signing in is create your own admin account and delete or change the password of the default one. Everybody who has ever seen this project knows that `admin123` is the default.

3.5 If something goes wrong

What you see	What it means and how to fix it
"Database connection failed. Did you import database.sql?"	PHP reached the page but could not reach MySQL. Either MySQL is not started in the XAMPP panel, or `database.sql` was never imported, or the username and password in `config/config.php` are wrong.
A completely blank white page	A PHP error is being hidden. Open `C:\xampp\php\php.ini`, set `display_errors = On`, restart Apache, and reload — the real message will now appear.
The page appears but has no colours or layout	The browser could not load `assets/css/style.css`. Almost always this means the folder was renamed or the project was copied one level too deep inside `htdocs`.
Search boxes do nothing	The background request is failing. Press F12, open the Console tab, and reload. A 403 in red means the session expired — sign in again.

What you see	What it means and how to fix it
"Security check failed (invalid CSRF token)"	The page sat open long enough for the session to expire, or the browser is blocking cookies. Go back to the sign-in page and start again.
Port 80 is busy and Apache will not start	Something else on the machine (often Skype or IIS) is using port 80. Change Apache’s port in the XAMPP config, or close the other program.

Part 4 — How the project is organised

The MVC pattern, the folder layout, and the exact journey a single click takes through the code.

4.1 MVC explained without jargon

MVC stands for **Model, View, Controller**. It is a rule about where code is allowed to live. The easiest way to understand it is a restaurant:

Part	In a restaurant	In this project
Model	The kitchen store room. It holds the ingredients and nothing else. It does not talk to customers.	The `models/` folder. Every SQL query lives here. A model function fetches or saves data and returns it. It never prints a single character of HTML.
View	The plate the food is served on. It makes things look good but decides nothing.	The `views/` folder. These files are mostly HTML with small pieces of PHP that print values. A view never runs a database query.
Controller	The waiter. Takes the order, checks it makes sense, fetches from the kitchen, and arranges the plate.	The `controllers/` folder. Reads what the user submitted, validates it, calls the model, then hands the result to the view.

Why bother? Because when the rule is followed, every change has exactly one home. If a price is displaying with the wrong number of decimal places, the bug is in a view. If a book is being saved with the wrong author, the bug is in a model. If a user is being allowed to do something they should not, the bug is in a controller. Without the rule, all three kinds of bug could be anywhere in three thousand lines.

4.2 The folder structure

```
library_management/
├── index.php                  The front controller: the ONLY way in
├── database.sql               The schema, plus a few sample books
├── README.md                  Short setup notes
│
├── config/
│   └── config.php             Database connection, session settings, constants
│
├── helpers/
│   └── helpers.php            Small tools used everywhere: escaping, CSRF,
│                              login guards, flash messages, fine maths
│
├── models/                    M  —  all SQL lives here
│   ├── user_model.php         All four roles (one users table)
│   ├── book_model.php         The catalogue and stock levels
│   ├── borrow_model.php       Borrow requests and the issue desk
│   ├── visitor_model.php      Passes, membership, suggestions
│   └── log_model.php          The activity log
│
├── controllers/               C  —  decisions and validation
│   ├── auth_controller.php    Sign in, sign up, sign out
│   ├── admin_controller.php
│   ├── librarian_controller.php
│   ├── student_controller.php
│   ├── visitor_controller.php
│   └── ajax_controller.php    Every background/JSON request
│
├── views/                     V  —  HTML only
│   ├── partials/              header.php and footer.php, shared by all screens
│   ├── auth/                  login.php, register.php
│   ├── admin/dashboard.php
│   ├── librarian/dashboard.php
│   ├── student/dashboard.php
│   └── visitor/dashboard.php
│
└── assets/
    ├── css/style.css          Every rule that controls how things look
    └── js/app.js              Validation, escaping, live search, AJAX tables
```

4.3 The front controller idea

In many beginner projects every screen is its own file: `login.php`, `add_book.php`, `delete_book.php`, and so on. That works, but it has a serious weakness — every one of those files has to remember to start the session, connect to the database and check the user's role. Forget the check in one file and the whole system has a hole.

This project uses a **front controller** instead. There is exactly one entry point, `index.php`. Every address in the system is that one file with different values after the question mark:

```
index.php?page=<which dashboard>&action=<what to do>&id=<which row>

index.php?page=login                                    the sign-in screen
index.php?page=register                                 the sign-up screen
index.php?page=admin                                    the admin dashboard
index.php?page=librarian&action=edit&id=4               load book 4 into the form
index.php?page=student&action=delete&id=7&csrf_token=..  cancel request 7
index.php?page=ajax&action=search_books&q=php           returns JSON, not HTML
index.php?page=logout                                   sign out
```

Because there is only one door, the session, the database connection and the role check can be done once, in one place, and they can never be forgotten.

4.4 The journey of one click

Suppose a librarian is looking at the catalogue and clicks **Delete** on a book. Here is every step, in order.

#	Where	What happens
1	Browser	The link is `index.php?page=librarian&action=delete&id=12&csrf_token=…`. A JavaScript `confirm()` box asks "Delete this book?" If the user presses Cancel, nothing is sent at all.
2	Apache	The web server receives the request and, because the address ends in `.php`, hands it to PHP rather than sending the file as text.
3	`index.php`	Loads `config.php`, which starts the session and connects to MySQL. Then loads the helpers, all five models and all six controllers.
4	`index.php`	Calls `check_session_timeout()`. If the librarian has been idle for more than thirty minutes they are signed out here and never reach the delete.
5	`index.php`	Reads `page=librarian` and calls `require_role('librarian')`. A student arriving at this address is redirected away at this exact point.
6	`librarian_controller.php`	Reads `action=delete`, calls `csrf_check()` to prove the request came from our own page, and converts `id=12` to a whole number with `(int)`.
7	`book_model.php`	`delete_book()` runs a prepared `DELETE` statement with 12 bound as an integer parameter.
8	MySQL	The row is removed. Because `borrow_requests` has a foreign key with `ON DELETE CASCADE`, any borrow records for that book go too, so no orphan rows are left behind.
9	`log_model.php`	`log_activity()` writes "Deleted book #12" into the activity log with the username, role, time and IP address.
10	`librarian_controller.php`	Stores a one-time flash message, "Book deleted.", in the session and redirects the browser back to `index.php?page=librarian`.
11	Browser	Follows the redirect. The controller runs again, this time in plain list mode, fetches the books, and hands them to the view.
12	View	Prints the green success message once (reading it removes it), then draws the table without the deleted book.

NOTE

Step 10 is a pattern worth naming: **Post/Redirect/Get**. After anything that changes data, the code redirects instead of printing a page directly. Without it, pressing F5 would repeat the delete, and the browser would keep asking "resend this form?"

Part 5 — The database

Every table and every column, explained one at a time.

5.1 Why a database and not files

A database is a program whose only job is to store information and answer questions about it quickly. It could all be kept in text files instead, but then the project would have to write its own code for searching, for sorting, for stopping two people writing at the same moment, and for making sure a book is not left pointing at a librarian who no longer exists. A database already solves all of that, and has done for forty years.

The information here is split across **seven tables**. A table is a grid: the columns are the fields, and each row is one record.

Table	One row means
`users`	One person who can sign in — an admin, a librarian, a student or a visitor.
`books`	One title in the catalogue, with the number of copies currently on the shelf.
`borrow_requests`	One student asking for one book, and everything that happened to that request afterwards.
`visit_passes`	One day pass booked by one visitor.
`membership_applications`	One visitor asking to be upgraded to a student.
`book_suggestions`	One book a visitor thinks the library should buy.
`activity_log`	One thing that somebody did, recorded so the admin can audit it.

5.2 One table for all four roles

A common first instinct is to make four tables: `admins`, `librarians`, `students`, `visitors`. This project deliberately does not. All four live in one `users` table with a `role` column that says which kind they are.

The reason is that almost everything about a user is the same whatever their role: they have a name, an email, a contact number, a username and a password, and they all sign in through the same form. With four tables, the sign-in code would have to try each table in turn, and every feature that lists people would need four near-identical queries. With one table and a `role` column, signing in is a single query and listing people is a single query with an optional filter.

The cost is that a query which only wants students must say `WHERE role = 'student'`. That is a small price, and the database makes it fast.

5.3 The `users` table

```
CREATE TABLE IF NOT EXISTS users (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    email      VARCHAR(120) NOT NULL,
    contact    VARCHAR(30)  NOT NULL,
    username   VARCHAR(50)  NOT NULL UNIQUE,
    password   VARCHAR(255) NOT NULL,             -- stored as a password_hash()
    role       ENUM('admin','librarian','student','visitor') NOT NULL DEFAULT 'student',
    status     ENUM('active','suspended') NOT NULL DEFAULT 'active',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Column	Type	What it holds and why
id	INT, auto	A number the database invents for each new row, never reused. Everything else in the system points at a user by this number, not by name, so a person can change their name without breaking anything.
name	VARCHAR(100)	The person's full name, shown in the navigation bar and in lists.
email	VARCHAR(120)	Contact address. Checked with PHP's built-in email validator before it is stored.
contact	VARCHAR(30)	A phone number. Stored as text, not a number, because it may begin with + or 0, both of which a numeric column would destroy.
username	VARCHAR(50), UNIQUE	What the person types to sign in. UNIQUE means the database itself refuses a duplicate, even if a bug ever let one past the PHP check.
password	VARCHAR(255)	Never the real password — always the scrambled result of password_hash(). 255 characters because the hashing algorithm may get longer in future PHP versions.
role	ENUM	One of exactly four words: admin, librarian, student, visitor. ENUM means the database rejects anything else, which is a second line of defence behind the PHP check.
status	ENUM	Either active or suspended. A suspended person still exists, with all their history, but cannot sign in.
created_at	DATETIME	Filled in automatically. Used for the "member since" line on the student's library card.

5.4 The `books` table

```
CREATE TABLE IF NOT EXISTS books (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    title       VARCHAR(200) NOT NULL,
    author      VARCHAR(120) NOT NULL,
    category    VARCHAR(60)  NOT NULL,
    isbn        VARCHAR(30)  NOT NULL,
    quantity    INT NOT NULL DEFAULT 0,             -- copies currently on the shelf
    price       DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    added_by    INT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (added_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;
```

Column	Type	What it holds and why
id	INT, auto	The catalogue number for this title.
title, author	VARCHAR	The obvious. Both are searchable.
category	VARCHAR(60)	A free-text grouping such as "Programming" or "Science". Kept as text rather than a separate table to keep the project small; turning it into its own table is a good exercise.
isbn	VARCHAR(30)	The book industry's standard identifier. Text, not a number, because ISBNs contain dashes and can begin with zero.
quantity	INT	**Copies currently on the shelf**, not copies owned. Issuing a book lowers it by one; a return raises it by one. This is the single number that decides whether a student may request the book.
price	DECIMAL(10,2)	Money is stored as DECIMAL, never as a floating-point number, because floating point cannot represent values like 0.10 exactly and small errors accumulate.

Column	Type	What it holds and why
added_by	INT, foreign key	Which librarian added it. ON DELETE SET NULL means that if that librarian's account is later deleted, the book stays in the catalogue and simply forgets who added it.

5.5 The `borrow_requests` table

This is the busiest table in the system, because one row lives through four stages: a student creates it, a librarian issues it, time passes, and finally it comes back.

```
CREATE TABLE IF NOT EXISTS borrow_requests (
    id           INT AUTO_INCREMENT PRIMARY KEY,
    student_id   INT NOT NULL,
    book_id      INT NOT NULL,
    pickup_date  DATE NOT NULL,
    notes        VARCHAR(255) NOT NULL DEFAULT '',
    status       ENUM('pending','issued','returned','rejected') NOT NULL DEFAULT 'pending',
    request_date DATE NOT NULL,
    issue_date   DATE NULL,
    due_date     DATE NULL,
    return_date  DATE NULL,
    fine         DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    FOREIGN KEY (student_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (book_id)    REFERENCES books(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

Column	Type	What it holds and why
student_id	INT, foreign key	Which student. ON DELETE CASCADE means deleting the student deletes their requests too, so no request is ever left pointing at nobody.
book_id	INT, foreign key	Which book. Also cascades.
pickup_date	DATE	When the student says they will collect it. This is the field they are allowed to change while the request is still waiting.
notes	VARCHAR(255)	An optional message to the librarian.
status	ENUM	The stage this request has reached: pending, issued, returned or rejected. Almost every rule in the borrowing system is really a rule about this one column.
request_date	DATE	When the student asked. Set by PHP, not by the student.
issue_date	DATE, may be empty	The day the librarian handed the book over.
due_date	DATE, may be empty	Issue date plus fourteen days. Everything about lateness and fines is measured against this one value.
return_date	DATE, may be empty	The day it came back.
fine	DECIMAL(10,2)	Zero until the book is returned; then the calculated amount is frozen here so it stops growing.

NOTE

The four status values are a **state machine**. A request may only move pending → issued → returned, or pending → rejected. The SQL enforces this: the issue query ends with AND status = 'pending' and the return query ends with AND status = 'issued', so an out-of-order attempt simply changes no rows.

5.6 The three visitor tables

```
CREATE TABLE IF NOT EXISTS visit_passes (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    visitor_id INT NOT NULL,
    pass_code  VARCHAR(20) NOT NULL,
    visit_date DATE NOT NULL,
```

```
    purpose    VARCHAR(150) NOT NULL,
    guests     INT NOT NULL DEFAULT 1,
    status     ENUM('booked','cancelled') NOT NULL DEFAULT 'booked',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (visitor_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

`pass_code` is a short human-friendly code such as `LIB-4821-KQ`, generated in PHP so a visitor can read it aloud at the front desk. `guests` allows a visitor to bring up to ten people. `status` is booked or cancelled — cancelling keeps the record rather than erasing history.

```
CREATE TABLE IF NOT EXISTS membership_applications (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    visitor_id INT NOT NULL,
    reason     VARCHAR(255) NOT NULL,
    status     ENUM('pending','approved','rejected') NOT NULL DEFAULT 'pending',
    applied_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (visitor_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

One row per application. The admin approves or rejects it; approving also changes that user's `role` in the `users` table from visitor to student. Old applications are kept, so a visitor who was rejected once and approved later has both records.

```
CREATE TABLE IF NOT EXISTS book_suggestions (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    visitor_id INT NOT NULL,
    title      VARCHAR(200) NOT NULL,
    author     VARCHAR(120) NOT NULL,
    reason     VARCHAR(255) NOT NULL DEFAULT '',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (visitor_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

The suggestion box. A visitor may add and remove their own suggestions.

5.7 The `activity_log` table

```
CREATE TABLE IF NOT EXISTS activity_log (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    user_id    INT NULL,
    username   VARCHAR(50)  NOT NULL,
    role       VARCHAR(20)  NOT NULL,
    action     VARCHAR(255) NOT NULL,
    ip         VARCHAR(45)  NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Notice that `username` and `role` are copied into this table as plain text rather than only being linked by `user_id`. That is deliberate. If an account is later deleted, the log must still be readable — "who deleted the entire catalogue last Tuesday?" is exactly the question you ask after an account has disappeared. `user_id` is allowed to be empty for the same reason.

5.8 How the tables connect

```
                      users
                        |  id
      +-----------------+-----------------+------------------+
      |                 |                 |                  |
  books.added_by   borrow_requests    visit_passes    membership_applications
  (SET NULL)        .student_id        .visitor_id       .visitor_id
                    (CASCADE)          (CASCADE)         (CASCADE)
                         |
                         |  book_id  (CASCADE)
                         v
                      books

  book_suggestions.visitor_id -> users.id   (CASCADE)
  activity_log.user_id        -> no link, kept as plain text on purpose
```

A **foreign key** is a promise the database enforces: the number in this column must exist in that other table. Without it, a bug could leave a borrow request pointing at book number 999 which was deleted last year, and every page that joined the two tables would break. The two rules used here are:

- **ON DELETE CASCADE** — "if the parent goes, take the children too." Used where a child record is meaningless without its parent: a borrow request without a student, a pass without a visitor.
- **ON DELETE SET NULL** — "if the parent goes, just forget it." Used for `books.added_by`, because a book must obviously survive the departure of the librarian who catalogued it.

5.9 The sample data

The bottom of `database.sql` inserts six real books so that no screen is empty on the first day. They are ordinary titles with sensible categories, quantities and prices; two of them are deliberately given a quantity of two so that the librarian's low-stock alert has something to show immediately.

```
INSERT INTO books (title, author, category, isbn, quantity, price) VALUES
('The Pragmatic Programmer', 'Andrew Hunt',      'Programming', '9780201616224', 5, 45.00),
('Clean Code',               'Robert C. Martin', 'Programming', '9780132350884', 3, 39.50),
('Introduction to Algorithms','Thomas H. Cormen','Computer Science','9780262033848', 2, 89.00),
('Database System Concepts', 'Abraham Silberschatz','Database',  '9780078022159', 4, 72.00),
('Head First PHP & MySQL',   'Lynn Beighley',    'Web Development','9780596006303', 6, 34.99),
('A Brief History of Time',  'Stephen Hawking',  'Science',     '9780553380163', 2, 18.00);
```

Part 6 — Code walkthrough: the foundation

The four files that everything else depends on: the configuration, the helper toolbox, the router, and the pattern every model follows.

From here on, each file is presented twice. First the **complete listing** with line numbers, so nothing is hidden. Then a **table that explains the lines by number**. The line numbers were read from the real files while this document was being generated, so they cannot be out of date.

Where a pattern repeats — and in a project like this it repeats a great deal — it is explained in full the first time and then referred back to. That is a promise that nothing is being skipped, not an excuse to skip it.

6.1 config/config.php

Purpose. This is the first file loaded on every single request. It does four things: it stores the settings, it starts a session safely, it opens the database connection, and it makes sure at least one administrator exists.

Full listing of **config/config.php** (53 lines).

```
1  <?php
2  // ============================================================
3  // CONFIG - database connection, session setup and app settings
4  // Everything in the project starts from here.
5  // ============================================================
6
7  /* ---------- 1. Database settings (change if your XAMPP differs) ---------- */
8  define('DB_HOST', 'localhost');
9  define('DB_USER', 'root');
10 define('DB_PASS', '');
11 define('DB_NAME', 'library_db');
12
13 /* ---------- 2. App settings ---------- */
14 define('APP_NAME',     'LibSys');
15 define('CURRENCY',     '$');
16 define('LOAN_DAYS',    14);   // how many days a student may keep a book
17 define('FINE_PER_DAY', 5);    // fine charged for every day a book is late
18 define('LOW_STOCK',    3);    // a book at or below this count is "low stock"
19 define('SESSION_TIMEOUT', 1800); // auto logout after 30 minutes of no activity
20
21 /* ---------- 3. Start a hardened session ---------- */
22 // SECURITY: the cookie cannot be read by JavaScript (httponly) and is not
23 // sent on cross-site requests (samesite), which blocks most XSS/CSRF tricks.
24 if (session_status() === PHP_SESSION_NONE) {
25     session_set_cookie_params([
26         'lifetime' => 0,
27         'path'     => '/',
28         'httponly' => true,
29         'samesite' => 'Lax',
30     ]);
31     session_start();
32 }
33
34 /* ---------- 4. Connect to MySQL (procedural mysqli) ---------- */
35 $conn = mysqli_connect(DB_HOST, DB_USER, DB_PASS, DB_NAME);
36 if (!$conn) {
37     die('Database connection failed. Did you import database.sql? Details: '
38         . mysqli_connect_error());
39 }
40 mysqli_set_charset($conn, 'utf8mb4');
41
42 /* ---------- 5. Create the default admin the first time the app runs ---------- */
43 // Runs only when there is no admin yet.  Login: admin / admin123
44 $check = mysqli_query($conn, "SELECT id FROM users WHERE role = 'admin' LIMIT 1");
45 if ($check && mysqli_num_rows($check) === 0) {
46     $hash = password_hash('admin123', PASSWORD_DEFAULT);
47     $stmt = mysqli_prepare($conn,
48         "INSERT INTO users (name, email, contact, username, password, role)
49          VALUES ('Administrator', 'admin@libsys.test', '0000000000', 'admin', ?, 'admin')");
50     mysqli_stmt_bind_param($stmt, 's', $hash);
51     mysqli_stmt_execute($stmt);
52     mysqli_stmt_close($stmt);
53 }
```

Line by line

Line	Code	What it does and why
1	`<?php`	Tells the server that everything below is PHP and must be executed, not sent to the browser as text.

Line	Code	What it does and why

8–11	`define('DB_HOST', 'localhost');`
	`...`
	`define('DB_NAME', 'library_db');`
		The four database settings. `define()` creates a **constant** — a value that cannot be changed once set, unlike a variable. Constants are used here because a database password should never be accidentally overwritten halfway through a request. These are the only four lines most people ever need to edit.

14	`define('APP_NAME',  'LibSys');`	The name shown in the navigation bar and page titles. Change it here and it changes everywhere.
15	`define('CURRENCY',  '$');`	The currency symbol printed before every price and fine.
16	`define('LOAN_DAYS',  14);  // how many days a student may keep a book`	How many days a student may keep a book. The due date is calculated as today plus this number.
17	`define('FINE_PER_DAY', 5);  // fine charged for every day a book is late`	The fine charged for each day a book is late. Used by `calculate_fine()` in the helpers.
18	`define('LOW_STOCK',  3);  // a book at or below this count is "low stock"`	A book with this many copies or fewer appears in the librarian's low-stock alert.
19	`define('SESSION_TIMEOUT', 1800); // auto logout after 30 minutes of no activity`	1800 seconds is thirty minutes. After this much inactivity the user is signed out automatically.
24	`if (session_status() === PHP_SESSION_NONE) {`	Only start a session if one is not running already. Calling `session_start()` twice produces a warning, and this file may be loaded from more than one place.

25–30	`session_set_cookie_params([`
	`...`
	`]);`
		Settings applied to the session cookie **before** the session starts — afterwards is too late. `lifetime => 0` means the cookie dies when the browser closes.

28	`'httponly' => true,`	**Security.** JavaScript is forbidden from reading this cookie. If an attacker ever managed to inject a script into a page, it still could not steal the session and impersonate the user.
29	`'samesite' => 'Lax',`	**Security.** The browser will not send this cookie when another website triggers the request. This blocks a large family of cross-site request forgery attacks before the code even runs.
31	`session_start();`	Starts the session. From this line onwards `$_SESSION` is available, and PHP remembers this particular visitor between page loads.
35	`$conn = mysqli_connect(DB_HOST, DB_USER, DB_PASS, DB_NAME);`	Opens the connection to MySQL using the four constants. The result is stored in `$conn`, which is then passed as the first argument to every model function in the project.
36–37	`if (!$conn) {` `die('Database connection failed. Did you import database.sql? Details: '`	If the connection failed there is no point continuing: every later line would fail too. `die()` stops immediately with a message that names the most likely cause — a forgotten import.
40	`mysqli_set_charset($conn, 'utf8mb4');`	Tells MySQL that text travelling in both directions is UTF-8. Without it, names with accents or non-English characters arrive corrupted. `utf8mb4` is the version that also supports emoji.
44	`$check = mysqli_query($conn, "SELECT id FROM users WHERE role = 'admin' LIMIT 1");`	Asks whether any administrator exists. `LIMIT 1` stops the database after the first match, because the count does not matter — only whether there is at least one.
45	`if ($check && mysqli_num_rows($check) === 0) {`	True only when the query succeeded **and** returned nothing, meaning this is a brand-new database. On every later request this is false and the whole block is skipped.
46	`$hash = password_hash('admin123', PASSWORD_DEFAULT);`	Scrambles the default password. Even this throwaway account is never stored in readable form. `PASSWORD_DEFAULT` lets PHP choose the best

Line	Code	What it does and why
		algorithm available, so the code improves automatically as PHP is updated.
47	`$stmt = mysqli_prepare($conn,`	Prepares the insert. Note the `?` where the hash will go — the value is never glued into the SQL text.
50	`mysqli_stmt_bind_param($stmt, 's', $hash);`	Attaches the hash to the `?`, declaring it as a string (`s`). MySQL now treats it strictly as data.
51	`mysqli_stmt_execute($stmt);`	Runs the insert. The default administrator now exists.
52	`mysqli_stmt_close($stmt);`	Releases the prepared statement. Not strictly required, since PHP cleans up at the end of the request, but closing what you open is a good habit to teach.

6.2 helpers/helpers.php

Purpose. A toolbox of small functions that would otherwise be copied into a dozen files. They fall into six groups: printing safely, protecting forms, checking who is signed in, one-time messages, sending JSON, and small calculations such as fines.

If you only learn three functions from this project, learn `esc()`, `csrf_check()` and `require_role()`. Between them they carry most of the security.

Full listing of **helpers/helpers.php** (172 lines).

```php
<?php
// ====================================================================
// HELPERS - small functions used everywhere (security, output, dates)
// ====================================================================

/* ================ Output safety ================ */

// SECURITY: always print user data through esc() to stop XSS.
function esc($value) {
    return htmlspecialchars((string)($value ?? ''), ENT_QUOTES, 'UTF-8');
}

// Send the browser to another page and stop the script.
function redirect($url) {
    header('Location: ' . $url);
    exit;
}

function is_post() {
    return $_SERVER['REQUEST_METHOD'] === 'POST';
}

/* ================ CSRF protection ================ */
// A CSRF token is a secret value stored in the session and copied into every
// form. A different website cannot read it, so it cannot forge a request.

function csrf_token() {
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

// Prints the hidden input that every POST form must contain.
function csrf_field() {
    echo '<input type="hidden" name="csrf_token" value="' . csrf_token() . '">';
}

// Adds the token to a link (used by Delete / Issue / Return links).
function csrf_url($url) {
    return $url . '&csrf_token=' . csrf_token();
}

// Stops the request if the token is missing or wrong.
function csrf_check() {
    $token = $_POST['csrf_token'] ?? $_GET['csrf_token'] ?? '';
    if (!is_string($token) || !hash_equals(csrf_token(), $token)) {
        http_response_code(403);
        die('Security check failed (invalid CSRF token). Please go back and try again.');
    }
}

/* ================ Login / role guards ================ */

```

```php
function is_logged_in() {
    return isset($_SESSION['user']);
}

function current_user() {
    return $_SESSION['user'] ?? null;
}

function current_role() {
    return $_SESSION['user']['role'] ?? '';
}

// Logs the user out automatically after SESSION_TIMEOUT seconds of inactivity.
function check_session_timeout() {
    if (!is_logged_in()) {
        return;
    }
    if (isset($_SESSION['last_active']) && (time() - $_SESSION['last_active']) > SESSION_TIMEOUT) {
        $_SESSION = [];
        session_regenerate_id(true);
        set_flash('error', 'Your session expired after 30 minutes. Please log in again.');
        redirect('index.php?page=login');
    }
    $_SESSION['last_active'] = time();
}

// Only lets the matching role through; everybody else is sent away.
function require_role($role) {
    if (!is_logged_in()) {
        set_flash('error', 'Please log in to continue.');
        redirect('index.php?page=login');
    }
    if (current_role() !== $role) {
        redirect('index.php?page=' . current_role());
    }
}

/* ================ Flash messages ================ */
// A flash message is shown once on the next page, then deleted.

function set_flash($type, $message) {
    $_SESSION['flash'][] = ['type' => $type, 'message' => $message];
}

function get_flash() {
    $messages = $_SESSION['flash'] ?? [];
    unset($_SESSION['flash']);
    return $messages;
}

/* ================ AJAX / JSON ================ */

function json_out($data, $code = 200) {
    http_response_code($code);
    header('Content-Type: application/json; charset=utf-8');
    echo json_encode($data);
    exit;
}

/* ================ Small utilities ================ */

function money($amount) {
    return CURRENCY . number_format((float)$amount, 2);
}

function nice_date($date) {
    return ($date && $date !== '0000-00-00') ? date('d M Y', strtotime($date)) : '-';
}

// Fine = FINE_PER_DAY for every day past the due date.
function calculate_fine($dueDate, $returnDate = null) {
    if (empty($dueDate)) {
        return 0.0;
    }
    $end  = $returnDate ? strtotime($returnDate) : time();
    $days = floor(($end - strtotime($dueDate)) / 86400);
    return $days > 0 ? (float)($days * FINE_PER_DAY) : 0.0;
}

// Negative = days left, positive = days late.
function days_late($dueDate) {
    if (empty($dueDate)) {
        return 0;
    }
    return (int)floor((time() - strtotime($dueDate)) / 86400);
}

function role_label($role) {
    $labels = [
        'admin'     => 'Administrator',
        'librarian' => 'Librarian',
        'student'   => 'Student',
        'visitor'   => 'Visitor',
    ];
    return $labels[$role] ?? ucfirst($role);
}

/* ================ Server-side validation helpers ================ */
// PHP VALIDATION: never trust the browser. The JavaScript checks are only for
// convenience; these functions are the real gate.
```

```
155
156 function is_blank($value) {
157     return trim((string)$value) === '';
158 }
159
160 function valid_email($email) {
161     return filter_var($email, FILTER_VALIDATE_EMAIL) !== false;
162 }
163
164 function valid_contact($contact) {
165     return preg_match('/^[0-9+\-\s()]{6,20}$/', $contact) === 1;
166 }
167
168 function valid_date($date) {
169     $parts = explode('-', $date);
170     return count($parts) === 3
171         && checkdate((int)$parts[1], (int)$parts[2], (int)$parts[0]);
172 }
```

Group 1 — printing safely

Line	Code	What it does and why
9–10	`function esc($value) { return htmlspecialchars((string)($value ?? ''), ENT_QUOTES, 'UTF-8');`	**The single most important function in the project.** It converts characters that mean something in HTML into harmless codes: `<` becomes `<`, `"` becomes `&quot;`, and so on. Every value that came from a user is printed through this. The `?? ''` turns a missing value into an empty string, which stops PHP 8 warning about nulls. `ENT_QUOTES` also converts single quotes, which matters when a value is printed inside an HTML attribute.
14–16	`function redirect($url) { header('Location: ' . $url); exit;`	Sends the browser to another address. The `exit` is essential: `header()` only *schedules* the redirect, and without `exit` the rest of the script would keep running and could still change data or print a page.
19	`function is_post() {`	A readable way of asking "did this request arrive from a submitted form?". Used to separate "show me the form" from "here is the filled-in form".

Group 2 — CSRF protection

Cross-Site Request Forgery is an attack where a different website quietly makes **your** browser send a request to this system while you are signed in. Because your browser attaches your session cookie automatically, the server would think you meant it. The defence is a secret value that the attacker's site cannot read.

Line	Code	What it does and why
27–31	`function csrf_token() { ... return $_SESSION['csrf_token'];`	Creates a long random secret the first time it is needed and keeps it in the session for the rest of the visit. `random_bytes(32)` is a cryptographically secure generator — unlike `rand()`, its output cannot be predicted from earlier values. `bin2hex` turns the raw bytes into 64 readable characters.
35	`function csrf_field() {`	Prints the hidden input that carries the token inside a form. Every POST form in the project calls this.
40	`function csrf_url($url) {`	The same idea for links. Delete, Issue, Return and Suspend are links rather than forms, so the token is added to the address instead.
45–49	`function csrf_check() { ... die('Security check failed (invalid CSRF token). Please go back and try again.');`	The guard. It reads the token from either a form or a link, compares it with the one in the session, and stops the whole request if they differ. `hash_equals` is used rather than `==` because it always takes the same amount of time, which prevents an attacker from guessing the token one character at a time by measuring how long each attempt takes.

Group 3 — who is signed in

Line	Code	What it does and why
55	`function is_logged_in() {`	True when the session contains a user. Everything about "am I signed in?" reduces to this one question.
59	`function current_user() {`	Returns the signed-in user's details, or `null` for a guest. Views use it to print the name in the navigation bar.
63	`function current_role() {`	A shortcut for the role, returning an empty string rather than an error when nobody is signed in.
68	`function check_session_timeout() {`	Called once from `index.php` on every request. If nobody is signed in it returns immediately.
72	`if (isset($_SESSION['last_active']) && (time() - $_SESSION['last_active']) > SESSION_TIMEOUT) {`	Compares the current time with the time of the last request. If the gap is larger than thirty minutes the session is emptied, the id is regenerated, a message is stored and the user is sent to the sign-in page.
78	`$_SESSION['last_active'] = time();`	Otherwise the clock is reset. Because this runs on every request, the thirty minutes measures **idle** time, not total time — an active user is never thrown out mid-task.
82	`function require_role($role) {`	The gate in front of every dashboard. Two checks in order: not signed in at all → go to the sign-in page; signed in as the wrong role → go to your own dashboard, not to an error page. Sending people somewhere useful is friendlier than shouting at them, and it reveals nothing about what exists at the address they tried.

Group 4 — flash messages

After the code changes something it redirects. A redirect throws away all local variables, so "Book deleted." would be lost. A **flash message** is stored in the session, printed on the next page, and deleted as it is read.

Line	Code	What it does and why
95	`function set_flash($type, $message) {`	Adds a message to a list in the session. A list rather than a single value, so two messages can queue up if a request produces both.
99–102	`function get_flash() { ... return $messages;`	Takes a copy, deletes the original with `unset`, then returns the copy. The delete is what makes it "one time only" — refreshing the page will not show the message again.

Group 5 — JSON for AJAX

Line	Code	What it does and why
107–111	`function json_out($data, $code = 200) { ... exit;`	Used by every background request. Sets the HTTP status (200 for success, 403 for "not allowed"), announces that the reply is JSON rather than HTML, converts the PHP array into JSON text, and stops. The `exit` guarantees that no stray HTML is appended, which would make the reply unreadable to JavaScript.

Group 6 — dates, money and fines

Line	Code	What it does and why
116	`function money($amount) {`	Formats a number as money with the configured symbol and exactly two decimal places, plus thousands separators.
120	`function nice_date($date) {`	Turns the database format `2026-08-31` into the readable `31 Aug 2026`, and prints a dash for an empty date instead of the confusing `01 Jan 1970`.
125	`function calculate_fine($dueDate, $returnDate = null) {`	**The fine rule, in one place.** Everywhere else in the project asks this function rather than doing its own arithmetic, so changing the rule means changing one function.
129	`$end  = $returnDate ? strtotime($returnDate) : time();`	If the book has been returned, measure to the return date. If it is still out, measure to right now — which is why a student watching their own screen sees the fine grow day by day.
130	`$days = floor(($end - strtotime($dueDate)) / 86400);`	86400 is the number of seconds in a day. Subtracting two timestamps gives seconds; dividing and rounding down gives whole days. `floor` matters: half a day late is not yet a day late.
131	`return $days > 0 ? (float)($days * FINE_PER_DAY) : 0.0;`	A negative result means the book is early, and no fine is due. Returning `0.0` rather than a negative number stops a "credit" appearing anywhere in the system.
135	`function days_late($dueDate) {`	The same arithmetic without the money. The student dashboard uses the sign to choose its wording: positive prints "4 days late", negative prints "6 days left".
142	`function role_label($role) {`	Turns the stored word `admin` into the printable "Administrator". Keeping display names apart from stored values means the wording on screen can change without touching the database.

Group 7 — validation helpers

These four are used by the controllers. They are deliberately tiny, so that a validation rule reads like a sentence.

Line	Code	What it does and why
156	`function is_blank($value) {`	True when a value is empty or contains only spaces. Note the `trim`: a field containing three spaces is empty as far as a human is concerned, and this project agrees.
160	`function valid_email($email) {`	Uses PHP's built-in email validator. Writing your own email pattern is a classic beginner trap — the real rules are far more complicated than they look, and the built-in one has already solved them.
164	`function valid_contact($contact) {`	Allows digits, plus, dash, spaces and brackets, between 6 and 20 characters. Deliberately permissive, because phone number formats differ around the world.
168–171	`function valid_date($date) { ... && checkdate((int)$parts[1], (int)$parts[2], (int)$parts[0]);`	Splits `2026-02-30` into parts and asks `checkdate()` whether such a day exists. This catches the 30th of February, which looks perfectly valid to a simple text check.

6.3 index.php — the router

Purpose. The single entry point. It loads everything, applies the two universal checks, and hands the request to exactly one controller. It contains no business logic at all, and that is the point: it should be readable in under a minute.

Full listing of **index.php** (79 lines).

```php
<?php
// ================================================================
// FRONT CONTROLLER (the router)
// Every request in the whole app enters here:
//   index.php?page=<where>&action=<what>
// ================================================================

/* ---- 1. Load the app (config first: it starts the session) ---- */
require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/helpers/helpers.php';

/* ---- 2. Load the MODELS ---- */
require_once __DIR__ . '/models/user_model.php';
require_once __DIR__ . '/models/book_model.php';
require_once __DIR__ . '/models/borrow_model.php';
require_once __DIR__ . '/models/visitor_model.php';
require_once __DIR__ . '/models/log_model.php';

/* ---- 3. Load the CONTROLLERS ---- */
require_once __DIR__ . '/controllers/auth_controller.php';
require_once __DIR__ . '/controllers/admin_controller.php';
require_once __DIR__ . '/controllers/librarian_controller.php';
require_once __DIR__ . '/controllers/student_controller.php';
require_once __DIR__ . '/controllers/visitor_controller.php';
require_once __DIR__ . '/controllers/ajax_controller.php';

/* ---- 4. Sign the user out if they have been idle too long ---- */
check_session_timeout();

/* ---- 5. Decide which page was asked for ---- */
$page = $_GET['page'] ?? 'login';

switch ($page) {

    // Public pages
    case 'login':
        login_controller($conn);
        break;

    case 'register':
        register_controller($conn);
        break;

    case 'logout':
        logout_controller($conn);
        break;

    // JSON endpoints (each one checks the role itself)
    case 'ajax':
        ajax_controller($conn);
        break;

    // Private dashboards - require_role() blocks everyone else
    case 'admin':
        require_role('admin');
        admin_controller($conn);
        break;

    case 'librarian':
        require_role('librarian');
        librarian_controller($conn);
        break;

    case 'student':
        require_role('student');
        student_controller($conn);
        break;

    case 'visitor':
        require_role('visitor');
        visitor_controller($conn);
        break;

    // Anything else goes home
    default:
        redirect('index.php?page=login');
}

mysqli_close($conn);
```

Line by line

Line	Code	What it does and why
9	`require_once __DIR__ . '/config/config.php';`	Loaded first because it starts the session and opens `$conn`, both of which everything else needs. `__DIR__` is the folder this file is in, which makes the path work no matter which folder the server was started from. `require_once` refuses to load the same file twice.

Line	Code	What it does and why
10	`require_once __DIR__ . '/helpers/helpers.php';`	The toolbox. Loaded second because the models and controllers call it.
13–17	`require_once __DIR__ . '/models/user_model.php';` ... `require_once __DIR__ . '/models/log_model.php';`	All five models. Loading a model only defines its functions; nothing runs and no query is sent until a controller calls one.
20–25	`require_once __DIR__ . '/controllers/auth_controller.php';` ... `require_once __DIR__ . '/controllers/ajax_controller.php';`	All six controllers, defining one function each. Again, defining is not running.
28	`check_session_timeout();`	**Universal check 1.** Runs before any decision is made, so an expired session is dealt with once rather than in every controller.
31	`$page = $_GET['page'] ?? 'login';`	Reads the requested page from the address. `??` supplies a default, so a bare `index.php` with nothing after it lands on the sign-in screen instead of failing.
33	`switch ($page) {`	The whole router. A `switch` is used rather than a chain of `if`s because the entire list of valid pages is then visible in one glance.
36–38	`case 'login': login_controller($conn); break;`	Public pages. Anybody may reach these, signed in or not — although `login_controller` itself sends an already-signed-in user to their dashboard.
49	`case 'ajax':`	The JSON endpoints. There is no `require_role` here on purpose, because different endpoints belong to different roles; each one performs its own check inside `ajax_controller.php`.
55	`require_role('admin');`	**Universal check 2**, applied per dashboard. The role gate runs *before* the controller, so a controller function can safely assume the visitor is entitled to be there.
75–76	`default: redirect('index.php?page=login');`	Anything unrecognised — a typo, a probe from a scanner, an old bookmark — lands on the sign-in page. There is no error message, because an unknown address should not tell a stranger anything.
79	`mysqli_close($conn);`	Closes the database connection at the very end. Most controllers redirect or exit before reaching this line; PHP would close the connection anyway, but writing it makes the lifecycle explicit for a reader.

6.4 The shape of every model function

The five model files contain fifty-odd functions between them, and almost every one follows the same five steps. Learn the shape once and the rest of the folder reads itself.

The pattern, using get_user() as the example

```
17  function get_user($conn, $id) {
18      $sql  = "SELECT id, name, email, contact, username, role, status, created_at
19               FROM users WHERE id = ?";
20      $stmt = mysqli_prepare($conn, $sql);
21      mysqli_stmt_bind_param($stmt, 'i', $id);
22      mysqli_stmt_execute($stmt);
23      $row = mysqli_fetch_assoc(mysqli_stmt_get_result($stmt));
24      mysqli_stmt_close($stmt);
25      return $row;
```

Step	Code	Why
1	`$sql = "… WHERE id = ?"`	Write the query with a question mark wherever a user-supplied value will go. The value is **not** in this string.

Step	Code	Why
2	`mysqli_prepare($conn, $sql)`	Send the query shape to MySQL, which parses and plans it now, before it has seen any data.
3	`mysqli_stmt_bind_param($stmt, 'i', $id)`	Attach the value, declaring its type. `i` = integer, `s` = string, `d` = decimal. MySQL now treats it as data and nothing else.
4	`mysqli_stmt_execute($stmt)`	Run it.
5	`fetch_assoc` / `fetch_all`, then `close`	Collect the rows as PHP arrays and release the statement.

NOTE

This is why SQL injection cannot happen here. In a query built by gluing text together, a value such as `'; DROP TABLE books;--` becomes part of the command. With a prepared statement the command was already fixed in step 2, before the value existed. The system was tested with exactly that input: it was stored as a book title and the table survived.

Two shorthand rules used throughout the models:

- `mysqli_fetch_assoc()` returns **one row** as an array keyed by column name, or `null` if there was none.
- `mysqli_fetch_all(…, MYSQLI_ASSOC)` returns **every row** as an array of such arrays, or an empty array.

A handful of queries in the project use `mysqli_query()` directly instead of preparing. Look closely and you will see that every one of them contains no user input at all — for example `SELECT id, name … FROM users ORDER BY id DESC`. Where there is nothing to inject, there is nothing to protect against.

6.5 models/user_model.php

Purpose. Every question and every change that concerns a person: finding them to sign in, listing them, searching them, creating, updating, deleting, suspending, promoting, and counting them for the statistics panel.

Full listing of **models/user_model.php** (168 lines).

```php
<?php
// ================================================================
// MODEL: users  (admin, librarian, student, visitor all live here)
// Every query uses a prepared statement -> no SQL injection.
// ================================================================

function find_user_by_username($conn, $username) {
    $sql  = "SELECT * FROM users WHERE username = ?";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 's', $username);
    mysqli_stmt_execute($stmt);
    $row = mysqli_fetch_assoc(mysqli_stmt_get_result($stmt));
    mysqli_stmt_close($stmt);
    return $row;
}

function get_user($conn, $id) {
    $sql  = "SELECT id, name, email, contact, username, role, status, created_at
             FROM users WHERE id = ?";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'i', $id);
    mysqli_stmt_execute($stmt);
    $row = mysqli_fetch_assoc(mysqli_stmt_get_result($stmt));
    mysqli_stmt_close($stmt);
    return $row;
}

// $role = '' means "every role".
function get_users($conn, $role = '') {
    if ($role === '') {
        $sql  = "SELECT id, name, email, contact, username, role, status, created_at
                 FROM users ORDER BY id DESC";
        $stmt = mysqli_prepare($conn, $sql);
    } else {
        $sql  = "SELECT id, name, email, contact, username, role, status, created_at
                 FROM users WHERE role = ? ORDER BY id DESC";
        $stmt = mysqli_prepare($conn, $sql);
        mysqli_stmt_bind_param($stmt, 's', $role);
    }
    mysqli_stmt_execute($stmt);
    $rows = mysqli_fetch_all(mysqli_stmt_get_result($stmt), MYSQLI_ASSOC);
    mysqli_stmt_close($stmt);
    return $rows;
}
```

```php

function search_users($conn, $term, $role = '') {
    $like = '%' . $term . '%';
    if ($role === '') {
        $sql = "SELECT id, name, email, contact, username, role, status, created_at
                FROM users
                WHERE name LIKE ? OR username LIKE ? OR email LIKE ? OR contact LIKE ?
                ORDER BY id DESC";
        $stmt = mysqli_prepare($conn, $sql);
        mysqli_stmt_bind_param($stmt, 'ssss', $like, $like, $like, $like);
    } else {
        $sql = "SELECT id, name, email, contact, username, role, status, created_at
                FROM users
                WHERE role = ?
                  AND (name LIKE ? OR username LIKE ? OR email LIKE ? OR contact LIKE ?)
                ORDER BY id DESC";
        $stmt = mysqli_prepare($conn, $sql);
        mysqli_stmt_bind_param($stmt, 'sssss', $role, $like, $like, $like, $like);
    }
    mysqli_stmt_execute($stmt);
    $rows = mysqli_fetch_all(mysqli_stmt_get_result($stmt), MYSQLI_ASSOC);
    mysqli_stmt_close($stmt);
    return $rows;
}

function username_exists($conn, $username, $excludeId = 0) {
    $sql  = "SELECT id FROM users WHERE username = ? AND id != ?";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'si', $username, $excludeId);
    mysqli_stmt_execute($stmt);
    mysqli_stmt_store_result($stmt);
    $exists = mysqli_stmt_num_rows($stmt) > 0;
    mysqli_stmt_close($stmt);
    return $exists;
}

function create_user($conn, $name, $email, $contact, $username, $password, $role) {
    // SECURITY: passwords are hashed, never stored as plain text.
    $hash = password_hash($password, PASSWORD_DEFAULT);
    $sql  = "INSERT INTO users (name, email, contact, username, password, role)
             VALUES (?, ?, ?, ?, ?, ?)";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'ssssss', $name, $email, $contact, $username, $hash, $role);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

function update_user($conn, $id, $name, $email, $contact, $username, $role, $status) {
    $sql  = "UPDATE users
             SET name = ?, email = ?, contact = ?, username = ?, role = ?, status = ?
             WHERE id = ?";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'ssssssi', $name, $email, $contact, $username, $role, $status, $id);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

function update_password($conn, $id, $password) {
    $hash = password_hash($password, PASSWORD_DEFAULT);
    $stmt = mysqli_prepare($conn, "UPDATE users SET password = ? WHERE id = ?");
    mysqli_stmt_bind_param($stmt, 'si', $hash, $id);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

function delete_user($conn, $id) {
    $stmt = mysqli_prepare($conn, "DELETE FROM users WHERE id = ?");
    mysqli_stmt_bind_param($stmt, 'i', $id);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

// ADMIN FEATURE: suspend or re-activate an account without deleting it.
function set_user_status($conn, $id, $status) {
    $stmt = mysqli_prepare($conn, "UPDATE users SET status = ? WHERE id = ?");
    mysqli_stmt_bind_param($stmt, 'si', $status, $id);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

function set_user_role($conn, $id, $role) {
    $stmt = mysqli_prepare($conn, "UPDATE users SET role = ? WHERE id = ?");
    mysqli_stmt_bind_param($stmt, 'si', $role, $id);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

// ADMIN FEATURE: numbers for the live statistics panel.
function get_system_stats($conn) {
    $stats = [
        'admins' => 0, 'librarians' => 0, 'students' => 0, 'visitors' => 0,
        'suspended' => 0, 'books' => 0, 'copies' => 0,
        'pending_requests' => 0, 'issued' => 0, 'passes' => 0,
    ];
```

```
145
146     $res = mysqli_query($conn, "SELECT role, COUNT(*) AS total FROM users GROUP BY role");
147     while ($row = mysqli_fetch_assoc($res)) {
148         if ($row['role'] === 'admin')     $stats['admins']     = (int)$row['total'];
149         if ($row['role'] === 'librarian') $stats['librarians'] = (int)$row['total'];
150         if ($row['role'] === 'student')   $stats['students']   = (int)$row['total'];
151         if ($row['role'] === 'visitor')   $stats['visitors']   = (int)$row['total'];
152     }
153
154     $one = function ($conn, $sql) {
155         $res = mysqli_query($conn, $sql);
156         $row = mysqli_fetch_row($res);
157         return (int)($row[0] ?? 0);
158     };
159
160     $stats['suspended']        = $one($conn, "SELECT COUNT(*) FROM users WHERE status = 'suspended'");
161     $stats['books']            = $one($conn, "SELECT COUNT(*) FROM books");
162     $stats['copies']           = $one($conn, "SELECT COALESCE(SUM(quantity),0) FROM books");
163     $stats['pending_requests'] = $one($conn, "SELECT COUNT(*) FROM borrow_requests WHERE status = 'pending'");
164     $stats['issued']           = $one($conn, "SELECT COUNT(*) FROM borrow_requests WHERE status = 'issued'");
165     $stats['passes']           = $one($conn, "SELECT COUNT(*) FROM visit_passes WHERE status = 'booked'");
166
167     return $stats;
168 }
```

Line by line

Line	Code	What it does and why
7	`function find_user_by_username($conn, $username) {`	Used only by sign-in. It is the one function that selects *, because sign-in needs the password hash, which every other function deliberately leaves behind.
8	`$sql  = "SELECT * FROM users WHERE username = ?";`	The username arrives from a form, so it goes in as a bound parameter.
17–19	`function get_user($conn, $id) { $sql  = "SELECT id, name, email, contact, username, role, status, created_at FROM users WHERE id = ?";`	Fetches one person for the edit form. The column list is written out rather than using *, so the password hash is never even loaded into memory when it is not needed.
29	`function get_users($conn, $role = '') {`	Lists people. The default empty role means "everybody", which is what the admin sees before choosing a filter.
30	`if ($role === '') {`	Two different queries rather than one clever query with a condition built into the text. Slightly longer, far easier to read, and keeps the parameter binding simple.
46	`function search_users($conn, $term, $role = '') {`	The live search behind the admin's search box.
47	`$like = '%' . $term . '%';`	The percent signs are SQL wildcards meaning "any characters here", so typing `ami` finds "Jasmine". The wildcards are added in PHP and the whole thing is still bound as a parameter — the user's text never touches the SQL.
51	`WHERE name LIKE ? OR username LIKE ? OR email LIKE ? OR contact LIKE ?`	One typed word is compared against four columns, which is why the search box can honestly say "search by name, username, email or phone". The same `$like` value is bound to all four question marks.
70	`function username_exists($conn, $username, $excludeId = 0) {`	Answers "is this username taken?" The `$excludeId` exists for editing: when a user keeps their own username, that row must not count as a clash. Passing 0 means "exclude nothing", because no row ever has id 0.
75–76	`mysqli_stmt_store_result($stmt); $exists = mysqli_stmt_num_rows($stmt) > 0;`	Pulls the result into PHP so the rows can be counted. Only the count matters here, never the contents.
81	`function create_user($conn, $name, $email, $contact, $username, $password, $role) {`	Adds a person. Notice the parameter list spells out each field rather than accepting one big array — a reader can see exactly what a user is made of.
83	`$hash = password_hash($password, PASSWORD_DEFAULT);`	**Security.** The plain password is turned into a long hash and the original is discarded. Hashing is one-way: nobody, including the administrator and including whoever steals the database, can turn the hash back into

Line	Code	What it does and why
		the password. Verifying works by hashing the typed password and comparing.
87	`mysqli_stmt_bind_param($stmt, 'ssssss', $name, $email, $contact, $username, $hash, $role);`	Six values, all strings, hence six `s` characters. The count and order of these letters must match the question marks exactly — the most common mistake in mysqli code.
93	`function update_user($conn, $id, $name, $email, $contact, $username, $role, $status) {`	Changes the details of an existing person. Deliberately does **not** touch the password, so an admin editing a phone number cannot accidentally wipe someone's login.
98	`mysqli_stmt_bind_param($stmt, 'sssssssi', $name, $email, $contact, $username, $role, $status, $id);`	Six strings and one integer: the six text fields, then the id at the end for the `WHERE` clause. The id is bound as `i`, which is a second layer of protection on top of the `(int)` cast in the controller.
104	`function update_password($conn, $id, $password) {`	A separate function, called only when the admin actually typed a new password. Keeping it apart is what makes "leave blank to keep the current password" possible.
113	`function delete_user($conn, $id) {`	Removes the row. The foreign keys in the schema take care of the person's requests, passes and applications automatically.
122	`function set_user_status($conn, $id, $status) {`	**Admin feature.** Flips one account between active and suspended. Suspending is almost always better than deleting: the person cannot sign in, but their borrowing history stays intact for the records.
130	`function set_user_role($conn, $id, $role) {`	Changes somebody's role. Used in exactly one place — when the admin approves a membership application, turning a visitor into a student.
139	`function get_system_stats($conn) {`	**Admin feature.** Collects the ten numbers shown on the statistics panel, which the browser refreshes every fifteen seconds.
140	`$stats = [`	Every key is given a starting value of zero first. This matters: if a role has no members at all it is missing from the query result, and without these defaults the view would fail when it tried to print it.
146	`$res = mysqli_query($conn, "SELECT role, COUNT(*) AS total FROM users GROUP BY role");`	`GROUP BY` collapses the table into one row per role with a count beside it — four counts from one trip to the database instead of four separate queries.
154	`$one = function ($conn, $sql) {`	A small anonymous function stored in a variable, used to run the six single-number queries without repeating the same three lines six times.
162	`$stats['copies'] = $one($conn, "SELECT COALESCE(SUM(quantity),0) FROM books");`	`COALESCE(SUM(quantity),0)` guards against an empty catalogue: `SUM` of no rows is null, not zero, and null would print as an empty box on the dashboard.

6.6 models/book_model.php

Purpose. The catalogue: create, read, update, delete, search, plus two stock-related functions that the librarian's and student's screens depend on.

Full listing of **models/book_model.php** (99 lines).

```php
<?php
// ================================================================
// MODEL: books  (created and maintained by librarians)
// ================================================================

function get_books($conn) {
    $sql = "SELECT id, title, author, category, isbn, quantity, price
            FROM books ORDER BY id DESC";
    $res = mysqli_query($conn, $sql);
    return mysqli_fetch_all($res, MYSQLI_ASSOC);
}

```

```php
function get_book($conn, $id) {
    $sql  = "SELECT id, title, author, category, isbn, quantity, price
             FROM books WHERE id = ?";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'i', $id);
    mysqli_stmt_execute($stmt);
    $row = mysqli_fetch_assoc(mysqli_stmt_get_result($stmt));
    mysqli_stmt_close($stmt);
    return $row;
}

function search_books($conn, $term) {
    $like = '%' . $term . '%';
    $sql  = "SELECT id, title, author, category, isbn, quantity, price
             FROM books
             WHERE title LIKE ? OR author LIKE ? OR category LIKE ? OR isbn LIKE ?
             ORDER BY id DESC";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'ssss', $like, $like, $like, $like);
    mysqli_stmt_execute($stmt);
    $rows = mysqli_fetch_all(mysqli_stmt_get_result($stmt), MYSQLI_ASSOC);
    mysqli_stmt_close($stmt);
    return $rows;
}

function add_book($conn, $title, $author, $category, $isbn, $quantity, $price, $addedBy) {
    $sql  = "INSERT INTO books (title, author, category, isbn, quantity, price, added_by)
             VALUES (?, ?, ?, ?, ?, ?, ?)";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'ssssidi',
        $title, $author, $category, $isbn, $quantity, $price, $addedBy);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

function update_book($conn, $id, $title, $author, $category, $isbn, $quantity, $price) {
    $sql  = "UPDATE books
             SET title = ?, author = ?, category = ?, isbn = ?, quantity = ?, price = ?
             WHERE id = ?";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'ssssidi',
        $title, $author, $category, $isbn, $quantity, $price, $id);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

function delete_book($conn, $id) {
    $stmt = mysqli_prepare($conn, "DELETE FROM books WHERE id = ?");
    mysqli_stmt_bind_param($stmt, 'i', $id);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

// Used when a book is issued (-1 copy) or returned (+1 copy).
function change_book_stock($conn, $id, $amount) {
    $stmt = mysqli_prepare($conn,
        "UPDATE books SET quantity = quantity + ? WHERE id = ? AND quantity + ? >= 0");
    mysqli_stmt_bind_param($stmt, 'iii', $amount, $id, $amount);
    $ok = mysqli_stmt_execute($stmt);
    $changed = mysqli_stmt_affected_rows($stmt) > 0;
    mysqli_stmt_close($stmt);
    return $ok && $changed;
}

// LIBRARIAN FEATURE: books that are running out.
function get_low_stock_books($conn) {
    $limit = LOW_STOCK;
    $sql   = "SELECT id, title, author, category, isbn, quantity, price
              FROM books WHERE quantity <= ? ORDER BY quantity ASC";
    $stmt  = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'i', $limit);
    mysqli_stmt_execute($stmt);
    $rows = mysqli_fetch_all(mysqli_stmt_get_result($stmt), MYSQLI_ASSOC);
    mysqli_stmt_close($stmt);
    return $rows;
}

// Books a student is allowed to request (at least one copy on the shelf).
function get_available_books($conn) {
    $sql = "SELECT id, title, author, category, isbn, quantity, price
            FROM books WHERE quantity > 0 ORDER BY title ASC";
    $res = mysqli_query($conn, $sql);
    return mysqli_fetch_all($res, MYSQLI_ASSOC);
}
```

The interesting parts

The first six functions follow the five-step pattern from section 6.4 exactly, so only the parts that differ are explained here.

Line	Code	What it does and why
6	`function get_books($conn) {`	No user input at all, so a plain `mysqli_query` is used. `ORDER BY id DESC` puts the newest book at the top, which is what a librarian who just added one expects to see.
28	`WHERE title LIKE ? OR author LIKE ? OR category LIKE ? OR isbn LIKE ?`	The catalogue search covers four columns, so a librarian can find a book by any of the things they might remember about it.
42	`mysqli_stmt_bind_param($stmt, 'ssssidi',`	Seven values with mixed types: four strings, one integer (quantity), one decimal (price), one integer (the librarian id). Getting this string wrong is the classic mysqli bug — count the question marks, then count the letters.
70	`function change_book_stock($conn, $id, $amount) {`	**The heart of the issue desk.** Called with -1 when a book goes out and +1 when it comes back.
72	`"UPDATE books SET quantity = quantity + ? WHERE id = ? AND quantity + ? >= 0");`	Two things worth studying. First, `quantity = quantity + ?` lets the **database** do the arithmetic, so two librarians clicking at the same instant cannot both read "2" and both write "1". Second, `AND quantity + ? >= 0` makes it impossible to issue a book that is not there — the condition simply fails and no row changes.
75	`$changed = mysqli_stmt_affected_rows($stmt) > 0;`	The query "succeeding" is not the same as it "doing something". A query that matched no rows succeeds happily. This line asks the more useful question: did a row actually change? The controller relies on the answer to decide whether the issue went through.
81	`function get_low_stock_books($conn) {`	**Librarian feature.** Everything at or below the `LOW_STOCK` threshold, sorted with the emptiest shelf first so the most urgent title is at the top.
82	`$limit = LOW_STOCK;`	A constant cannot be passed straight into `bind_param`, which needs a variable it can reference. Copying it into a variable first is the standard way around that.
94	`function get_available_books($conn) {`	**Student feature.** Only titles with at least one copy on the shelf, sorted alphabetically. This is the list that fills the "Request a book" drop-down, so a student can never even select something that is entirely out on loan.

6.7 models/borrow_model.php

Purpose. The whole borrowing lifecycle. It is the largest model because one record passes through several hands: a student creates and edits it, a librarian issues it, and later returns it.

The file is split into two halves by comment banners: the student side, then the librarian side.

Full listing of **models/borrow_model.php** (216 lines).

```
<?php
// ================================================================
// MODEL: borrow_requests
// Students create/edit/cancel them; librarians issue and return them.
// ================================================================

/* ---------------- Student side ---------------- */

function add_request($conn, $studentId, $bookId, $pickupDate, $notes) {
    $today = date('Y-m-d');
    $sql   = "INSERT INTO borrow_requests (student_id, book_id, pickup_date, notes, request_date)
              VALUES (?, ?, ?, ?, ?)";
    $stmt  = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'iisss', $studentId, $bookId, $pickupDate, $notes, $today);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

function get_student_requests($conn, $studentId) {
    $sql  = "SELECT r.*, b.title, b.author
```

```
            FROM borrow_requests r
            JOIN books b ON b.id = r.book_id
            WHERE r.student_id = ?
            ORDER BY r.id DESC";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'i', $studentId);
    mysqli_stmt_execute($stmt);
    $rows = mysqli_fetch_all(mysqli_stmt_get_result($stmt), MYSQLI_ASSOC);
    mysqli_stmt_close($stmt);
    return $rows;
}

function search_student_requests($conn, $studentId, $term) {
    $like = '%' . $term . '%';
    $sql  = "SELECT r.*, b.title, b.author
            FROM borrow_requests r
            JOIN books b ON b.id = r.book_id
            WHERE r.student_id = ?
              AND (b.title LIKE ? OR b.author LIKE ? OR r.status LIKE ? OR r.notes LIKE ?)
            ORDER BY r.id DESC";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'issss', $studentId, $like, $like, $like, $like);
    mysqli_stmt_execute($stmt);
    $rows = mysqli_fetch_all(mysqli_stmt_get_result($stmt), MYSQLI_ASSOC);
    mysqli_stmt_close($stmt);
    return $rows;
}

// The $studentId argument makes sure one student cannot open another
// student's request by typing a different id in the address bar.
function get_request($conn, $id, $studentId = 0) {
    if ($studentId > 0) {
        $sql  = "SELECT r.*, b.title, b.author
                 FROM borrow_requests r
                 JOIN books b ON b.id = r.book_id
                 WHERE r.id = ? AND r.student_id = ?";
        $stmt = mysqli_prepare($conn, $sql);
        mysqli_stmt_bind_param($stmt, 'ii', $id, $studentId);
    } else {
        $sql  = "SELECT r.*, b.title, b.author
                 FROM borrow_requests r
                 JOIN books b ON b.id = r.book_id
                 WHERE r.id = ?";
        $stmt = mysqli_prepare($conn, $sql);
        mysqli_stmt_bind_param($stmt, 'i', $id);
    }
    mysqli_stmt_execute($stmt);
    $row = mysqli_fetch_assoc(mysqli_stmt_get_result($stmt));
    mysqli_stmt_close($stmt);
    return $row;
}

// A student may only edit a request that is still pending.
function update_request($conn, $id, $studentId, $bookId, $pickupDate, $notes) {
    $sql  = "UPDATE borrow_requests
             SET book_id = ?, pickup_date = ?, notes = ?
             WHERE id = ? AND student_id = ? AND status = 'pending'";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'issii', $bookId, $pickupDate, $notes, $id, $studentId);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

function delete_request($conn, $id, $studentId) {
    $sql  = "DELETE FROM borrow_requests
             WHERE id = ? AND student_id = ? AND status = 'pending'";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'ii', $id, $studentId);
    mysqli_stmt_execute($stmt);
    $deleted = mysqli_stmt_affected_rows($stmt) > 0;
    mysqli_stmt_close($stmt);
    return $deleted;
}

// Stops a student from requesting the same book twice at the same time.
function has_open_request($conn, $studentId, $bookId, $excludeId = 0) {
    $sql  = "SELECT id FROM borrow_requests
             WHERE student_id = ? AND book_id = ?
               AND status IN ('pending','issued') AND id != ?";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'iii', $studentId, $bookId, $excludeId);
    mysqli_stmt_execute($stmt);
    mysqli_stmt_store_result($stmt);
    $exists = mysqli_stmt_num_rows($stmt) > 0;
    mysqli_stmt_close($stmt);
    return $exists;
}

// STUDENT FEATURE: numbers for the due-date and fine tracker.
function get_student_stats($conn, $studentId) {
    $sql = "SELECT
              SUM(status = 'pending')  AS pending,
              SUM(status = 'issued')   AS issued,
              SUM(status = 'returned') AS returned,
              SUM(status = 'issued' AND due_date < CURDATE()) AS overdue,
              COALESCE(SUM(fine), 0)   AS paid_fine
            FROM borrow_requests WHERE student_id = ?";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'i', $studentId);
```

```
    mysqli_stmt_execute($stmt);
    $row = mysqli_fetch_assoc(mysqli_stmt_get_result($stmt));
    mysqli_stmt_close($stmt);
    return [
        'pending'   => (int)($row['pending']   ?? 0),
        'issued'    => (int)($row['issued']    ?? 0),
        'returned'  => (int)($row['returned']  ?? 0),
        'overdue'   => (int)($row['overdue']   ?? 0),
        'paid_fine' => (float)($row['paid_fine'] ?? 0),
    ];
}

/* ---------------- Librarian side ---------------- */

function get_all_requests($conn) {
    $sql = "SELECT r.*, b.title, b.author, u.name AS student_name, u.username AS student_username
            FROM borrow_requests r
            JOIN books b ON b.id = r.book_id
            JOIN users u ON u.id = r.student_id
            ORDER BY FIELD(r.status,'pending','issued','returned','rejected'), r.id DESC";
    $res = mysqli_query($conn, $sql);
    return mysqli_fetch_all($res, MYSQLI_ASSOC);
}

function search_all_requests($conn, $term) {
    $like = '%'. $term . '%';
    $sql  = "SELECT r.*, b.title, b.author, u.name AS student_name, u.username AS student_username
             FROM borrow_requests r
             JOIN books b ON b.id = r.book_id
             JOIN users u ON u.id = r.student_id
             WHERE b.title LIKE ? OR u.name LIKE ? OR u.username LIKE ? OR r.status LIKE ?
             ORDER BY FIELD(r.status,'pending','issued','returned','rejected'), r.id DESC";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'ssss', $like, $like, $like, $like);
    mysqli_stmt_execute($stmt);
    $rows = mysqli_fetch_all(mysqli_stmt_get_result($stmt), MYSQLI_ASSOC);
    mysqli_stmt_close($stmt);
    return $rows;
}

// LIBRARIAN FEATURE (issue desk): hand the book over and start the clock.
function issue_request($conn, $id) {
    $issue = date('Y-m-d');
    $due   = date('Y-m-d', strtotime('+' . LOAN_DAYS . ' days'));
    $sql   = "UPDATE borrow_requests
              SET status = 'issued', issue_date = ?, due_date = ?
              WHERE id = ? AND status = 'pending'";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'ssi', $issue, $due, $id);
    mysqli_stmt_execute($stmt);
    $done = mysqli_stmt_affected_rows($stmt) > 0;
    mysqli_stmt_close($stmt);
    return $done;
}

// LIBRARIAN FEATURE (issue desk): take the book back and store the fine.
function return_request($conn, $id, $fine) {
    $today = date('Y-m-d');
    $sql   = "UPDATE borrow_requests
              SET status = 'returned', return_date = ?, fine = ?
              WHERE id = ? AND status = 'issued'";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'sdi', $today, $fine, $id);
    mysqli_stmt_execute($stmt);
    $done = mysqli_stmt_affected_rows($stmt) > 0;
    mysqli_stmt_close($stmt);
    return $done;
}

function reject_request($conn, $id) {
    $sql  = "UPDATE borrow_requests SET status = 'rejected'
             WHERE id = ? AND status = 'pending'";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'i', $id);
    mysqli_stmt_execute($stmt);
    $done = mysqli_stmt_affected_rows($stmt) > 0;
    mysqli_stmt_close($stmt);
    return $done;
}

function get_librarian_stats($conn) {
    $one = function ($conn, $sql) {
        $res = mysqli_query($conn, $sql);
        $row = mysqli_fetch_row($res);
        return (int)($row[0] ?? 0);
    };
    return [
        'books'     => $one($conn, "SELECT COUNT(*) FROM books"),
        'copies'    => $one($conn, "SELECT COALESCE(SUM(quantity),0) FROM books"),
        'low_stock' => $one($conn, "SELECT COUNT(*) FROM books WHERE quantity <= " . (int)LOW_STOCK),
        'pending'   => $one($conn, "SELECT COUNT(*) FROM borrow_requests WHERE status = 'pending'"),
        'issued'    => $one($conn, "SELECT COUNT(*) FROM borrow_requests WHERE status = 'issued'"),
        'overdue'   => $one($conn, "SELECT COUNT(*) FROM borrow_requests WHERE status = 'issued' AND due_date <
CURDATE()"),
    ];
}
```

The student half

Line	Code	What it does and why
9	`function add_request($conn, $studentId, $bookId, $pickupDate, $notes) {`	Creates a request. Only four values come from the browser; the status defaults to `pending` in the schema and the request date is set here in PHP.
10	`$today = date('Y-m-d');`	The date is taken from the **server**, never from the form. A date the user can supply is a date the user can lie about.
20	`function get_student_requests($conn, $studentId) {`	One student's own requests.
23	`JOIN books b ON b.id = r.book_id`	A `JOIN` stitches two tables together for one query. Without it the screen would show "book 14"; with it, the title and author come along. This is exactly why the request table stores only the book's id — the details live in one place and are borrowed when needed.
24	`WHERE r.student_id = ?`	**Security.** Every student query is filtered by the id held in the session, not by anything from the address bar. This is what stops one student reading another's borrowing history.
52	`function get_request($conn, $id, $studentId = 0) {`	Fetches one request. The optional second id is the access check: a student passes their own id and can then only ever load their own row, while the librarian passes nothing and can load any row.
75	`function update_request($conn, $id, $studentId, $bookId, $pickupDate, $notes) {`	Edits a request.
78	`WHERE id = ? AND student_id = ? AND status = 'pending'";`	**Three conditions carrying three separate rules**: the right record, belonging to this student, and still waiting. Putting the rules in the SQL means that even if a controller check were ever removed by mistake, the database would still refuse to change an issued book.
86	`function delete_request($conn, $id, $studentId) {`	Cancels a request, with the same ownership and status conditions.
98	`function has_open_request($conn, $studentId, $bookId, $excludeId = 0) {`	Stops a student queueing for the same title twice. "Open" means pending or issued; a returned or rejected request does not block a fresh one.
101	`AND status IN ('pending','issued') AND id != ?";`	`IN` is a tidy way of saying "either of these". The `id != ?` exclusion is needed while editing, so a request does not detect itself as a clash.
112	`function get_student_stats($conn, $studentId) {`	**Student feature.** The five numbers along the top of the student dashboard, gathered in a single query.
114	`SUM(status = 'pending')  AS pending,`	A neat trick: in MySQL a comparison is 1 when true and 0 when false, so summing it counts the matches. Four counters and a total therefore come from one pass over the table.
117	`SUM(status = 'issued' AND due_date < CURDATE()) AS overdue,`	Counts books that are out **and** past their due date. `CURDATE()` is the database's own today, so the answer does not depend on the clock of the machine running the browser.

The librarian half

Line	Code	What it does and why
136	`function get_all_requests($conn) {`	Every request from every student, for the issue desk.
140	`JOIN users u ON u.id = r.student_id`	A second join, this time to fetch the student's name and username so the librarian sees a person rather than a number.

Line	Code	What it does and why
141	`ORDER BY FIELD(r.status,'pending','issued','returned','rejected'), r.id DESC";`	A custom sort order. `FIELD()` returns the position of a value in a list, so requests that need action float to the top and finished ones sink. Plain alphabetical order would put "issued" above "pending", which is exactly backwards for the person doing the work.
163	`function issue_request($conn, $id) {`	**Librarian feature.** Hands the book over and starts the clock.
165	`$due  = date('Y-m-d', strtotime('+' . LOAN_DAYS . ' days'));`	The due date is calculated from the constant, so changing `LOAN_DAYS` in the config changes the loan period across the whole system.
168	`WHERE id = ? AND status = 'pending'";`	Guarantees a book cannot be issued twice. If the row is not pending, nothing changes and the function reports failure.
172	`$done = mysqli_stmt_affected_rows($stmt) > 0;`	Again, "did a row really change?" rather than "did the query run?". The controller uses this to decide whether to put the copy back on the shelf.
178	`function return_request($conn, $id, $fine) {`	Takes the book back and freezes the fine. The amount is calculated in the controller by `calculate_fine()` and passed in, so the model stays free of business rules.
184	`mysqli_stmt_bind_param($stmt, 'sdi', $today, $fine, $id);`	`d` is the type letter for a decimal number, used here for money.
191	`function reject_request($conn, $id) {`	Turns a request down without ever issuing it, so the shelf count is untouched.
202	`function get_librarian_stats($conn) {`	**Librarian feature.** The six numbers along the top of the librarian dashboard, including how many titles are low on stock and how many books are overdue right now.

6.8 models/visitor_model.php

Purpose. Everything a visitor owns: day passes, one membership application, and the suggestion box. Three related areas are kept in one file because none of them is big enough to deserve its own.

Full listing of **models/visitor_model.php** (191 lines).

```php
<?php
// ================================================================
// MODEL: visit_passes + membership_applications + book_suggestions
// Everything a visitor owns.
// ================================================================

/* ---------------- Visit passes (the visitor's CRUD table) ---------------- */

// Builds a short human-readable pass code, e.g. LIB-4821-K7.
function make_pass_code() {
    $letters = strtoupper(substr(str_shuffle('ABCDEFGHJKLMNPQRSTUVWXYZ'), 0, 2));
    return 'LIB-' . random_int(1000, 9999) . '-' . $letters;
}

function add_pass($conn, $visitorId, $visitDate, $purpose, $guests) {
    $code = make_pass_code();
    $sql  = "INSERT INTO visit_passes (visitor_id, pass_code, visit_date, purpose, guests)
             VALUES (?, ?, ?, ?, ?)";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'isssi', $visitorId, $code, $visitDate, $purpose, $guests);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

function get_passes($conn, $visitorId) {
```

```php
    $sql  = "SELECT * FROM visit_passes WHERE visitor_id = ? ORDER BY visit_date DESC, id DESC";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'i', $visitorId);
    mysqli_stmt_execute($stmt);
    $rows = mysqli_fetch_all(mysqli_stmt_get_result($stmt), MYSQLI_ASSOC);
    mysqli_stmt_close($stmt);
    return $rows;
}

function search_passes($conn, $visitorId, $term) {
    $like = '%' . $term . '%';
    $sql  = "SELECT * FROM visit_passes
             WHERE visitor_id = ?
               AND (purpose LIKE ? OR pass_code LIKE ? OR visit_date LIKE ? OR status LIKE ?)
             ORDER BY visit_date DESC, id DESC";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'issss', $visitorId, $like, $like, $like, $like);
    mysqli_stmt_execute($stmt);
    $rows = mysqli_fetch_all(mysqli_stmt_get_result($stmt), MYSQLI_ASSOC);
    mysqli_stmt_close($stmt);
    return $rows;
}

function get_pass($conn, $id, $visitorId) {
    $sql  = "SELECT * FROM visit_passes WHERE id = ? AND visitor_id = ?";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'ii', $id, $visitorId);
    mysqli_stmt_execute($stmt);
    $row = mysqli_fetch_assoc(mysqli_stmt_get_result($stmt));
    mysqli_stmt_close($stmt);
    return $row;
}

function update_pass($conn, $id, $visitorId, $visitDate, $purpose, $guests, $status) {
    $sql  = "UPDATE visit_passes
             SET visit_date = ?, purpose = ?, guests = ?, status = ?
             WHERE id = ? AND visitor_id = ?";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'ssisii', $visitDate, $purpose, $guests, $status, $id, $visitorId);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

function delete_pass($conn, $id, $visitorId) {
    $stmt = mysqli_prepare($conn, "DELETE FROM visit_passes WHERE id = ? AND visitor_id = ?");
    mysqli_stmt_bind_param($stmt, 'ii', $id, $visitorId);
    mysqli_stmt_execute($stmt);
    $deleted = mysqli_stmt_affected_rows($stmt) > 0;
    mysqli_stmt_close($stmt);
    return $deleted;
}

// Only one booking per calendar day per visitor.
function pass_exists_on_date($conn, $visitorId, $visitDate, $excludeId = 0) {
    $sql  = "SELECT id FROM visit_passes
             WHERE visitor_id = ? AND visit_date = ? AND status = 'booked' AND id != ?";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'isi', $visitorId, $visitDate, $excludeId);
    mysqli_stmt_execute($stmt);
    mysqli_stmt_store_result($stmt);
    $exists = mysqli_stmt_num_rows($stmt) > 0;
    mysqli_stmt_close($stmt);
    return $exists;
}

/* ---------------- Membership application (visitor feature) ---------------- */

function get_application($conn, $visitorId) {
    $sql  = "SELECT * FROM membership_applications
             WHERE visitor_id = ? ORDER BY id DESC LIMIT 1";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'i', $visitorId);
    mysqli_stmt_execute($stmt);
    $row = mysqli_fetch_assoc(mysqli_stmt_get_result($stmt));
    mysqli_stmt_close($stmt);
    return $row;
}

function add_application($conn, $visitorId, $reason) {
    $sql  = "INSERT INTO membership_applications (visitor_id, reason) VALUES (?, ?)";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'is', $visitorId, $reason);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

// Admin reads this list on the dashboard.
function get_pending_applications($conn) {
    $sql = "SELECT a.*, u.name, u.username, u.email
            FROM membership_applications a
            JOIN users u ON u.id = a.visitor_id
            WHERE a.status = 'pending'
            ORDER BY a.id DESC";
    $res = mysqli_query($conn, $sql);
    return mysqli_fetch_all($res, MYSQLI_ASSOC);
}

function decide_application($conn, $id, $decision) {
```

```
    $sql  = "UPDATE membership_applications SET status = ? WHERE id = ? AND status = 'pending'";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'si', $decision, $id);
    mysqli_stmt_execute($stmt);
    $done = mysqli_stmt_affected_rows($stmt) > 0;
    mysqli_stmt_close($stmt);
    return $done;
}

function get_application_by_id($conn, $id) {
    $stmt = mysqli_prepare($conn, "SELECT * FROM membership_applications WHERE id = ?");
    mysqli_stmt_bind_param($stmt, 'i', $id);
    mysqli_stmt_execute($stmt);
    $row = mysqli_fetch_assoc(mysqli_stmt_get_result($stmt));
    mysqli_stmt_close($stmt);
    return $row;
}

/* ---------------- Book suggestion box (visitor feature) ---------------- */

function add_suggestion($conn, $visitorId, $title, $author, $reason) {
    $sql  = "INSERT INTO book_suggestions (visitor_id, title, author, reason)
             VALUES (?, ?, ?, ?)";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'isss', $visitorId, $title, $author, $reason);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

function get_suggestions($conn, $visitorId) {
    $sql  = "SELECT * FROM book_suggestions WHERE visitor_id = ? ORDER BY id DESC";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'i', $visitorId);
    mysqli_stmt_execute($stmt);
    $rows = mysqli_fetch_all(mysqli_stmt_get_result($stmt), MYSQLI_ASSOC);
    mysqli_stmt_close($stmt);
    return $rows;
}

function delete_suggestion($conn, $id, $visitorId) {
    $stmt = mysqli_prepare($conn, "DELETE FROM book_suggestions WHERE id = ? AND visitor_id = ?");
    mysqli_stmt_bind_param($stmt, 'ii', $id, $visitorId);
    mysqli_stmt_execute($stmt);
    $deleted = mysqli_stmt_affected_rows($stmt) > 0;
    mysqli_stmt_close($stmt);
    return $deleted;
}

function get_visitor_stats($conn, $visitorId) {
    $sql  = "SELECT
                (SELECT COUNT(*) FROM visit_passes WHERE visitor_id = ? AND status = 'booked') AS booked,
                (SELECT COUNT(*) FROM visit_passes WHERE visitor_id = ? AND status = 'booked' AND visit_date >=
CURDATE()) AS upcoming,
                (SELECT COUNT(*) FROM book_suggestions WHERE visitor_id = ?) AS suggestions";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'iii', $visitorId, $visitorId, $visitorId);
    mysqli_stmt_execute($stmt);
    $row = mysqli_fetch_assoc(mysqli_stmt_get_result($stmt));
    mysqli_stmt_close($stmt);
    return [
        'booked'      => (int)($row['booked']      ?? 0),
        'upcoming'    => (int)($row['upcoming']    ?? 0),
        'suggestions' => (int)($row['suggestions'] ?? 0),
    ];
}
```

Line by line

Line	Code	What it does and why
10	`function make_pass_code() {`	**Visitor feature.** Builds a code such as `LIB-4821-KQ` that a person can read out at the desk.
11	`$letters = strtoupper(substr(str_shuffle('ABCDEFGHJKLMNPQRSTUVWXYZ'), 0, 2));`	Look closely at the alphabet: **I and O are missing**. They are too easily confused with 1 and 0 when a code is read aloud or written by hand. Small details like this are what makes a system pleasant to use at a busy desk.
12	`return 'LIB-' . random_int(1000, 9999) . '-' . $letters;`	`random_int` rather than `rand`, because it is the cryptographically secure generator and there is no reason to use the weaker one.

Line	Code	What it does and why
15	`function add_pass($conn, $visitorId, $visitDate, $purpose, $guests) {`	Books a visit. The code is generated here rather than being accepted from the browser, so a visitor cannot choose their own.
26	`function get_passes($conn, $visitorId) {`	One visitor's passes, newest visit first. Filtered by the session id, like every other visitor query.
50	`function get_pass($conn, $id, $visitorId) {`	Note that `$visitorId` is **required** here, unlike the borrow model's version. Nobody else in the system ever needs to load a single pass, so there is no reason to allow it.
60	`function update_pass($conn, $id, $visitorId, $visitDate, $purpose, $guests, $status) {`	Edits a pass, again with the owner id in the `WHERE` clause.
71	`function delete_pass($conn, $id, $visitorId) {`	Deletes a pass, and reports whether a row actually disappeared so the controller can tell the difference between success and a tampered-with id.
81	`function pass_exists_on_date($conn, $visitorId, $visitDate, $excludeId = 0) {`	Enforces one booking per visitor per day. The `$excludeId` plays the same role as in `has_open_request` — without it, editing a pass without changing its date would report a clash with itself.
95	`function get_application($conn, $visitorId) {`	The visitor's most recent application. `ORDER BY id DESC LIMIT 1` means a visitor who was rejected and applied again sees only the latest attempt, while the older row stays in the table as history.
116	`function get_pending_applications($conn) {`	The admin's queue: applications still waiting, joined to the users table so the admin sees who is asking.
126	`function decide_application($conn, $id, $decision) {`	Approves or rejects.
127	`$sql  = "UPDATE membership_applications SET status = ? WHERE id = ? AND status = 'pending'";`	The status condition makes the decision safe to click twice. A second click changes nothing, so an impatient double-click cannot approve an already-rejected application.
147	`function add_suggestion($conn, $visitorId, $title, $author, $reason) {`	**Visitor feature.** Records a book the visitor would like the library to buy.
167	`function delete_suggestion($conn, $id, $visitorId) {`	A visitor may remove their own suggestions, and only their own.
176	`function get_visitor_stats($conn, $visitorId) {`	**Visitor feature.** The counters on the visitor dashboard.
178	`(SELECT COUNT(*) FROM visit_passes WHERE visitor_id = ? AND status = 'booked') AS booked,`	Three sub-queries inside one `SELECT`, so the dashboard needs one database round trip instead of three. The same visitor id is bound three times, hence `iii`.

6.9 models/log_model.php

Purpose. The audit trail. The smallest model in the project, and the one that makes the admin's activity log possible.

Full listing of **models/log_model.php** (43 lines).

```php
<?php
// ================================================================
// MODEL: activity_log
// Every important action is written here so the admin can audit it.
// ================================================================

function log_activity($conn, $action) {
    $user = current_user();
    $id       = $user['id']       ?? null;
    $username = $user['username'] ?? 'guest';
    $role     = $user['role']     ?? 'guest';
    $ip       = $_SERVER['REMOTE_ADDR'] ?? '0.0.0.0';

    $sql  = "INSERT INTO activity_log (user_id, username, role, action, ip)
             VALUES (?, ?, ?, ?, ?)";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'issss', $id, $username, $role, $action, $ip);
    mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
}

function get_logs($conn, $limit = 20) {
    $sql  = "SELECT * FROM activity_log ORDER BY id DESC LIMIT ?";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'i', $limit);
    mysqli_stmt_execute($stmt);
    $rows = mysqli_fetch_all(mysqli_stmt_get_result($stmt), MYSQLI_ASSOC);
    mysqli_stmt_close($stmt);
    return $rows;
}

function search_logs($conn, $term, $limit = 50) {
    $like = '%' . $term . '%';
    $sql  = "SELECT * FROM activity_log
             WHERE username LIKE ? OR role LIKE ? OR action LIKE ? OR ip LIKE ?
             ORDER BY id DESC LIMIT ?";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, 'ssssi', $like, $like, $like, $like, $limit);
    mysqli_stmt_execute($stmt);
    $rows = mysqli_fetch_all(mysqli_stmt_get_result($stmt), MYSQLI_ASSOC);
    mysqli_stmt_close($stmt);
    return $rows;
}
```

Line	Code	What it does and why
7	`function log_activity($conn, $action) {`	Called from the controllers after anything worth remembering. The caller supplies only a description; everything else is worked out here, so no caller can log a false identity.
8	`$user = current_user();`	Reads the signed-in user from the session rather than from any parameter.
10	`$username = $user['username'] ?? 'guest';`	Falls back to "guest" so that actions taken before signing in — a failed registration, for instance — can still be recorded.
12	`$ip = $_SERVER['REMOTE_ADDR'] ?? '0.0.0.0';`	The network address of the computer that made the request, useful when investigating who did something. The fallback covers command-line runs where there is no address.
22	`function get_logs($conn, $limit = 20) {`	The newest entries. The limit is bound as a parameter rather than pasted into the text, because even a number that comes from our own code is safer bound.
32	`function search_logs($conn, $term, $limit = 50) {`	**Admin feature.** Searches four columns of the log at once, so the admin can look up a person, a role, an action or an address.

Part 7 — Code walkthrough: the controllers

The decision-making layer. Every rule about what a user may and may not do lives here.

7.1 The shape of every dashboard controller

The four dashboard controllers are written to the same plan, so once you have read one you can navigate all four. The plan is:

1. Read `action` from the address and set up `$error = ''` and `$editing = null`.
2. If the action is **add** and this is a POST: check the token, validate every field, call the model, redirect on success.
3. If the action is **update** and this is a POST: the same, plus the empty-field check and ownership rules.
4. If the action is **edit**: load one row into `$editing` so the form switches to edit mode.
5. If the action is **delete**: check the token, delete, redirect.
6. Any extra feature actions for this role.
7. Fetch the lists the screen needs and `require` the view.

Two variables carry the state of the form between the controller and the view:

- `$error` — empty when all is well; otherwise the one message to print in red at the top.
- `$editing` — `null` means the form is in "add" mode; an array means "edit" mode, and its values pre-fill the boxes.

NOTE

When validation fails during an update, the controller fills `$editing` with **what the user typed**, not with what is in the database. That is why a rejected form comes back with the user's own words still in it instead of silently throwing their work away.

7.2 controllers/auth_controller.php

Purpose. The three doors: signing in, signing up and signing out. This is the most security-sensitive file in the project.

Full listing of **controllers/auth_controller.php** (142 lines).

```php
<?php
// ================================================================
// CONTROLLER: login / register / logout
// ================================================================

/* ==================== LOGIN (all 4 roles) ==================== */
function login_controller($conn) {
    // Already signed in? Go straight to your own dashboard.
    if (is_logged_in()) {
        redirect('index.php?page=' . current_role());
    }

    $error = '';
    // COOKIE: "remember me" only refills the username, never the password.
    $prefill = $_COOKIE['remember_user'] ?? '';

    if (is_post()) {
        csrf_check();

        $username = trim($_POST['username'] ?? '');
        $password = $_POST['password'] ?? '';
        $remember = isset($_POST['remember']);

        // ---- PHP validation ----
        if (is_blank($username) || is_blank($password)) {
            $error = 'Enter both your username and password.';
        } else {
            $user = find_user_by_username($conn, $username);

            // The same message for a wrong username and a wrong password, so
            // nobody can find out which usernames exist.
            if (!$user || !password_verify($password, $user['password'])) {
                $error = 'Wrong username or password.';
            } elseif ($user['status'] === 'suspended') {
                $error = 'This account is suspended. Please contact the administrator.';
```

```
            } else {
                // SECURITY: a fresh session id blocks session-fixation attacks.
                session_regenerate_id(true);

                $_SESSION['user'] = [
                    'id'         => (int)$user['id'],
                    'name'       => $user['name'],
                    'username'   => $user['username'],
                    'email'      => $user['email'],
                    'contact'    => $user['contact'],
                    'role'       => $user['role'],
                    'created_at' => $user['created_at'],
                ];
                $_SESSION['last_active'] = time();

                if ($remember) {
                    setcookie('remember_user', $username, [
                        'expires'  => time() + 60 * 60 * 24 * 30,
                        'path'     => '/',
                        'httponly' => true,
                        'samesite' => 'Lax',
                    ]);
                } else {
                    setcookie('remember_user', '', time() - 3600, '/');
                }

                log_activity($conn, 'Signed in');
                redirect('index.php?page=' . $user['role']);
            }
        }
    }

    require __DIR__ . '/../views/auth/login.php';
}

/* ============ REGISTER (student / librarian / visitor) ============ */
function register_controller($conn) {
    if (is_logged_in()) {
        redirect('index.php?page=' . current_role());
    }

    $error = '';
    $success = '';
    $old = ['name' => '', 'email' => '', 'contact' => '', 'username' => '', 'role' => 'student'];

    if (is_post()) {
        csrf_check();

        $name     = trim($_POST['name'] ?? '');
        $email    = trim($_POST['email'] ?? '');
        $contact  = trim($_POST['contact'] ?? '');
        $username = trim($_POST['username'] ?? '');
        $password = $_POST['password'] ?? '';
        $confirm  = $_POST['confirm_password'] ?? '';
        $role     = $_POST['role'] ?? '';

        $old = compact('name', 'email', 'contact', 'username', 'role');

        // Nobody can sign up as an admin: only these three are accepted.
        $allowedRoles = ['student', 'librarian', 'visitor'];

        // ---- PHP validation (the real gate) ----
        if (is_blank($name) || is_blank($email) || is_blank($contact)
            || is_blank($username) || is_blank($password)) {
            $error = 'Fill in every field.';
        } elseif (!in_array($role, $allowedRoles, true)) {
            $error = 'Choose an account type.';
        } elseif (strlen($name) < 3) {
            $error = 'Name must be at least 3 characters.';
        } elseif (!valid_email($email)) {
            $error = 'Enter a valid email address.';
        } elseif (!valid_contact($contact)) {
            $error = 'Enter a valid contact number (6-20 digits).';
        } elseif (!preg_match('/^[A-Za-z0-9_]{4,20}$/', $username)) {
            $error = 'Username must be 4-20 letters, numbers or underscores.';
        } elseif (strlen($password) < 6) {
            $error = 'Password must be at least 6 characters.';
        } elseif ($password !== $confirm) {
            $error = 'The two passwords do not match.';
        } elseif (username_exists($conn, $username)) {
            $error = 'That username is already taken.';
        } else {
            if (create_user($conn, $name, $email, $contact, $username, $password, $role)) {
                log_activity($conn, 'New ' . $role . ' account registered: ' . $username);
                set_flash('success', 'Account created. You can sign in now.');
                redirect('index.php?page=login');
            }
            $error = 'Could not create the account. Please try again.';
        }
    }

    require __DIR__ . '/../views/auth/register.php';
}

/* ==================== LOGOUT ==================== */
function logout_controller($conn) {
    if (is_logged_in()) {
        log_activity($conn, 'Signed out');
    }
    $_SESSION = [];
```

```
    session_regenerate_id(true);
    session_destroy();
    setcookie('remember_user', '', time() - 3600, '/');
    session_start();
    set_flash('success', 'You have been signed out.');
    redirect('index.php?page=login');
}
```

Signing in

Line	Code	What it does and why
7	`function login_controller($conn) {`	Handles both jobs: showing the empty form, and processing it when it comes back.
9	`if (is_logged_in()) {`	Somebody already signed in has no business on the sign-in page, so they are sent to their dashboard.
15	`$prefill = $_COOKIE['remember_user'] ?? '';`	**The "remember me" cookie.** It holds only the username. It never holds the password, and it is not a way of staying signed in — it simply saves typing. Treating it as harmless is safe precisely because it is harmless.
17	`if (is_post()) {`	Everything below runs only when the form was submitted. On a normal visit the code falls straight through to the view at the bottom.
18	`csrf_check();`	The forgery check, before anything else is read.
20	`$username = trim($_POST['username'] ?? '');`	Reads the username, removing stray spaces. The `?? ''` covers a request that arrived without the field at all, which is what a hand-made request looks like.
25	`if (is_blank($username) \|\| is_blank($password)) {`	The cheapest check first: if either box is empty there is no point troubling the database.
28	`$user = find_user_by_username($conn, $username);`	One query fetches the row, including the stored hash.
32	`if (!$user \|\| !password_verify($password, $user['password'])) {`	**Two failures, one message.** `password_verify` hashes what was typed and compares it with the stored hash. Whether the username was wrong or the password was wrong, the user is told the same thing — otherwise an attacker could use the difference to discover which usernames exist.
34	`} elseif ($user['status'] === 'suspended') {`	The suspension check comes **after** the password check on purpose. Announcing "this account is suspended" to somebody who has not proved they own it would leak information.
38	`session_regenerate_id(true);`	**Security.** Issues a brand-new session id at the moment privileges change. This defeats session fixation, where an attacker plants a known session id on a victim and waits for them to sign in. The `true` deletes the old session file as well.
40–47	`$_SESSION['user'] = [ ... 'created_at' => $user['created_at'],`	Copies the details the screens need into the session — but not the password hash, which is deliberately left behind and never travels with the session.
49	`$_SESSION['last_active'] = time();`	Starts the idle clock used by `check_session_timeout()`.
51–56	`if ($remember) { ... 'samesite' => 'Lax',`	Stores the username for thirty days, with the same `httponly` and `samesite` protections as the session cookie.
59	`setcookie('remember_user', '', time() - 3600, '/');`	Unticking the box deletes the cookie. A cookie is removed by setting its expiry in the past — there is no "delete cookie" instruction.
62	`log_activity($conn, 'Signed in');`	Records the sign-in. It happens after the session is filled so the log knows who to name.

Line	Code	What it does and why
63	`redirect('index.php?page=' . $user['role']);`	**The role decides the destination.** There is no list of "if admin go here, if student go there" — the role word is the page name, which is why adding a fifth role would need no change to this line.
68	`require __DIR__ . '/../views/auth/login.php';`	Reached only when the form was not submitted or the sign-in failed. The view can see `$error` and `$prefill` because it is included inside this function.

Signing up

Line	Code	What it does and why
72	`function register_controller($conn) {`	Self-registration for three of the four roles.
79	`$old = ['name' => '', 'email' => '', 'contact' => '', 'username' => '', 'role' => 'student'];`	Holds what the user typed so a rejected form can be redrawn with their answers still in place.
95	`$allowedRoles = ['student', 'librarian', 'visitor'];`	**A white list, not a black list.** Rather than trying to spot the word "admin" and block it, the code lists what is permitted and refuses everything else. White lists fail safely; black lists fail open.
101	`} elseif (!in_array($role, $allowedRoles, true)) {`	The check itself. The third argument `true` demands a strict comparison, so no type-juggling surprise can slip a value through. A hand-made request asking for the admin role is rejected here.
103	`} elseif (strlen($name) < 3) {`	The validation chain begins. `elseif` means the user is told about one problem at a time, starting with the most basic.
105	`} elseif (!valid_email($email)) {`	Uses PHP's own email validator.
107	`} elseif (!valid_contact($contact)) {`	Digits, spaces, plus, dash and brackets only.
109	`} elseif (!preg_match('/^[A-Za-z0-9_]{4,20}$/', $username)) {`	The username pattern, read piece by piece: `^` start, `[A-Za-z0-9_]` letters, digits or underscore, `{4,20}` between four and twenty of them, `$` end. The anchors matter — without them the pattern would accept anything that merely *contains* four valid characters.
111	`} elseif (strlen($password) < 6) {`	A minimum length. Short and honest: this is a teaching project, and a real one would ask for more.
113	`} elseif ($password !== $confirm) {`	The two password boxes must match. `!==` compares value and type.
115	`} elseif (username_exists($conn, $username)) {`	The database check comes **last**, because it is the only one that costs a query. Cheap checks first is a habit worth forming.
118	`if (create_user($conn, $name, $email, $contact, $username, $password, $role)) {`	Only now is the account created. `create_user` hashes the password.
120	`set_flash('success', 'Account created. You can sign in now.');`	The message survives the redirect and appears on the sign-in page.

Signing out

Line	Code	What it does and why
131	`function logout_controller($conn) {`	Ends the visit completely.
133	`log_activity($conn, 'Signed out');`	Logged before the session is destroyed, while the identity is still known.
135	`$_SESSION = [];`	Empties the session data in memory.
137	`session_destroy();`	Removes the session on the server. Emptying the array alone would not do that.

Line	Code	What it does and why
138	`setcookie('remember_user', '', time() - 3600, '/');`	Clears the remembered username as well, so the next user of a shared computer starts clean.
139	`session_start();`	A fresh, empty session is started for one reason only: to carry the "You have been signed out." message to the next page.

7.3 controllers/admin_controller.php

Purpose. User management plus the admin's three features. The most powerful controller, and therefore the one with the most guards protecting the administrator from themselves.

Full listing of **controllers/admin_controller.php** (177 lines).

```php
<?php
// ============================================================
// CONTROLLER: ADMIN dashboard
// CRUD  : user accounts (every role)
// Extras: 1) account status + membership approvals
//         2) system activity log
//         3) live statistics panel (AJAX)
// ============================================================

function admin_controller($conn) {
    $action     = $_GET['action'] ?? 'list';
    $roleFilter = $_GET['role']   ?? '';
    $me         = current_user();

    $error   = '';
    $editing = null;   // when filled the form switches to "edit" mode

    // Only these role names are ever accepted from the browser.
    $roles    = ['admin', 'librarian', 'student', 'visitor'];
    $statuses = ['active', 'suspended'];

    /* ---------------- CREATE ---------------- */
    if ($action === 'add' && is_post()) {
        csrf_check();

        $name     = trim($_POST['name'] ?? '');
        $email    = trim($_POST['email'] ?? '');
        $contact  = trim($_POST['contact'] ?? '');
        $username = trim($_POST['username'] ?? '');
        $password = $_POST['password'] ?? '';
        $role     = $_POST['role'] ?? '';

        if (is_blank($name) || is_blank($email) || is_blank($contact)
            || is_blank($username) || is_blank($password)) {
            $error = 'Fill in every field.';
        } elseif (!in_array($role, $roles, true)) {
            $error = 'Choose a valid role.';
        } elseif (!valid_email($email)) {
            $error = 'Enter a valid email address.';
        } elseif (!valid_contact($contact)) {
            $error = 'Enter a valid contact number.';
        } elseif (!preg_match('/^[A-Za-z0-9_]{4,20}$/', $username)) {
            $error = 'Username must be 4-20 letters, numbers or underscores.';
        } elseif (strlen($password) < 6) {
            $error = 'Password must be at least 6 characters.';
        } elseif (username_exists($conn, $username)) {
            $error = 'That username is already taken.';
        } else {
            if (create_user($conn, $name, $email, $contact, $username, $password, $role)) {
                log_activity($conn, 'Created ' . $role . ' account: ' . $username);
                set_flash('success', 'Account created.');
                redirect('index.php?page=admin');
            }
            $error = 'Could not create the account.';
        }
    }

    /* ---------------- UPDATE ---------------- */
    if ($action === 'update' && is_post()) {
        csrf_check();

        $id       = (int)($_GET['id'] ?? 0);
        $name     = trim($_POST['name'] ?? '');
        $email    = trim($_POST['email'] ?? '');
        $contact  = trim($_POST['contact'] ?? '');
        $username = trim($_POST['username'] ?? '');
        $password = $_POST['password'] ?? '';          // blank = keep old password
        $role     = $_POST['role'] ?? '';
        $status   = $_POST['status'] ?? 'active';

        // Keep the typed values on screen when validation fails.
        $editing = ['id' => $id, 'name' => $name, 'email' => $email, 'contact' => $contact,
                    'username' => $username, 'role' => $role, 'status' => $status];
```

```
74
75          // ===== NULL / EMPTY VALIDATION ON UPDATE =====
76          if (is_blank($name) || is_blank($email) || is_blank($contact) || is_blank($username)) {
77              $error = 'No field can be left empty (NULL). All fields are required.';
78          } elseif (!in_array($role, $roles, true) || !in_array($status, $statuses, true)) {
79              $error = 'Choose a valid role and status.';
80          } elseif (!valid_email($email)) {
81              $error = 'Enter a valid email address.';
82          } elseif (!valid_contact($contact)) {
83              $error = 'Enter a valid contact number.';
84          } elseif (username_exists($conn, $username, $id)) {
85              $error = 'Another account already uses that username.';
86          } elseif ($password !== '' && strlen($password) < 6) {
87              $error = 'Password must be at least 6 characters (or leave it blank).';
88          } elseif ($id === (int)$me['id'] && ($role !== 'admin' || $status !== 'active')) {
89              $error = 'You cannot remove your own admin access.';
90          } else {
91              if (update_user($conn, $id, $name, $email, $contact, $username, $role, $status)) {
92                  if ($password !== '') {
93                      update_password($conn, $id, $password);
94                  }
95                  log_activity($conn, 'Updated account #' . $id . ' (' . $username . ')');
96                  set_flash('success', 'Account updated.');
97                  redirect('index.php?page=admin');
98              }
99              $error = 'Update failed.';
100         }
101     }
102
103     /* ---------------- READ one row into the form ---------------- */
104     if ($action === 'edit' && !$editing) {
105         $editing = get_user($conn, (int)($_GET['id'] ?? 0));
106         if (!$editing) {
107             set_flash('error', 'That account no longer exists.');
108             redirect('index.php?page=admin');
109         }
110     }
111
112     /* ---------------- DELETE ---------------- */
113     if ($action === 'delete') {
114         csrf_check();
115         $id = (int)($_GET['id'] ?? 0);
116
117         if ($id === (int)$me['id']) {
118             set_flash('error', 'You cannot delete your own account.');
119         } elseif ($id > 0 && delete_user($conn, $id)) {
120             log_activity($conn, 'Deleted account #' . $id);
121             set_flash('success', 'Account deleted.');
122         } else {
123             set_flash('error', 'Could not delete that account.');
124         }
125         redirect('index.php?page=admin');
126     }
127
128     /* -------- FEATURE 1a: suspend / activate an account -------- */
129     if ($action === 'status') {
130         csrf_check();
131         $id     = (int)($_GET['id'] ?? 0);
132         $status = $_GET['to'] ?? '';
133
134         if ($id === (int)$me['id']) {
135             set_flash('error', 'You cannot suspend your own account.');
136         } elseif (in_array($status, $statuses, true) && set_user_status($conn, $id, $status)) {
137             log_activity($conn, 'Set account #' . $id . ' to ' . $status);
138             set_flash('success', 'Account is now ' . $status . '.');
139         } else {
140             set_flash('error', 'Could not change that account.');
141         }
142         redirect('index.php?page=admin');
143     }
144
145     /* -------- FEATURE 1b: approve / reject membership upgrades -------- */
146     if ($action === 'membership') {
147         csrf_check();
148         $id       = (int)($_GET['id'] ?? 0);
149         $decision = $_GET['to'] ?? '';
150         $app      = get_application_by_id($conn, $id);
151
152         if ($app && in_array($decision, ['approved', 'rejected'], true)
153             && decide_application($conn, $id, $decision)) {
154
155             if ($decision === 'approved') {
156                 // Approving turns the visitor into a student.
157                 set_user_role($conn, (int)$app['visitor_id'], 'student');
158             }
159             log_activity($conn, 'Membership application #' . $id . ' ' . $decision);
160             set_flash('success', 'Application ' . $decision . '.');
161         } else {
162             set_flash('error', 'Could not update that application.');
163         }
164         redirect('index.php?page=admin');
165     }
166
167     /* ---------------- Data for the view ---------------- */
168     if (!in_array($roleFilter, $roles, true)) {
169         $roleFilter = '';
170     }
171     $users        = get_users($conn, $roleFilter);
172     $stats        = get_system_stats($conn);
173     $logs         = get_logs($conn, 12);
```

```
    $applications = get_pending_applications($conn);

    require __DIR__ . '/../views/admin/dashboard.php';
}
```

Line by line

Line	Code	What it does and why
11	`$action     = $_GET['action'] ?? 'list';`	Defaults to plain list mode, so a bare `index.php?page=admin` shows the dashboard.
19	`$roles    = ['admin', 'librarian', 'student', 'visitor'];`	The white list of acceptable roles, used by both the add and update branches.
23	`if ($action === 'add' && is_post()) {`	Create. Both conditions are required: the right action **and** an actual form submission.
36	`} elseif (!in_array($role, $roles, true)) {`	Even an administrator cannot invent a fifth role by editing the page in their browser.
49	`if (create_user($conn, $name, $email, $contact, $username, $password, $role)) {`	On success the code logs, sets a message and redirects — the Post/Redirect/Get pattern that stops F5 from creating the account twice.
59	`if ($action === 'update' && is_post()) {`	Update.
67	`$password = $_POST['password'] ?? '';  // blank = keep old password`	On the edit form the password box is optional. Blank means "leave the password alone".
72	`$editing = ['id' => $id, 'name' => $name, 'email' => $email, 'contact' => $contact,`	Captures the typed values immediately, so that whichever validation branch fails, the form can be redrawn exactly as the user left it.
76	`if (is_blank($name) \|\| is_blank($email) || is_blank($contact) \|\| is_blank($username)) {`	**The empty-field rule on update.** Without it a user could be saved with a blank name, and the account would become impossible to identify in any list.
84	`} elseif (username_exists($conn, $username, $id)) {`	The clash check, passing the current id so the row does not collide with itself.
86	`} elseif ($password !== '' && strlen($password) < 6) {`	The length rule applies only when a new password was actually typed.
88	`} elseif ($id === (int)$me['id'] && ($role !== 'admin' || $status !== 'active')) {`	**Self-protection.** An admin may not demote or suspend themselves. Without this line one careless click could leave the system with no working administrator and no way back in.
92	`if ($password !== '') {`	The password is only rewritten when one was supplied, which is what makes the "leave blank" behaviour work.
104	`if ($action === 'edit' && !$editing) {`	Loads a row into the form. The `&& !$editing` matters: if a failed update already filled `$editing`, it must not be overwritten with the old database values.
106	`if (!$editing) {`	A deleted or invented id produces a friendly message rather than a blank form or a crash.
113	`if ($action === 'delete') {`	Delete. Notice `csrf_check()` even though this arrives as a link, not a form.
117	`if ($id === (int)$me['id']) {`	Self-protection again: an administrator cannot delete their own account.
129	`if ($action === 'status') {`	**Admin feature 1a.** Suspend or activate an account.
132	`$status = $_GET['to'] ?? '';`	The target state travels in the address; the next line checks it against the white list before it can reach the database.
146	`if ($action === 'membership') {`	**Admin feature 1b.** Approve or reject a visitor’s membership application.

Line	Code	What it does and why
152	`if ($app && in_array($decision, ['approved', 'rejected'], true)`	Three conditions in one: the application exists, the decision is one of the two permitted words, and the update actually changed a row.
157	`set_user_role($conn, (int)$app['visitor_id'], 'student');`	**The upgrade itself.** Approving does two things: it stamps the application, and it changes the person's role, so the next time they sign in they land on the student dashboard.
168	`if (!in_array($roleFilter, $roles, true)) {`	The filter from the drop-down is checked before use. Anything unexpected becomes "all roles" rather than an error.
171–174	`$users = get_users($conn, $roleFilter);` ... `$applications = get_pending_applications($conn);`	The four lists the dashboard needs. This is the last thing the controller does before handing over to the view.

7.4 controllers/librarian_controller.php

Purpose. The catalogue plus the librarian's three features. The issue desk in this file is the only place in the project where two tables must change together, so it is worth reading slowly.

Full listing of **controllers/librarian_controller.php** (186 lines).

```php
<?php
// ================================================================
// CONTROLLER: LIBRARIAN dashboard
// CRUD  : books
// Extras: 1) issue & return desk (stock updates automatically)
//         2) low stock alert list
//         3) export the whole catalogue as a CSV file
// ================================================================

function librarian_controller($conn) {
    $action = $_GET['action'] ?? 'list';
    $me     = current_user();

    $error   = '';
    $editing = null;

    /* ---------------- CREATE ---------------- */
    if ($action === 'add' && is_post()) {
        csrf_check();

        $title    = trim($_POST['title'] ?? '');
        $author   = trim($_POST['author'] ?? '');
        $category = trim($_POST['category'] ?? '');
        $isbn     = trim($_POST['isbn'] ?? '');
        $quantity = trim($_POST['quantity'] ?? '');
        $price    = trim($_POST['price'] ?? '');

        if (is_blank($title) || is_blank($author) || is_blank($category)
            || is_blank($isbn) || is_blank($quantity) || is_blank($price)) {
            $error = 'Fill in every field.';
        } elseif (!ctype_digit($quantity)) {
            $error = 'Quantity must be a whole number (0 or more).';
        } elseif (!is_numeric($price) || (float)$price < 0) {
            $error = 'Price must be a number (0 or more).';
        } elseif (!preg_match('/^[0-9\-]{6,20}$/', $isbn)) {
            $error = 'ISBN may only contain digits and dashes (6-20 characters).';
        } else {
            if (add_book($conn, $title, $author, $category, $isbn,
                         (int)$quantity, (float)$price, (int)$me['id'])) {
                log_activity($conn, 'Added book: ' . $title);
                set_flash('success', 'Book added to the catalogue.');
                redirect('index.php?page=librarian');
            }
            $error = 'Could not add the book.';
        }
    }

    /* ---------------- UPDATE ---------------- */
    if ($action === 'update' && is_post()) {
        csrf_check();

        $id       = (int)($_GET['id'] ?? 0);
        $title    = trim($_POST['title'] ?? '');
        $author   = trim($_POST['author'] ?? '');
        $category = trim($_POST['category'] ?? '');
        $isbn     = trim($_POST['isbn'] ?? '');
        $quantity = trim($_POST['quantity'] ?? '');
        $price    = trim($_POST['price'] ?? '');

        $editing = ['id' => $id, 'title' => $title, 'author' => $author,
                    'category' => $category, 'isbn' => $isbn,
                    'quantity' => $quantity, 'price' => $price];
```

```
63
64          // ===== NULL / EMPTY VALIDATION ON UPDATE =====
65          if (is_blank($title) || is_blank($author) || is_blank($category)
66              || is_blank($isbn) || is_blank($quantity) || is_blank($price)) {
67              $error = 'No field can be left empty (NULL). All fields are required.';
68          } elseif (!ctype_digit($quantity)) {
69              $error = 'Quantity must be a whole number (0 or more).';
70          } elseif (!is_numeric($price) || (float)$price < 0) {
71              $error = 'Price must be a number (0 or more).';
72          } elseif (!preg_match('/^[0-9\-]{6,20}$/', $isbn)) {
73              $error = 'ISBN may only contain digits and dashes (6-20 characters).';
74          } else {
75              if (update_book($conn, $id, $title, $author, $category, $isbn,
76                              (int)$quantity, (float)$price)) {
77                  log_activity($conn, 'Updated book #' . $id . ': ' . $title);
78                  set_flash('success', 'Book updated.');
79                  redirect('index.php?page=librarian');
80              }
81              $error = 'Update failed.';
82          }
83      }
84
85      /* ---------------- READ one row into the form ---------------- */
86      if ($action === 'edit' && !$editing) {
87          $editing = get_book($conn, (int)($_GET['id'] ?? 0));
88          if (!$editing) {
89              set_flash('error', 'That book no longer exists.');
90              redirect('index.php?page=librarian');
91          }
92      }
93
94      /* ---------------- DELETE ---------------- */
95      if ($action === 'delete') {
96          csrf_check();
97          $id = (int)($_GET['id'] ?? 0);
98          if ($id > 0 && delete_book($conn, $id)) {
99              log_activity($conn, 'Deleted book #' . $id);
100             set_flash('success', 'Book deleted.');
101         } else {
102             set_flash('error', 'Could not delete that book.');
103         }
104         redirect('index.php?page=librarian');
105     }
106
107     /* -------- FEATURE 1: issue desk -------- */
108     if ($action === 'issue') {
109         csrf_check();
110         $id  = (int)($_GET['id'] ?? 0);
111         $req = get_request($conn, $id);
112
113         if (!$req || $req['status'] !== 'pending') {
114             set_flash('error', 'That request cannot be issued.');
115         } elseif (!change_book_stock($conn, (int)$req['book_id'], -1)) {
116             set_flash('error', 'No copies of "' . $req['title'] . '" are on the shelf.');
117         } elseif (issue_request($conn, $id)) {
118             log_activity($conn, 'Issued "' . $req['title'] . '" to ' . $req['student_id']);
119             set_flash('success', 'Book issued. Due in ' . LOAN_DAYS . ' days.');
120         } else {
121             change_book_stock($conn, (int)$req['book_id'], 1); // undo the stock change
122             set_flash('error', 'Could not issue that book.');
123         }
124         redirect('index.php?page=librarian');
125     }
126
127     if ($action === 'return') {
128         csrf_check();
129         $id  = (int)($_GET['id'] ?? 0);
130         $req = get_request($conn, $id);
131
132         if (!$req || $req['status'] !== 'issued') {
133             set_flash('error', 'That book is not currently issued.');
134         } else {
135             $fine = calculate_fine($req['due_date']);       // late days x FINE_PER_DAY
136             if (return_request($conn, $id, $fine)) {
137                 change_book_stock($conn, (int)$req['book_id'], 1);
138                 log_activity($conn, 'Returned "' . $req['title'] . '", fine ' . money($fine));
139                 set_flash('success', $fine > 0
140                     ? 'Book returned. Fine due: ' . money($fine) . '.'
141                     : 'Book returned on time. No fine.');
142             } else {
143                 set_flash('error', 'Could not return that book.');
144             }
145         }
146         redirect('index.php?page=librarian');
147     }
148
149     if ($action === 'reject') {
150         csrf_check();
151         $id = (int)($_GET['id'] ?? 0);
152         if (reject_request($conn, $id)) {
153             log_activity($conn, 'Rejected borrow request #' . $id);
154             set_flash('success', 'Request rejected.');
155         } else {
156             set_flash('error', 'Could not reject that request.');
157         }
158         redirect('index.php?page=librarian');
159     }
160
161     /* -------- FEATURE 3: CSV export -------- */
162     if ($action === 'export') {
```

```
        $books = get_books($conn);
        log_activity($conn, 'Exported the book catalogue');

        header('Content-Type: text/csv; charset=utf-8');
        header('Content-Disposition: attachment; filename="book_catalogue_' . date('Y-m-d') . '.csv"');

        $out = fopen('php://output', 'w');
        fputcsv($out, ['ID', 'Title', 'Author', 'Category', 'ISBN', 'Quantity', 'Price']);
        foreach ($books as $b) {
            fputcsv($out, [$b['id'], $b['title'], $b['author'], $b['category'],
                           $b['isbn'], $b['quantity'], $b['price']]);
        }
        fclose($out);
        exit;
    }

    /* ---------------- Data for the view ---------------- */
    $books    = get_books($conn);
    $requests = get_all_requests($conn);
    $lowStock = get_low_stock_books($conn);
    $stats    = get_librarian_stats($conn);

    require __DIR__ . '/../views/librarian/dashboard.php';
}
```

Book validation

Line	Code	What it does and why
31	`} elseif (!ctype_digit($quantity)) {`	`ctype_digit` is true only when every character is a digit. It rejects `2.5`, `-1`, `two` and an empty box in one step — and because it works on the text before conversion, it also rejects `2abc`, which `intval()` would silently turn into 2.
33	`} elseif (!is_numeric($price) \|\| (float)$price < 0) {`	Price is allowed decimals, so a different test is needed. `is_numeric` accepts `27` and `27.50`; the second half rejects negative money.
35	`} elseif (!preg_match('/^[0-9\-]{6,20}$/', $isbn)) {`	ISBNs are digits and dashes, six to twenty characters. Inside a character class the dash is escaped so it means a literal dash rather than a range.

The issue desk

Line	Code	What it does and why
108	`if ($action === 'issue') {`	**Librarian feature 1.** Handing a book to a student. Two tables must change: the request becomes issued, and the shelf count drops by one.
113	`if (!$req || $req['status'] !== 'pending') {`	Refuses anything that is not a waiting request — an invented id, or a request somebody has already dealt with in another browser tab.
115	`} elseif (!change_book_stock($conn, (int)$req['book_id'], -1)) {`	**Stock is taken first, and the result is checked.** If the shelf is empty the query changes no rows, this returns false, and the librarian is told which title is unavailable. The request is left untouched.
117	`} elseif (issue_request($conn, $id)) {`	Only after a copy has been successfully reserved is the request marked as issued and the due date set.
121	`change_book_stock($conn, (int)$req['book_id'], 1); // undo the stock change`	**The undo.** If the stock came down but the status update then failed, this puts the copy back. Without it a copy could vanish from the shelf while no student had it — the classic symptom of a half-finished operation.
127	`if ($action === 'return') {`	Taking a book back.
135	`$fine = calculate_fine($req['due_date']); // late days x FINE_PER_DAY`	The fine is worked out **now**, at the moment of return, from the due date. Calling the shared helper means the number the student saw growing on their own screen and the number the librarian charges come from the same rule.
136	`if (return_request($conn, $id, $fine)) {`	Stores the fine and the return date, and flips the status.

Line	Code	What it does and why
137	`change_book_stock($conn, (int)$req['book_id'], 1);`	The copy goes back on the shelf, but only after the return was recorded successfully. The order is the mirror image of issuing.
139	`set_flash('success', $fine > 0`	The message adapts: a late return names the amount, an on-time return says so plainly. Small courtesies like this are what make software feel finished.
149	`if ($action === 'reject') {`	Turning a request down. No stock changes, because none was ever taken.

The CSV export

Line	Code	What it does and why
162	`if ($action === 'export') {`	**Librarian feature 3.** Produces a spreadsheet file instead of a web page.
166	`header('Content-Type: text/csv; charset=utf-8');`	Tells the browser this is a spreadsheet, not a page, so it does not try to display it as text.
167	`header('Content-Disposition: attachment; filename="book_catalogue_' . date('Y-m-d') . '.csv"');`	`attachment` makes the browser download rather than display, and supplies a filename that includes today's date so repeated exports do not overwrite each other.
169	`$out = fopen('php://output', 'w');`	`php://output` is a special name meaning "write straight to the browser". Nothing is saved on the server, so the export leaves no files behind to tidy up.
170	`fputcsv($out, ['ID', 'Title', 'Author', 'Category', 'ISBN', 'Quantity', 'Price']);`	The header row. `fputcsv` is used rather than joining with commas by hand because it knows the escaping rules — a book title containing a comma or a quotation mark will not break the file.
176	`exit;`	Stops immediately. Without it, the dashboard HTML would be appended to the spreadsheet and the file would be unreadable.

7.5 controllers/student_controller.php

Purpose. A student's own borrow requests, and the numbers behind the fine tracker. Every query in this file is tied to the id in the session, which is what keeps students out of each other's records.

Full listing of **controllers/student_controller.php** (124 lines).

```php
<?php
// ================================================================
// CONTROLLER: STUDENT dashboard
// CRUD  : my borrow requests
// Extras: 1) catalogue browser with live availability
//         2) due-date and fine tracker
//         3) printable digital library card
// ================================================================

function student_controller($conn) {
    $action    = $_GET['action'] ?? 'list';
    $me        = current_user();
    $studentId = (int)$me['id'];

    $error   = '';
    $editing = null;

    /* ---------------- CREATE (request a book) ---------------- */
    if ($action === 'add' && is_post()) {
        csrf_check();

        $bookId     = (int)($_POST['book_id'] ?? 0);
        $pickupDate = trim($_POST['pickup_date'] ?? '');
        $notes      = trim($_POST['notes'] ?? '');
        $book       = get_book($conn, $bookId);

        if ($bookId <= 0 || !$book) {
            $error = 'Choose a book from the list.';
        } elseif (is_blank($pickupDate) || !valid_date($pickupDate)) {
```

```
            $error = 'Choose a valid pick-up date.';
        } elseif (strtotime($pickupDate) < strtotime(date('Y-m-d'))) {
            $error = 'The pick-up date cannot be in the past.';
        } elseif (strlen($notes) > 255) {
            $error = 'Notes must be shorter than 255 characters.';
        } elseif ((int)$book['quantity'] <= 0) {
            $error = 'No copies of that book are on the shelf right now.';
        } elseif (has_open_request($conn, $studentId, $bookId)) {
            $error = 'You already have an open request for that book.';
        } else {
            if (add_request($conn, $studentId, $bookId, $pickupDate, $notes)) {
                log_activity($conn, 'Requested "' . $book['title'] . '"');
                set_flash('success', 'Request sent. A librarian will review it.');
                redirect('index.php?page=student');
            }
            $error = 'Could not send the request.';
        }
    }

    /* ---------------- UPDATE (only while pending) ---------------- */
    if ($action === 'update' && is_post()) {
        csrf_check();

        $id         = (int)($_GET['id'] ?? 0);
        $bookId     = (int)($_POST['book_id'] ?? 0);
        $pickupDate = trim($_POST['pickup_date'] ?? '');
        $notes      = trim($_POST['notes'] ?? '');
        $original   = get_request($conn, $id, $studentId);

        $editing = ['id' => $id, 'book_id' => $bookId,
                    'pickup_date' => $pickupDate, 'notes' => $notes];

        // ===== NULL / EMPTY VALIDATION ON UPDATE =====
        if (!$original) {
            set_flash('error', 'That request does not belong to you.');
            redirect('index.php?page=student');
        } elseif ($original['status'] !== 'pending') {
            set_flash('error', 'Only pending requests can be edited.');
            redirect('index.php?page=student');
        } elseif ($bookId <= 0 || is_blank($pickupDate)) {
            $error = 'No field can be left empty (NULL). Book and pick-up date are required.';
        } elseif (!valid_date($pickupDate)) {
            $error = 'Choose a valid pick-up date.';
        } elseif (strlen($notes) > 255) {
            $error = 'Notes must be shorter than 255 characters.';
        } elseif (has_open_request($conn, $studentId, $bookId, $id)) {
            $error = 'You already have another open request for that book.';
        } else {
            if (update_request($conn, $id, $studentId, $bookId, $pickupDate, $notes)) {
                log_activity($conn, 'Updated borrow request #' . $id);
                set_flash('success', 'Request updated.');
                redirect('index.php?page=student');
            }
            $error = 'Update failed.';
        }
    }

    /* ---------------- READ one row into the form ---------------- */
    if ($action === 'edit' && !$editing) {
        $editing = get_request($conn, (int)($_GET['id'] ?? 0), $studentId);
        if (!$editing) {
            set_flash('error', 'That request does not belong to you.');
            redirect('index.php?page=student');
        }
    }

    /* ---------------- DELETE (cancel) ---------------- */
    if ($action === 'delete') {
        csrf_check();
        $id = (int)($_GET['id'] ?? 0);
        if ($id > 0 && delete_request($conn, $id, $studentId)) {
            log_activity($conn, 'Cancelled borrow request #' . $id);
            set_flash('success', 'Request cancelled.');
        } else {
            set_flash('error', 'Only your own pending requests can be cancelled.');
        }
        redirect('index.php?page=student');
    }

    /* ---------------- Data for the view ---------------- */
    $requests  = get_student_requests($conn, $studentId);
    $books     = get_books($conn);
    $available = get_available_books($conn);
    $stats     = get_student_stats($conn, $studentId);

    // FEATURE 2: fine that is building up right now on late books.
    $liveFine = 0.0;
    foreach ($requests as $r) {
        if ($r['status'] === 'issued') {
            $liveFine += calculate_fine($r['due_date']);
        }
    }

    require __DIR__ . '/../views/student/dashboard.php';
}
```

Line by line

Line	Code	What it does and why
13	`$studentId = (int)$me['id'];`	**The identity that matters.** Taken from the session once, at the top, and passed to every model call below. Nothing in this file ever takes a student id from the address bar.
25	`$book = get_book($conn, $bookId);`	The chosen book is fetched so the code can check it really exists and has copies, rather than trusting the drop-down.
27	`if ($bookId <= 0 \|\| !$book) {`	Catches both an empty selection and an invented id.
29	`} elseif (is_blank($pickupDate) \|\| !valid_date($pickupDate)) {`	The date must exist as a real calendar date.
31	`} elseif (strtotime($pickupDate) < strtotime(date('Y-m-d'))) {`	And it must not be in the past. Comparing timestamps sidesteps every text-comparison trap.
35	`} elseif ((int)$book['quantity'] <= 0) {`	A last-moment stock check. The drop-down only offers available books, but the last copy may have been issued while this page sat open.
37	`} elseif (has_open_request($conn, $studentId, $bookId)) {`	One open request per title per student.
57	`$original = get_request($conn, $id, $studentId);`	**The ownership check on update.** Passing the session id means a request belonging to somebody else simply comes back empty.
63	`if (!$original) {`	A missing row here means either a bad id or an attempt to edit another student's request; both get the same brief answer.
66	`} elseif ($original['status'] !== 'pending') {`	Once a librarian has issued the book the request is frozen. The model repeats this rule in its `WHERE` clause, so it is enforced twice.
89	`$editing = get_request($conn, (int)($_GET['id'] ?? 0), $studentId);`	The edit form loads through the same ownership-checked function.
100	`if ($id > 0 && delete_request($conn, $id, $studentId)) {`	Cancelling. Because the model checks owner and status in SQL, a failure covers every wrong case at once and produces one honest message.
116	`$liveFine = 0.0;`	**Student feature 2.** The fine that is accruing right now.
117	`foreach ($requests as $r) {`	Adds up `calculate_fine()` for every book still out. Books returned long ago are ignored, because their fines were frozen when they came back.

7.6 controllers/visitor_controller.php

Purpose. Day passes, the membership application and the suggestion box — three small features in one file.

Full listing of **controllers/visitor_controller.php** (167 lines).

```php
<?php
// ============================================================
// CONTROLLER: VISITOR dashboard
// CRUD  : my visit passes
// Extras: 1) apply for a student membership upgrade
//         2) book suggestion box
//         3) printable day pass with a unique pass code
// ============================================================

function visitor_controller($conn) {
    $action    = $_GET['action'] ?? 'list';
    $me        = current_user();
    $visitorId = (int)$me['id'];

    $error   = '';
    $editing = null;

    /* ---------------- CREATE (book a visit) ---------------- */
    if ($action === 'add' && is_post()) {
        csrf_check();

        $visitDate = trim($_POST['visit_date'] ?? '');
        $purpose   = trim($_POST['purpose'] ?? '');
        $guests    = trim($_POST['guests'] ?? '');

```

```
        if (is_blank($visitDate) || is_blank($purpose) || is_blank($guests)) {
            $error = 'Fill in every field.';
        } elseif (!valid_date($visitDate)) {
            $error = 'Choose a valid visit date.';
        } elseif (strtotime($visitDate) < strtotime(date('Y-m-d'))) {
            $error = 'The visit date cannot be in the past.';
        } elseif (!ctype_digit($guests) || (int)$guests < 1 || (int)$guests > 10) {
            $error = 'Guests must be a whole number between 1 and 10.';
        } elseif (strlen($purpose) > 150) {
            $error = 'Purpose must be shorter than 150 characters.';
        } elseif (pass_exists_on_date($conn, $visitorId, $visitDate)) {
            $error = 'You already have a pass for that day.';
        } else {
            if (add_pass($conn, $visitorId, $visitDate, $purpose, (int)$guests)) {
                log_activity($conn, 'Booked a visit for ' . $visitDate);
                set_flash('success', 'Visit pass booked. Show the pass code at the desk.');
                redirect('index.php?page=visitor');
            }
            $error = 'Could not book the pass.';
        }
    }

    /* ---------------- UPDATE ---------------- */
    if ($action === 'update' && is_post()) {
        csrf_check();

        $id        = (int)($_GET['id'] ?? 0);
        $visitDate = trim($_POST['visit_date'] ?? '');
        $purpose   = trim($_POST['purpose'] ?? '');
        $guests    = trim($_POST['guests'] ?? '');
        $status    = $_POST['status'] ?? 'booked';
        $original  = get_pass($conn, $id, $visitorId);

        $editing = ['id' => $id, 'visit_date' => $visitDate, 'purpose' => $purpose,
                    'guests' => $guests, 'status' => $status];

        // ===== NULL / EMPTY VALIDATION ON UPDATE =====
        if (!$original) {
            set_flash('error', 'That pass does not belong to you.');
            redirect('index.php?page=visitor');
        } elseif (is_blank($visitDate) || is_blank($purpose) || is_blank($guests)) {
            $error = 'No field can be left empty (NULL). All fields are required.';
        } elseif (!valid_date($visitDate)) {
            $error = 'Choose a valid visit date.';
        } elseif (!ctype_digit($guests) || (int)$guests < 1 || (int)$guests > 10) {
            $error = 'Guests must be a whole number between 1 and 10.';
        } elseif (!in_array($status, ['booked', 'cancelled'], true)) {
            $error = 'Choose a valid status.';
        } elseif ($status === 'booked' && pass_exists_on_date($conn, $visitorId, $visitDate, $id)) {
            $error = 'You already have another pass for that day.';
        } else {
            if (update_pass($conn, $id, $visitorId, $visitDate, $purpose, (int)$guests, $status)) {
                log_activity($conn, 'Updated visit pass #' . $id);
                set_flash('success', 'Visit pass updated.');
                redirect('index.php?page=visitor');
            }
            $error = 'Update failed.';
        }
    }

    /* ---------------- READ one row into the form ---------------- */
    if ($action === 'edit' && !$editing) {
        $editing = get_pass($conn, (int)($_GET['id'] ?? 0), $visitorId);
        if (!$editing) {
            set_flash('error', 'That pass does not belong to you.');
            redirect('index.php?page=visitor');
        }
    }

    /* ---------------- DELETE ---------------- */
    if ($action === 'delete') {
        csrf_check();
        $id = (int)($_GET['id'] ?? 0);
        if ($id > 0 && delete_pass($conn, $id, $visitorId)) {
            log_activity($conn, 'Deleted visit pass #' . $id);
            set_flash('success', 'Visit pass deleted.');
        } else {
            set_flash('error', 'Could not delete that pass.');
        }
        redirect('index.php?page=visitor');
    }

    /* -------- FEATURE 1: membership upgrade application -------- */
    if ($action === 'apply' && is_post()) {
        csrf_check();

        $reason  = trim($_POST['reason'] ?? '');
        $current = get_application($conn, $visitorId);

        if ($current && $current['status'] === 'pending') {
            set_flash('error', 'Your application is already waiting for a decision.');
        } elseif (strlen($reason) < 10) {
            set_flash('error', 'Tell us in at least 10 characters why you want to join.');
        } elseif (add_application($conn, $visitorId, $reason)) {
            log_activity($conn, 'Applied for student membership');
            set_flash('success', 'Application sent to the administrator.');
        } else {
            set_flash('error', 'Could not send the application.');
        }
        redirect('index.php?page=visitor');
```

```
    }

    /* -------- FEATURE 2: book suggestion box -------- */
    if ($action === 'suggest' && is_post()) {
        csrf_check();

        $title  = trim($_POST['s_title'] ?? '');
        $author = trim($_POST['s_author'] ?? '');
        $reason = trim($_POST['s_reason'] ?? '');

        if (is_blank($title) || is_blank($author)) {
            set_flash('error', 'A suggestion needs both a title and an author.');
        } elseif (strlen($title) > 200 || strlen($author) > 120) {
            set_flash('error', 'Title or author is too long.');
        } elseif (add_suggestion($conn, $visitorId, $title, $author, $reason)) {
            log_activity($conn, 'Suggested the book "' . $title . '"');
            set_flash('success', 'Thanks! Your suggestion was recorded.');
        } else {
            set_flash('error', 'Could not save the suggestion.');
        }
        redirect('index.php?page=visitor');
    }

    if ($action === 'unsuggest') {
        csrf_check();
        $id = (int)($_GET['id'] ?? 0);
        if ($id > 0 && delete_suggestion($conn, $id, $visitorId)) {
            set_flash('success', 'Suggestion removed.');
        } else {
            set_flash('error', 'Could not remove that suggestion.');
        }
        redirect('index.php?page=visitor');
    }

    /* ---------------- Data for the view ---------------- */
    $passes      = get_passes($conn, $visitorId);
    $application = get_application($conn, $visitorId);
    $suggestions = get_suggestions($conn, $visitorId);
    $stats       = get_visitor_stats($conn, $visitorId);

    require __DIR__ . '/../views/visitor/dashboard.php';
}
```

Line by line

Line	Code	What it does and why
13	`$visitorId = (int)$me['id'];`	The same pattern as the student controller: identity from the session, never from the address.
30	`} elseif (strtotime($visitDate) < strtotime(date('Y-m-d'))) {`	No booking a visit for last week.
32	`} elseif (!ctype_digit($guests) \|\| (int)$guests < 1 || (int)$guests > 10) {`	A whole number between one and ten. Three tests in one line: it is a number, it is not too small, it is not too large.
36	`} elseif (pass_exists_on_date($conn, $visitorId, $visitDate)) {`	One pass per visitor per day.
57	`$original  = get_pass($conn, $id, $visitorId);`	The ownership check before an edit.
74	`} elseif ($status === 'booked' && pass_exists_on_date($conn, $visitorId, $visitDate, $id)) {`	When editing, the clash test excludes this pass so it cannot conflict with itself. It also only applies when the pass is still booked — two cancelled passes on one day harm nobody.
109	`if ($action === 'apply' && is_post()) {`	**Visitor feature 1.** The membership application.
115	`if ($current && $current['status'] === 'pending') {`	Stops a visitor filling the admin's queue with copies of the same request.
117	`} elseif (strlen($reason) < 10) {`	A minimum length, so the admin has something to judge rather than a single word.
129	`if ($action === 'suggest' && is_post()) {`	**Visitor feature 2.** The suggestion box. The field names are prefixed `s_` because this form shares the page with the pass form and the names must not collide.
149	`if ($action === 'unsuggest') {`	Removing a suggestion, restricted by the model to the visitor's own rows.
161–164	`$passes      = get_passes($conn, $visitorId);` `...`	The four pieces of data the visitor dashboard needs.

Line	Code	What it does and why
	`$stats = get_visitor_stats($conn, $visitorId);`	

7.7 controllers/ajax_controller.php

Purpose. Every request that returns data instead of a page. This is what makes the search boxes update as you type and the admin's statistics refresh by themselves.

The router does **not** apply `require_role` to this file, because different endpoints belong to different roles. Each one therefore checks for itself — and the default answer is refusal.

Full listing of **controllers/ajax_controller.php** (81 lines).

```php
<?php
// ============================================================
// CONTROLLER: AJAX / JSON endpoints
// The dashboards call these with fetch() and redraw a table without
// reloading the page.  Every endpoint checks the role first.
// ============================================================

function ajax_controller($conn) {
    $action = $_GET['action'] ?? '';
    $term   = trim($_GET['q'] ?? '');

    /* ---- Public endpoint: live "is this username free?" on the signup page ---- */
    if ($action === 'check_username') {
        $username = trim($_GET['username'] ?? '');
        if (!preg_match('/^[A-Za-z0-9_]{4,20}$/', $username)) {
            json_out(['ok' => false, 'message' => 'Use 4-20 letters, numbers or underscores.']);
        }
        json_out(username_exists($conn, $username)
            ? ['ok' => false, 'message' => 'That username is taken.']
            : ['ok' => true,  'message' => 'That username is free.']);
    }

    /* ---- Everything below needs a logged-in user ---- */
    if (!is_logged_in()) {
        json_out(['error' => 'Please sign in first.'], 401);
    }

    $user = current_user();
    $role = $user['role'];
    $id   = (int)$user['id'];

    switch ($action) {

        /* ----------- Admin ----------- */
        case 'search_users':
            if ($role !== 'admin') break;
            $roleFilter = $_GET['role'] ?? '';
            if (!in_array($roleFilter, ['admin', 'librarian', 'student', 'visitor'], true)) {
                $roleFilter = '';
            }
            json_out($term === ''
                ? get_users($conn, $roleFilter)
                : search_users($conn, $term, $roleFilter));

        case 'stats':                       // live statistics panel
            if ($role !== 'admin') break;
            json_out(get_system_stats($conn));

        case 'search_logs':
            if ($role !== 'admin') break;
            json_out($term === '' ? get_logs($conn, 12) : search_logs($conn, $term, 50));

        /* ----------- Librarian ----------- */
        case 'search_books':
            if ($role !== 'librarian' && $role !== 'student') break;  // both may browse
            json_out($term === '' ? get_books($conn) : search_books($conn, $term));

        case 'low_stock':
            if ($role !== 'librarian') break;
            json_out(get_low_stock_books($conn));

        case 'search_issues':
            if ($role !== 'librarian') break;
            json_out($term === '' ? get_all_requests($conn) : search_all_requests($conn, $term));

        /* ----------- Student ----------- */
        case 'search_requests':
            if ($role !== 'student') break;
            json_out($term === ''
                ? get_student_requests($conn, $id)
                : search_student_requests($conn, $id, $term));

        /* ----------- Visitor ----------- */
        case 'search_passes':
            if ($role !== 'visitor') break;
```

```
            json_out($term === '' ? get_passes($conn, $id) : search_passes($conn, $id, $term));
    }

    // Wrong role, or an action that does not exist.
    json_out(['error' => 'You are not allowed to use this endpoint.'], 403);
}
```

Line by line

Line	Code	What it does and why
13	`if ($action === 'check_username') {`	**The one public endpoint.** It has to be public, because it serves the sign-up page where nobody is signed in yet.
15	`if (!preg_match('/^[A-Za-z0-9_]{4,20}$/', $username)) {`	The pattern is checked here too, so the browser gets useful feedback before it is worth asking the database anything.
18	`json_out(username_exists($conn, $username)`	Answers only yes or no. It never returns names, emails or any other detail, so it cannot be turned into a way of harvesting the user list.
24	`if (!is_logged_in()) {`	Everything past this point requires a session. The reply is status 401 with a short message.
32	`switch ($action) {`	One `case` per endpoint. The `switch` makes the whole API readable at a glance, which matters because this file is the system's data door.
36	`if ($role !== 'admin') break;`	**The pattern that protects everything.** `break` leaves the switch **without** returning any data, and execution falls through to the refusal at the bottom. Forgetting a role check therefore cannot leak data by accident — it can only produce a refusal.
45	`case 'stats':  // live statistics panel`	The live statistics panel. Admin only.
54	`case 'search_books':`	The catalogue search.
55	`if ($role !== 'librarian' && $role !== 'student') break;  // both may browse`	The only endpoint shared by two roles, because both need to browse the catalogue. A visitor is still refused.
67	`case 'search_requests':`	A student searching their own requests. The id comes from the session, so this endpoint cannot be pointed at anybody else's history.
74	`case 'search_passes':`	A visitor searching their own passes, filtered the same way.
80	`json_out(['error' => 'You are not allowed to use this endpoint.'], 403);`	**The safe default.** Any unknown action, and every role check that fell through, ends here with 403 Forbidden and no data at all.

Part 8 — Code walkthrough: the views

The files that produce HTML. They contain no queries and make no decisions — they display what the controller handed them.

8.1 The shared layout

All four dashboards share a header and a footer, so the navigation bar exists once rather than four times. A view sets three variables and includes the header; the header uses them for the page title and heading.

Full listing of **views/partials/header.php** (52 lines).

```
<?php
// Shared top of every dashboard page.
// The view that includes this must set: $pageTitle, $pageHeading, $pageSub
$navUser = current_user();
?>
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title><?= esc($pageTitle ?? APP_NAME) ?> &mdash; <?= esc(APP_NAME) ?></title>
<link rel="stylesheet" href="assets/css/style.css">
</head>
<body class="app-body">

<header class="navbar">
    <div class="navbar-inner">
        <a class="brand" href="index.php?page=<?= esc($navUser['role']) ?>">
            <span class="brand-icon">&#128218;</span>
            <span><?= esc(APP_NAME) ?></span>
        </a>
        <div class="nav-user">
            <span class="user-pill">
                <span class="user-avatar"><?= esc(strtoupper(substr($navUser['name'], 0, 1))) ?></span>
                <span class="user-meta">
                    <span class="user-name"><?= esc($navUser['name']) ?></span>
                    <span class="user-role"><?= esc(role_label($navUser['role'])) ?></span>
                </span>
            </span>
            <a href="index.php?page=logout" class="btn-logout">Sign out</a>
        </div>
    </div>
</header>

<main class="main-content">
    <div class="page-header">
        <div>
            <h1 class="page-title"><?= esc($pageHeading ?? '') ?></h1>
            <p class="page-sub"><?= esc($pageSub ?? '') ?></p>
        </div>
        <?php if (!empty($headerAction)): ?>
            <div><?= $headerAction ?></div>
        <?php endif; ?>
    </div>

    <?php foreach (get_flash() as $flash): ?>
        <div class="alert alert-<?= esc($flash['type']) ?>"><?= esc($flash['message']) ?></div>
    <?php endforeach; ?>

    <?php if (!empty($error)): ?>
        <div class="alert alert-error"><?= esc($error) ?></div>
    <?php endif; ?>
```

Line	Code	What it does and why
4	`$navUser = current_user();`	The signed-in user, for the navigation bar. The header is only ever included from a page behind `require_role`, so this can never be null.
11	`<title><?= esc($pageTitle ?? APP_NAME) ?> &mdash; <?= esc(APP_NAME) ?></title>`	The browser tab title, built from a variable the view set plus the application name.
12	`<link rel="stylesheet" href="assets/css/style.css">`	The single stylesheet. A relative path, which works because every address in the project is `index.php` sitting in the project root.
24	`<span class="user-avatar"><?= esc(strtoupper(substr($navUser['name'], 0, 1))) ?></span>`	The round avatar is not an image — it is the first letter of the name in a coloured circle. No uploads, no storage, no broken images, and it works for every user immediately.

Line	Code	What it does and why
27	`<span class="user-role"><?= esc(role_label($navUser['role'])) ?></span>`	Prints "Administrator" rather than the stored `admin`.
46	`<?php foreach (get_flash() as $flash): ?>`	Prints any one-time messages. Because `get_flash()` deletes as it reads, a refresh will not show them again.
50	`<?php if (!empty($error)): ?>`	The validation error, if the controller set one. Note the alternative `if: … endif;` syntax, which is easier to follow than braces when HTML is mixed in.

Full listing of **views/partials/footer.php** (11 lines).

```
</main>

<footer class="footer">
    &copy; <?= date('Y') ?> <?= esc(APP_NAME) ?> &mdash; Library Management System
</footer>

<script>
    // Settings from config.php that the JavaScript needs.
    window.FINE_RATE = <?= (int)FINE_PER_DAY ?>;
</script>
<script src="assets/js/app.js"></script>
```

The footer closes the layout and loads the JavaScript. The small script block before it passes one setting from PHP into JavaScript — the fine rate — so the browser can display a growing fine using the same number the server uses.

8.2 The sign-in screen

Full listing of **views/auth/login.php** (75 lines).

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Sign in &mdash; <?= esc(APP_NAME) ?></title>
<link rel="stylesheet" href="assets/css/style.css">
</head>
<body class="auth-body">

<div class="auth-shell">
    <!-- Left panel -->
    <div class="auth-side">
        <div class="logo-big">&#128218;</div>
        <h1>Library Management System</h1>
        <p>One place for staff, students and visitors. Sign in to open your dashboard.</p>
        <ul class="feature-list">
            <li>&#10003; Admin &mdash; accounts, approvals and the activity log</li>
            <li>&#10003; Librarian &mdash; catalogue, issue desk and stock alerts</li>
            <li>&#10003; Student &mdash; borrow books, track due dates and fines</li>
            <li>&#10003; Visitor &mdash; day passes, membership and suggestions</li>
        </ul>
    </div>

    <!-- Right panel: the form -->
    <div class="auth-form-wrap">
        <div class="auth-card">
            <h2>Welcome back</h2>
            <p class="muted">Sign in to continue</p>

            <?php foreach (get_flash() as $flash): ?>
                <div class="alert alert-<?= esc($flash['type']) ?>"><?= esc($flash['message']) ?></div>
            <?php endforeach; ?>

            <?php if (!empty($error)): ?>
                <div class="alert alert-error"><?= esc($error) ?></div>
            <?php endif; ?>

            <!-- onsubmit runs the JavaScript validation in assets/js/app.js -->
            <form method="POST" action="index.php?page=login" class="form"
                  novalidate onsubmit="return validateForm(this);">
                <?php csrf_field(); ?>

                <div class="field">
                    <label for="username">Username</label>
                    <input type="text" id="username" name="username" data-label="Username"
                           value="<?= esc($prefill ?? '') ?>"
                           placeholder="Your username" required autofocus>
                </div>
```

```
50
51                  <div class="field">
52                      <label for="password">Password</label>
53                      <input type="password" id="password" name="password" data-label="Password"
54                             placeholder="Your password" required>
55                  </div>
56
57                  <label class="checkbox">
58                      <input type="checkbox" name="remember" <?= !empty($prefill) ? 'checked' : '' ?>>
59                      <span>Remember my username on this computer</span>
60                  </label>
61
62                  <button type="submit" class="btn btn-primary btn-block">Sign in</button>
63              </form>
64
65              <p class="auth-foot">
66                  New here? <a href="index.php?page=register">Create an account</a>
67              </p>
68              <p class="hint"><strong>Default admin:</strong> admin / admin123</p>
69          </div>
70      </div>
71  </div>
72
73  <script src="assets/js/app.js"></script>
74  </body>
75  </html>
```

Line	Code	What it does and why
41	`novalidate onsubmit="return validateForm(this);">`	Runs the browser-side check first. Returning false stops the form being sent. This is convenience only — the server repeats every check.
41	`novalidate onsubmit="return validateForm(this);">`	Switches off the browser's own built-in validation messages so the project's consistent styling is used instead.
42	`<?php csrf_field(); ?>`	Drops the hidden token into the form.
47	`value="<?= esc($prefill ?? '') ?>"`	Pre-fills the remembered username. Even a value that came from our own cookie is escaped, because a cookie can be edited by the user.
58	`<input type="checkbox" name="remember" <?= !empty($prefill) ? 'checked' : '' ?>>`	The "remember me" tick box. It is pre-ticked when a remembered name was found, which matches what the user expects.
68	`<p class="hint"><strong>Default admin:</strong> admin / admin123</p>`	The demo credentials, shown deliberately because this is a teaching project. **Delete this line before any real deployment.**

8.3 The sign-up screen

Full listing of **views/auth/register.php** (126 lines).

```
1   <!DOCTYPE html>
2   <html lang="en">
3   <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Create an account &mdash; <?= esc(APP_NAME) ?></title>
7   <link rel="stylesheet" href="assets/css/style.css">
8   </head>
9   <body class="auth-body">
10
11  <div class="auth-shell">
12      <div class="auth-side">
13          <div class="logo-big">&#128218;</div>
14          <h1>Create your account</h1>
15          <p>Pick the account type that matches how you use the library.</p>
16          <ul class="feature-list">
17              <li>&#10003; <strong>Student</strong> &mdash; borrow books and track due dates</li>
18              <li>&#10003; <strong>Librarian</strong> &mdash; run the catalogue and issue desk</li>
19              <li>&#10003; <strong>Visitor</strong> &mdash; book day passes and suggest books</li>
20          </ul>
21          <p class="side-note">Administrator accounts are created by an existing admin.</p>
22      </div>
23
24      <div class="auth-form-wrap">
25          <div class="auth-card">
26              <h2>Sign up</h2>
27              <p class="muted">It takes less than a minute</p>
28
29              <?php if (!empty($error)): ?>
30                  <div class="alert alert-error"><?= esc($error) ?></div>
31              <?php endif; ?>
32              <?php if (!empty($success)): ?>
33                  <div class="alert alert-success"><?= esc($success) ?></div>
34              <?php endif; ?>
```

```
35
36              <form method="POST" action="index.php?page=register" class="form"
37                    novalidate onsubmit="return validateForm(this);">
38                  <?php csrf_field(); ?>
39
40                  <div class="field">
41                      <label for="role">Account type</label>
42                      <select id="role" name="role" data-label="Account type" required>
43                          <option value="student"   <?= $old['role'] === 'student'   ? 'selected' : ''
?>>Student</option>
44                          <option value="librarian" <?= $old['role'] === 'librarian' ? 'selected' : ''
?>>Librarian</option>
45                          <option value="visitor"   <?= $old['role'] === 'visitor'   ? 'selected' : ''
?>>Visitor</option>
46                      </select>
47                  </div>
48
49                  <div class="field">
50                      <label for="name">Full name</label>
51                      <input type="text" id="name" name="name" data-label="Full name" data-min="3"
52                             value="<?= esc($old['name']) ?>" placeholder="e.g. Rafiq Hasan" required>
53                  </div>
54
55                  <div class="field-row">
56                      <div class="field">
57                          <label for="email">Email</label>
58                          <input type="email" id="email" name="email" data-label="Email"
59                                 value="<?= esc($old['email']) ?>" placeholder="you@example.com" required>
60                      </div>
61                      <div class="field">
62                          <label for="contact">Contact number</label>
63                          <input type="text" id="contact" name="contact" data-label="Contact number" data-phone="1"
64                                 value="<?= esc($old['contact']) ?>" placeholder="+880 1XXXXXXXXX" required>
65                      </div>
66                  </div>
67
68                  <div class="field">
69                      <label for="username">Username</label>
70                      <input type="text" id="username" name="username" data-label="Username" data-min="4"
71                             value="<?= esc($old['username']) ?>" placeholder="4-20 letters or numbers" required>
72                      <!-- AJAX: filled in live by the script below -->
73                      <span id="usernameNote" class="field-note"></span>
74                  </div>
75
76                  <div class="field-row">
77                      <div class="field">
78                          <label for="password">Password</label>
79                          <input type="password" id="password" name="password" data-label="Password" data-min="6"
80                                 placeholder="At least 6 characters" required>
81                      </div>
82                      <div class="field">
83                          <label for="confirm_password">Repeat password</label>
84                          <input type="password" id="confirm_password" name="confirm_password"
85                                 data-label="Repeat password" data-match="password"
86                                 placeholder="Type it again" required>
87                      </div>
88                  </div>
89
90                  <button type="submit" class="btn btn-primary btn-block">Create account</button>
91              </form>
92
93              <p class="auth-foot">
94                  Already have an account? <a href="index.php?page=login">Sign in</a>
95              </p>
96          </div>
97      </div>
98  </div>
99
100 <script src="assets/js/app.js"></script>
101 <script>
102 // AJAX: ask the server whether the typed username is still free.
103 (function () {
104     var input = document.getElementById('username');
105     var note  = document.getElementById('usernameNote');
106     var timer;
107
108     input.addEventListener('input', function () {
109         clearTimeout(timer);
110         var value = input.value.trim();
111         if (value === '') { note.textContent = ''; note.className = 'field-note'; return; }
112
113         timer = setTimeout(function () {
114             fetch('index.php?page=ajax&action=check_username&username=' + encodeURIComponent(value))
115                 .then(function (r) { return r.json(); })
116                 .then(function (data) {
117                     note.textContent = data.message;
118                     note.className = 'field-note ' + (data.ok ? 'note-ok' : 'note-bad');
119                 })
120                 .catch(function () { note.textContent = ''; });
121         }, 300);
122     });
123 })();
124 </script>
125 </body>
126 </html>
```

Line	Code	What it does and why
42	`<select id="role" name="role" data-label="Account type" required>`	The role chooser offers three options. The server checks the answer again against its white list, because a drop-down in a browser can be edited freely.
51	`<input type="text" id="name" name="name" data-label="Full name" data-min="3"`	The JavaScript validator reads its rules from these `data-` attributes rather than from hard-coded field names, which is what lets one function validate every form in the project.
85	`data-label="Repeat password" data-match="password"`	Tells the validator that this box must equal the box named `password`.
73	`<span id="usernameNote" class="field-note"></span>`	The empty space where the live "that username is free" message will appear.
114	`fetch('index.php?page=ajax&action=check_username&username=' + encodeURIComponent(value))`	**AJAX in its simplest form.** As the user types, the browser quietly asks the server whether the username is taken and prints the answer, with no page reload and no form submission.
113	`timer = setTimeout(function () {`	**Debouncing.** The request is delayed 300 milliseconds and cancelled if another key is pressed, so typing a ten-letter name sends one request instead of ten.

8.4 The admin dashboard

The largest view. It contains the statistics panel, the add/edit form, the searchable account table, the membership queue and the activity log, followed by the JavaScript that drives its three background requests.

Full listing of **views/admin/dashboard.php** (338 lines).

```
<?php
$pageTitle   = 'Admin dashboard';
$pageHeading = 'User accounts';
$pageSub     = 'Create, edit, suspend and remove accounts for every role';
$isEdit      = !empty($editing);
require __DIR__ . '/../partials/header.php';
?>

<!-- ============ FEATURE 3: live statistics (refreshed by AJAX) ============ -->
<section class="stat-grid" id="statGrid">
    <div class="stat-card">
        <span class="stat-value" data-stat="students"><?= (int)$stats['students'] ?></span>
        <span class="stat-label">Students</span>
    </div>
    <div class="stat-card">
        <span class="stat-value" data-stat="librarians"><?= (int)$stats['librarians'] ?></span>
        <span class="stat-label">Librarians</span>
    </div>
    <div class="stat-card">
        <span class="stat-value" data-stat="visitors"><?= (int)$stats['visitors'] ?></span>
        <span class="stat-label">Visitors</span>
    </div>
    <div class="stat-card">
        <span class="stat-value" data-stat="suspended"><?= (int)$stats['suspended'] ?></span>
        <span class="stat-label">Suspended</span>
    </div>
    <div class="stat-card">
        <span class="stat-value" data-stat="books"><?= (int)$stats['books'] ?></span>
        <span class="stat-label">Book titles</span>
    </div>
    <div class="stat-card">
        <span class="stat-value" data-stat="issued"><?= (int)$stats['issued'] ?></span>
        <span class="stat-label">Books out</span>
    </div>
</section>
<p class="stat-note">Counts refresh every 15 seconds without reloading the page.</p>

<!-- ============ CREATE / UPDATE form ============ -->
<div class="card form-card">
    <h3 class="card-title">
        <?= $isEdit ? 'Edit account #' . (int)$editing['id'] : 'Add a new account' ?>
    </h3>
```

```
 43
 44      <form method="POST" class="form" novalidate onsubmit="return validateForm(this);"
 45            action="index.php?page=admin&amp;action=<?= $isEdit ? 'update&amp;id=' . (int)$editing['id'] : 'add'
?>">
 46          <?php csrf_field(); ?>
 47
 48          <div class="field-row">
 49              <div class="field">
 50                  <label for="name">Full name</label>
 51                  <input type="text" id="name" name="name" data-label="Full name" data-min="3"
 52                         value="<?= esc($editing['name'] ?? '') ?>" placeholder="Full name" required>
 53              </div>
 54              <div class="field">
 55                  <label for="email">Email</label>
 56                  <input type="email" id="email" name="email" data-label="Email"
 57                         value="<?= esc($editing['email'] ?? '') ?>" placeholder="name@example.com" required>
 58              </div>
 59          </div>
 60
 61          <div class="field-row">
 62              <div class="field">
 63                  <label for="contact">Contact number</label>
 64                  <input type="text" id="contact" name="contact" data-label="Contact number" data-phone="1"
 65                         value="<?= esc($editing['contact'] ?? '') ?>" placeholder="+880 1XXXXXXXXX" required>
 66              </div>
 67              <div class="field">
 68                  <label for="username">Username</label>
 69                  <input type="text" id="username" name="username" data-label="Username" data-min="4"
 70                         value="<?= esc($editing['username'] ?? '') ?>" placeholder="Login username" required>
 71              </div>
 72          </div>
 73
 74          <div class="field-row">
 75              <div class="field">
 76                  <label for="role">Role</label>
 77                  <select id="role" name="role" data-label="Role" required>
 78                      <?php foreach (['admin', 'librarian', 'student', 'visitor'] as $r): ?>
 79                          <option value="<?= $r ?>"
 80                              <?= (($editing['role'] ?? 'student') === $r) ? 'selected' : '' ?>>
 81                              <?= esc(role_label($r)) ?>
 82                          </option>
 83                      <?php endforeach; ?>
 84                  </select>
 85              </div>
 86
 87              <?php if ($isEdit): ?>
 88                  <div class="field">
 89                      <label for="status">Status</label>
 90                      <select id="status" name="status" data-label="Status" required>
 91                          <option value="active"    <?= (($editing['status'] ?? '') === 'active')    ? 'selected' :
'' ?>>Active</option>
 92                          <option value="suspended" <?= (($editing['status'] ?? '') === 'suspended') ? 'selected' :
'' ?>>Suspended</option>
 93                      </select>
 94                  </div>
 95              <?php else: ?>
 96                  <div class="field">
 97                      <label for="password">Password</label>
 98                      <input type="password" id="password" name="password" data-label="Password" data-min="6"
 99                             placeholder="At least 6 characters" required>
100                  </div>
101              <?php endif; ?>
102          </div>
103
104          <?php if ($isEdit): ?>
105              <div class="field">
106                  <label for="password">New password <span class="label-hint">(leave blank to keep the current
one)</span></label>
107                  <input type="password" id="password" name="password" data-label="New password" data-min="6"
108                         placeholder="Only fill this in to change the password">
109              </div>
110          <?php endif; ?>
111
112          <div class="form-actions">
113              <?php if ($isEdit): ?>
114                  <a href="index.php?page=admin" class="btn btn-ghost">Cancel</a>
115                  <button type="submit" class="btn btn-primary">Save changes</button>
116              <?php else: ?>
117                  <button type="submit" class="btn btn-primary">Create account</button>
118              <?php endif; ?>
119          </div>
120      </form>
121  </div>
122
123  <!-- ============ READ + SEARCH: all accounts ============ -->
124  <div class="card">
125      <div class="card-toolbar">
126          <div class="search-wrap">
127              <span class="search-icon">&#128269;</span>
128              <input type="text" id="userSearch" class="search-input"
129                     placeholder="Search by name, username, email or phone...">
130          </div>
131          <select id="roleFilter" class="filter-select">
132              <option value="">All roles</option>
133              <option value="admin"     <?= $roleFilter === 'admin'     ? 'selected' : ''
?>>Administrators</option>
134              <option value="librarian" <?= $roleFilter === 'librarian' ? 'selected' : '' ?>>Librarians</option>
135              <option value="student"   <?= $roleFilter === 'student'   ? 'selected' : '' ?>>Students</option>
136              <option value="visitor"   <?= $roleFilter === 'visitor'   ? 'selected' : '' ?>>Visitors</option>
137          </select>
```

```
          <span class="badge" id="userCount"><?= count($users) ?> accounts</span>
      </div>

      <div class="table-wrap">
          <table class="data-table">
              <thead>
                  <tr>
                      <th>#</th>
                      <th>Name</th>
                      <th>Username</th>
                      <th>Email</th>
                      <th>Contact</th>
                      <th>Role</th>
                      <th>Status</th>
                      <th class="text-right">Actions</th>
                  </tr>
              </thead>
              <tbody id="userTable">
                  <?php if (empty($users)): ?>
                      <tr><td colspan="8" class="empty">No accounts match this filter.</td></tr>
                  <?php else: ?>
                      <?php foreach ($users as $i => $u): ?>
                          <tr>
                              <td><?= $i + 1 ?></td>
                              <td><?= esc($u['name']) ?></td>
                              <td><?= esc($u['username']) ?></td>
                              <td><?= esc($u['email']) ?></td>
                              <td><?= esc($u['contact']) ?></td>
                              <td><span class="pill pill-<?= esc($u['role']) ?>"><?= esc(role_label($u['role']))
?></span></td>
                              <td><span class="pill pill-<?= esc($u['status']) ?>"><?= esc(ucfirst($u['status']))
?></span></td>
                              <td class="text-right">
                                  <a class="btn-sm btn-edit"
                                     href="index.php?page=admin&amp;action=edit&amp;id=<?= (int)$u['id']
?>">Edit</a>
                                  <?php if ($u['status'] === 'active'): ?>
                                      <a class="btn-sm btn-warn"
                                         href="<?=
esc(csrf_url('index.php?page=admin&action=status&to=suspended&id=' . (int)$u['id'])) ?>"
                                         onclick="return confirm('Suspend this account?');">Suspend</a>
                                  <?php else: ?>
                                      <a class="btn-sm btn-ok"
                                         href="<?= esc(csrf_url('index.php?page=admin&action=status&to=active&id='
. (int)$u['id'])) ?>">Activate</a>
                                  <?php endif; ?>
                                  <a class="btn-sm btn-delete"
                                     href="<?= esc(csrf_url('index.php?page=admin&action=delete&id=' .
(int)$u['id'])) ?>"
                                     onclick="return confirm('Delete this account for good?');">Delete</a>
                              </td>
                          </tr>
                      <?php endforeach; ?>
                  <?php endif; ?>
              </tbody>
          </table>
      </div>
  </div>

  <!-- ============ FEATURE 1b: membership upgrade requests ============ -->
  <div class="card">
      <h3 class="card-title card-title-pad">Membership upgrade requests</h3>
      <div class="table-wrap">
          <table class="data-table">
              <thead>
                  <tr><th>Visitor</th><th>Username</th><th>Why they want to join</th><th class="text-
right">Decision</th></tr>
              </thead>
              <tbody>
                  <?php if (empty($applications)): ?>
                      <tr><td colspan="4" class="empty">No visitor is waiting for a decision.</td></tr>
                  <?php else: ?>
                      <?php foreach ($applications as $app): ?>
                          <tr>
                              <td><?= esc($app['name']) ?></td>
                              <td><?= esc($app['username']) ?></td>
                              <td><?= esc($app['reason']) ?></td>
                              <td class="text-right">
                                  <a class="btn-sm btn-ok"
                                     href="<?=
esc(csrf_url('index.php?page=admin&action=membership&to=approved&id=' . (int)$app['id'])) ?>"
                                     onclick="return confirm('Approve and turn this visitor into a
student?');">Approve</a>
                                  <a class="btn-sm btn-delete"
                                     href="<?=
esc(csrf_url('index.php?page=admin&action=membership&to=rejected&id=' . (int)$app['id'])) ?>">Reject</a>
                              </td>
                          </tr>
                      <?php endforeach; ?>
                  <?php endif; ?>
              </tbody>
          </table>
      </div>
  </div>

  <!-- ============ FEATURE 2: activity log ============ -->
  <div class="card">
      <div class="card-toolbar">
          <div class="search-wrap">
              <span class="search-icon">&#128269;</span>
```

```
            <input type="text" id="logSearch" class="search-input"
                   placeholder="Search the activity log by user, role or action...">
        </div>
        <span class="badge" id="logCount"><?= count($logs) ?> entries</span>
    </div>
    <div class="table-wrap">
        <table class="data-table">
            <thead>
                <tr><th>When</th><th>User</th><th>Role</th><th>Action</th><th>IP address</th></tr>
            </thead>
            <tbody id="logTable">
                <?php if (empty($logs)): ?>
                    <tr><td colspan="5" class="empty">Nothing has happened yet.</td></tr>
                <?php else: ?>
                    <?php foreach ($logs as $log): ?>
                        <tr>
                            <td class="nowrap"><?= esc(date('d M, H:i', strtotime($log['created_at']))) ?></td>
                            <td><?= esc($log['username']) ?></td>
                            <td><span class="pill pill-<?= esc($log['role']) ?>"><?=
esc(role_label($log['role'])) ?></span></td>
                            <td><?= esc($log['action']) ?></td>
                            <td><?= esc($log['ip']) ?></td>
                        </tr>
                    <?php endforeach; ?>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
</div>

<?php require __DIR__ . '/../partials/footer.php'; ?>
<script>
/* ---------- AJAX 1: live account search + role filter ---------- */
var roleFilter = document.getElementById('roleFilter');

function userRow(u, i) {
    var suspendLink = (u.status === 'active')
        ? '<a class="btn-sm btn-warn" href="index.php?page=admin&action=status&to=suspended&id=' + u.id +
          '&csrf_token=<?= csrf_token() ?>" onclick="return confirm(\'Suspend this account?\');">Suspend</a>'
        : '<a class="btn-sm btn-ok" href="index.php?page=admin&action=status&to=active&id=' + u.id +
          '&csrf_token=<?= csrf_token() ?>">Activate</a>';

    return '<tr>' +
        '<td>' + (i + 1) + '</td>' +
        '<td>' + esc(u.name) + '</td>' +
        '<td>' + esc(u.username) + '</td>' +
        '<td>' + esc(u.email) + '</td>' +
        '<td>' + esc(u.contact) + '</td>' +
        '<td><span class="pill pill-' + esc(u.role) + '">' + esc(u.role) + '</span></td>' +
        '<td><span class="pill pill-' + esc(u.status) + '">' + esc(u.status) + '</span></td>' +
        '<td class="text-right">' +
            '<a class="btn-sm btn-edit" href="index.php?page=admin&action=edit&id=' + u.id + '">Edit</a>' +
            suspendLink +
            '<a class="btn-sm btn-delete" href="index.php?page=admin&action=delete&id=' + u.id +
            '&csrf_token=<?= csrf_token() ?>" onclick="return confirm(\'Delete this account for
good?\');">Delete</a>' +
        '</td>' +
    '</tr>';
}

function runUserSearch() {
    var term = document.getElementById('userSearch').value.trim();
    ajaxTable({
        url:     'index.php?page=ajax&action=search_users&role=' + encodeURIComponent(roleFilter.value) +
                 '&q=' + encodeURIComponent(term),
        tbody:   'userTable',
        counter: 'userCount',
        columns: 8,
        word:    'accounts',
        row:     userRow
    });
}

liveSearch('userSearch', runUserSearch);
roleFilter.addEventListener('change', runUserSearch);

/* ---------- AJAX 2: live statistics panel ---------- */
function refreshStats() {
    fetch('index.php?page=ajax&action=stats')
        .then(function (r) { return r.json(); })
        .then(function (data) {
            document.querySelectorAll('[data-stat]').forEach(function (el) {
                var key = el.getAttribute('data-stat');
                if (data[key] !== undefined) { el.textContent = data[key]; }
            });
        })
        .catch(function () { /* ignore a failed refresh */ });
}
setInterval(refreshStats, 15000);

/* ---------- AJAX 3: activity log search ---------- */
liveSearch('logSearch', function () {
    ajaxTable({
        url:     'index.php?page=ajax&action=search_logs&q=' +
                 encodeURIComponent(document.getElementById('logSearch').value.trim()),
        tbody:   'logTable',
        counter: 'logCount',
        columns: 5,
        word:    'entries',
        row: function (log) {
```

```
            return '<tr>' +
                '<td class="nowrap">' + esc(log.created_at) + '</td>' +
                '<td>' + esc(log.username) + '</td>' +
                '<td><span class="pill pill-' + esc(log.role) + '">' + esc(log.role) + '</span></td>' +
                '<td>' + esc(log.action) + '</td>' +
                '<td>' + esc(log.ip) + '</td>' +
            '</tr>';
        }
    });
});
</script>
</body>
</html>
```

Line	Code	What it does and why
5	`$isEdit      = !empty($editing);`	One variable decides everything about the form: its heading, its target address, and whether it shows a password box or a status box.
12	`<span class="stat-value" data-stat="students"><?= (int)$stats['students'] ?></span>`	Each statistic carries the name of the value it displays. The refresh function finds every element with a `data-stat` attribute and updates it by name, so adding a new statistic needs no new JavaScript.
45	`action="index.php?page=admin&action=<?= $isEdit ? 'update&id=' . (int)$editing['id'] : 'add' ?>">`	The same form posts to two different addresses depending on the mode. One form, two jobs — which is why the add and edit boxes can never drift apart.
52	`value="<?= esc($editing['name'] ?? '') ?>" placeholder="Full name" required>`	In edit mode the boxes are pre-filled; in add mode `?? ''` leaves them empty. The same line serves both.
87	`<?php if ($isEdit): ?>`	Edit mode shows a status chooser; add mode shows a password box instead. A new account must have a password, and an existing one must not be forced to change it.
106	`<label for="password">New password <span class="label-hint">(leave blank to keep the current one)</span></label>`	The optional password box, only in edit mode, with the "leave blank" instruction beside the label where it will actually be read.
166	`<td><span class="pill pill-<?= esc($u['role']) ?>"><?= esc(role_label($u['role'])) ?></span></td>`	The CSS class name is built from the data, so a role or status colours itself. No `if` chain is needed in the view.
173	`href="<?= esc(csrf_url('index.php?page=admin&action=status&to=suspended&id=' . (int)$u['id'])) ?>"`	The suspend link, carrying its token. The two branches of the `if` give the same button opposite meanings depending on the current status.
181	`onclick="return confirm('Delete this account for good?');">Delete</a>`	A last chance to change your mind. Returning false from `onclick` cancels the link.
262	`function userRow(u, i) {`	Builds one table row from one JSON object. The PHP loop above and this function must produce the same columns — the price of live search without a page reload.
271	`'<td>' + esc(u.name) + '</td>' +`	**Escaping again, this time in the browser.** Data arriving as JSON is escaped before it is

Line	Code	What it does and why
		inserted into the page, so a name containing a script tag is displayed as text rather than executed.
286	`function runUserSearch() {`	Reads the search box and the role filter together and asks for the matching accounts.
300	`roleFilter.addEventListener('change', runUserSearch);`	Changing the filter re-runs the same search, so the two controls cooperate instead of fighting.
303	`function refreshStats() {`	**Admin feature 3.** Fetches the numbers and writes them into the page.
314	`setInterval(refreshStats, 15000);`	Every fifteen seconds. Frequent enough to feel alive, rare enough to be invisible to the server.
317	`liveSearch('logSearch', function () {`	The activity log gets its own independent search, using the same shared helpers as everything else.

8.5 The librarian dashboard

Full listing of **views/librarian/dashboard.php** (293 lines).

```
  1  <?php
  2  $pageTitle    = 'Librarian dashboard';
  3  $pageHeading  = 'Book catalogue';
  4  $pageSub      = 'Add books, run the issue desk and watch your stock levels';
  5  $isEdit       = !empty($editing);
  6  $headerAction = '<a class="btn btn-ghost" href="index.php?page=librarian&amp;action=export">Download catalogue
(CSV)</a>';
  7  require __DIR__ . '/../partials/header.php';
  8  ?>
  9
 10  <section class="stat-grid">
 11      <div class="stat-card"><span class="stat-value"><?= (int)$stats['books'] ?></span><span class="stat-
label">Titles</span></div>
 12      <div class="stat-card"><span class="stat-value"><?= (int)$stats['copies'] ?></span><span class="stat-
label">Copies on shelf</span></div>
 13      <div class="stat-card"><span class="stat-value"><?= (int)$stats['pending'] ?></span><span class="stat-
label">Waiting requests</span></div>
 14      <div class="stat-card"><span class="stat-value"><?= (int)$stats['issued'] ?></span><span class="stat-
label">Currently out</span></div>
 15      <div class="stat-card <?= $stats['overdue'] > 0 ? 'stat-danger' : '' ?>"><span class="stat-value"><?=
(int)$stats['overdue'] ?></span><span class="stat-label">Overdue</span></div>
 16      <div class="stat-card <?= $stats['low_stock'] > 0 ? 'stat-warn' : '' ?>"><span class="stat-value"><?=
(int)$stats['low_stock'] ?></span><span class="stat-label">Low stock</span></div>
 17  </section>
 18
 19  <!-- ============ CREATE / UPDATE form ============ -->
 20  <div class="card form-card">
 21      <h3 class="card-title">
 22          <?= $isEdit ? 'Edit book #' . (int)$editing['id'] : 'Add a new book' ?>
 23      </h3>
 24
 25      <form method="POST" class="form" novalidate onsubmit="return validateForm(this);"
 26            action="index.php?page=librarian&amp;action=<?= $isEdit ? 'update&amp;id=' . (int)$editing['id'] :
'add' ?>">
 27          <?php csrf_field(); ?>
 28
 29          <div class="field-row">
 30              <div class="field">
 31                  <label for="title">Title</label>
 32                  <input type="text" id="title" name="title" data-label="Title" data-min="2"
 33                         value="<?= esc($editing['title'] ?? '') ?>"
 34                         placeholder="e.g. The Pragmatic Programmer" required>
 35              </div>
 36              <div class="field">
 37                  <label for="author">Author</label>
 38                  <input type="text" id="author" name="author" data-label="Author" data-min="2"
 39                         value="<?= esc($editing['author'] ?? '') ?>" placeholder="Author name" required>
 40              </div>
 41          </div>
 42
 43          <div class="field-row">
 44              <div class="field">
```

```
                <label for="category">Category</label>
                <input type="text" id="category" name="category" data-label="Category"
                       value="<?= esc($editing['category'] ?? '') ?>"
                       placeholder="e.g. Programming" required>
            </div>
            <div class="field">
                <label for="isbn">ISBN</label>
                <input type="text" id="isbn" name="isbn" data-label="ISBN" data-min="6"
                       value="<?= esc($editing['isbn'] ?? '') ?>"
                       placeholder="Digits and dashes only" required>
            </div>
        </div>

        <div class="field-row">
            <div class="field">
                <label for="quantity">Copies</label>
                <input type="number" id="quantity" name="quantity" data-label="Copies" min="0" step="1"
                       value="<?= esc($editing['quantity'] ?? '') ?>" placeholder="0" required>
            </div>
            <div class="field">
                <label for="price">Price (<?= esc(CURRENCY) ?>)</label>
                <input type="number" id="price" name="price" data-label="Price" min="0" step="0.01"
                       value="<?= esc($editing['price'] ?? '') ?>" placeholder="0.00" required>
            </div>
        </div>

        <div class="form-actions">
            <?php if ($isEdit): ?>
                <a href="index.php?page=librarian" class="btn btn-ghost">Cancel</a>
                <button type="submit" class="btn btn-primary">Save changes</button>
            <?php else: ?>
                <button type="submit" class="btn btn-primary">Add book</button>
            <?php endif; ?>
        </div>
    </form>
</div>

<!-- ============ READ + SEARCH: catalogue ============ -->
<div class="card">
    <div class="card-toolbar">
        <div class="search-wrap">
            <span class="search-icon">&#128269;</span>
            <input type="text" id="bookSearch" class="search-input"
                   placeholder="Search by title, author, category or ISBN...">
        </div>
        <span class="badge" id="bookCount"><?= count($books) ?> titles</span>
    </div>

    <div class="table-wrap">
        <table class="data-table">
            <thead>
                <tr>
                    <th>#</th><th>Title</th><th>Author</th><th>Category</th>
                    <th>ISBN</th><th>Copies</th><th>Price</th><th class="text-right">Actions</th>
                </tr>
            </thead>
            <tbody id="bookTable">
                <?php if (empty($books)): ?>
                    <tr><td colspan="8" class="empty">The catalogue is empty. Add the first book above.</td></tr>
                <?php else: ?>
                    <?php foreach ($books as $i => $b): ?>
                        <tr>
                            <td><?= $i + 1 ?></td>
                            <td><?= esc($b['title']) ?></td>
                            <td><?= esc($b['author']) ?></td>
                            <td><?= esc($b['category']) ?></td>
                            <td><?= esc($b['isbn']) ?></td>
                            <td>
                                <span class="pill <?= $b['quantity'] <= LOW_STOCK ? 'pill-suspended' : 'pill-
active' ?>">
                                    <?= (int)$b['quantity'] ?>
                                </span>
                            </td>
                            <td><?= esc(money($b['price'])) ?></td>
                            <td class="text-right">
                                <a class="btn-sm btn-edit"
                                   href="index.php?page=librarian&amp;action=edit&amp;id=<?= (int)$b['id']
?>">Edit</a>
                                <a class="btn-sm btn-delete"
                                   href="<?= esc(csrf_url('index.php?page=librarian&action=delete&id=' .
(int)$b['id'])) ?>"
                                   onclick="return confirm('Delete this book?');">Delete</a>
                            </td>
                        </tr>
                    <?php endforeach; ?>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
</div>

<!-- ============ FEATURE 1: issue and return desk ============ -->
<div class="card">
    <h3 class="card-title card-title-pad">Issue and return desk</h3>

    <div class="card-toolbar">
        <div class="search-wrap">
            <span class="search-icon">&#128269;</span>
            <input type="text" id="issueSearch" class="search-input"
                   placeholder="Search requests by book, student or status...">
```

```
        </div>
        <span class="badge" id="issueCount"><?= count($requests) ?> requests</span>
    </div>

    <div class="table-wrap">
        <table class="data-table">
            <thead>
                <tr>
                    <th>#</th><th>Student</th><th>Book</th><th>Pick-up</th>
                    <th>Due</th><th>Status</th><th>Fine</th><th class="text-right">Actions</th>
                </tr>
            </thead>
            <tbody id="issueTable">
                <?php if (empty($requests)): ?>
                    <tr><td colspan="8" class="empty">No student has asked for a book yet.</td></tr>
                <?php else: ?>
                    <?php foreach ($requests as $i => $r): ?>
                        <?php
                            // The fine grows every day a book is late.
                            $liveFine = ($r['status'] === 'issued')
                                ? calculate_fine($r['due_date'])
                                : (float)$r['fine'];
                        ?>
                        <tr>
                            <td><?= $i + 1 ?></td>
                            <td><?= esc($r['student_name']) ?><br><span class="sub"><?=
esc($r['student_username']) ?></span></td>
                            <td><?= esc($r['title']) ?></td>
                            <td class="nowrap"><?= esc(nice_date($r['pickup_date'])) ?></td>
                            <td class="nowrap"><?= esc(nice_date($r['due_date'])) ?></td>
                            <td><span class="pill pill-<?= esc($r['status']) ?>"><?= esc($r['status'])
?></span></td>
                            <td><?= esc(money($liveFine)) ?></td>
                            <td class="text-right">
                                <?php if ($r['status'] === 'pending'): ?>
                                    <a class="btn-sm btn-ok"
                                       href="<?= esc(csrf_url('index.php?page=librarian&action=issue&id=' .
(int)$r['id'])) ?>"
                                       onclick="return confirm('Issue this book for <?= (int)LOAN_DAYS ?>
days?');">Issue</a>
                                    <a class="btn-sm btn-delete"
                                       href="<?= esc(csrf_url('index.php?page=librarian&action=reject&id=' .
(int)$r['id'])) ?>">Reject</a>
                                <?php elseif ($r['status'] === 'issued'): ?>
                                    <a class="btn-sm btn-edit"
                                       href="<?= esc(csrf_url('index.php?page=librarian&action=return&id=' .
(int)$r['id'])) ?>"
                                       onclick="return confirm('Mark this book as returned?');">Return</a>
                                <?php else: ?>
                                    <span class="sub">Closed</span>
                                <?php endif; ?>
                            </td>
                        </tr>
                    <?php endforeach; ?>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
</div>

<!-- ============ FEATURE 2: low stock alerts ============ -->
<div class="card">
    <h3 class="card-title card-title-pad">
        Low stock alert
        <span class="label-hint">&mdash; <?= (int)LOW_STOCK ?> copies or fewer</span>
    </h3>
    <div class="table-wrap">
        <table class="data-table">
            <thead><tr><th>Title</th><th>Author</th><th>Category</th><th>Copies left</th></tr></thead>
            <tbody id="lowStockTable">
                <?php if (empty($lowStock)): ?>
                    <tr><td colspan="4" class="empty">Every book is well stocked.</td></tr>
                <?php else: ?>
                    <?php foreach ($lowStock as $b): ?>
                        <tr>
                            <td><?= esc($b['title']) ?></td>
                            <td><?= esc($b['author']) ?></td>
                            <td><?= esc($b['category']) ?></td>
                            <td><span class="pill pill-suspended"><?= (int)$b['quantity'] ?></span></td>
                        </tr>
                    <?php endforeach; ?>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
</div>

<?php require __DIR__ . '/../partials/footer.php'; ?>
<script>
/* ---------- AJAX 1: catalogue search ---------- */
liveSearch('bookSearch', function () {
    ajaxTable({
        url:     'index.php?page=ajax&action=search_books&q=' +
                 encodeURIComponent(document.getElementById('bookSearch').value.trim()),
        tbody:   'bookTable',
        counter: 'bookCount',
        columns: 8,
        word:    'titles',
        row: function (b, i) {
```

```
            var stockClass = (parseInt(b.quantity, 10) <= <?= (int)LOW_STOCK ?>) ? 'pill-suspended' : 'pill-
active';
            return '<tr>' +
                '<td>' + (i + 1) + '</td>' +
                '<td>' + esc(b.title) + '</td>' +
                '<td>' + esc(b.author) + '</td>' +
                '<td>' + esc(b.category) + '</td>' +
                '<td>' + esc(b.isbn) + '</td>' +
                '<td><span class="pill ' + stockClass + '">' + esc(b.quantity) + '</span></td>' +
                '<td><?= esc(CURRENCY) ?>' + parseFloat(b.price).toFixed(2) + '</td>' +
                '<td class="text-right">' +
                    '<a class="btn-sm btn-edit" href="index.php?page=librarian&action=edit&id=' + b.id +
'">Edit</a>' +
                    '<a class="btn-sm btn-delete" href="index.php?page=librarian&action=delete&id=' + b.id +
                    '&csrf_token=<?= csrf_token() ?>" onclick="return confirm(\'Delete this
book?\');">Delete</a>' +
                '</td>' +
            '</tr>';
        }
    });
});

/* ---------- AJAX 2: issue desk search ---------- */
liveSearch('issueSearch', function () {
    ajaxTable({
        url:     'index.php?page=ajax&action=search_issues&q=' +
                 encodeURIComponent(document.getElementById('issueSearch').value.trim()),
        tbody:   'issueTable',
        counter: 'issueCount',
        columns: 8,
        word:    'requests',
        row: function (r, i) {
            var token   = '&csrf_token=<?= csrf_token() ?>';
            var actions = '<span class="sub">Closed</span>';

            if (r.status === 'pending') {
                actions =
                    '<a class="btn-sm btn-ok" href="index.php?page=librarian&action=issue&id=' + r.id + token +
                    '" onclick="return confirm(\'Issue this book?\');">Issue</a>' +
                    '<a class="btn-sm btn-delete" href="index.php?page=librarian&action=reject&id=' + r.id +
token + '">Reject</a>';
            } else if (r.status === 'issued') {
                actions =
                    '<a class="btn-sm btn-edit" href="index.php?page=librarian&action=return&id=' + r.id + token
+
                    '" onclick="return confirm(\'Mark this book as returned?\');">Return</a>';
            }

            return '<tr>' +
                '<td>' + (i + 1) + '</td>' +
                '<td>' + esc(r.student_name) + '<br><span class="sub">' + esc(r.student_username) +
'</span></td>' +
                '<td>' + esc(r.title) + '</td>' +
                '<td class="nowrap">' + esc(r.pickup_date) + '</td>' +
                '<td class="nowrap">' + esc(r.due_date || '-') + '</td>' +
                '<td><span class="pill pill-' + esc(r.status) + '">' + esc(r.status) + '</span></td>' +
                '<td><?= esc(CURRENCY) ?>' + liveFine(r).toFixed(2) + '</td>' +
                '<td class="text-right">' + actions + '</td>' +
            '</tr>';
        }
    });
});
</script>
</body>
</html>
```

Line	Code	What it does and why
6	`$headerAction = '<a class="btn btn-ghost" href="index.php?page=librarian&action=export">Download catalogue (CSV)</a>';`	The download button is passed to the shared header as a small piece of HTML, so the layout can place it beside the page title without knowing what it is.
15	`<div class="stat-card <?= $stats['overdue'] > 0 ? 'stat-danger' : '' ?>"><span class="stat-value"><?= (int)$stats['overdue'] ?></span><span class="stat-label">Overdue</span></div>`	A statistic turns red only when its number is above zero. An overdue count of nought should not shout.
161	`$liveFine = ($r['status'] === 'issued')`	For a book still out, the fine shown is calculated fresh; for one already returned, the stored figure is used. The librarian sees the true amount to charge at this moment.
174	`<?php if ($r['status'] === 'pending'): ?>`	The action buttons depend on the stage: pending offers Issue and Reject, issued offers Return, anything finished offers nothing.

Line	Code	What it does and why
199	`Low stock alert`	**Librarian feature 2.** A short table of everything at or below the threshold, so restocking needs no searching.
235	`var stockClass = (parseInt(b.quantity, 10) <= <?= (int)LOW_STOCK ?>) ? 'pill-suspended' : 'pill-active';`	The same low-stock rule applied in JavaScript, using the PHP constant printed into the script, so search results and the first page agree.
285	`'<td><?= esc(CURRENCY) ?>' + liveFine(r).toFixed(2) + '</td>' +`	Search results use the shared JavaScript fine calculation, so a searched table shows the same number as the unsearched one.

8.6 The student dashboard

Full listing of **views/student/dashboard.php** (280 lines).

```
<?php
$pageTitle   = 'Student dashboard';
$pageHeading = 'My borrowing';
$pageSub     = 'Request books, follow your due dates and keep an eye on fines';
$isEdit      = !empty($editing);
$me          = current_user();
require __DIR__ . '/../partials/header.php';
?>

<!-- ============ FEATURE 2: due date and fine tracker ============ -->
<section class="stat-grid">
    <div class="stat-card"><span class="stat-value"><?= (int)$stats['pending'] ?></span><span class="stat-label">Waiting</span></div>
    <div class="stat-card"><span class="stat-value"><?= (int)$stats['issued'] ?></span><span class="stat-label">With me now</span></div>
    <div class="stat-card"><span class="stat-value"><?= (int)$stats['returned'] ?></span><span class="stat-label">Returned</span></div>
    <div class="stat-card <?= $stats['overdue'] > 0 ? 'stat-danger' : '' ?>"><span class="stat-value"><?= (int)$stats['overdue'] ?></span><span class="stat-label">Overdue</span></div>
    <div class="stat-card <?= $liveFine > 0 ? 'stat-danger' : '' ?>"><span class="stat-value"><?= esc(money($liveFine)) ?></span><span class="stat-label">Fine building up</span></div>
    <div class="stat-card"><span class="stat-value"><?= esc(money($stats['paid_fine'])) ?></span><span class="stat-label">Fines so far</span></div>
</section>

<div class="two-col">
    <!-- ============ CREATE / UPDATE: borrow request ============ -->
    <div class="card form-card">
        <h3 class="card-title">
            <?= $isEdit ? 'Edit request #' . (int)$editing['id'] : 'Request a book' ?>
        </h3>

        <form method="POST" class="form" novalidate onsubmit="return validateForm(this);"
              action="index.php?page=student&amp;action=<?= $isEdit ? 'update&amp;id=' . (int)$editing['id'] : 'add' ?>">
            <?php csrf_field(); ?>

            <div class="field">
                <label for="book_id">Book</label>
                <select id="book_id" name="book_id" data-label="Book" required>
                    <option value="">Choose a book...</option>
                    <?php foreach ($available as $b): ?>
                        <option value="<?= (int)$b['id'] ?>"
                            <?= ((int)($editing['book_id'] ?? 0) === (int)$b['id']) ? 'selected' : '' ?>>
                            <?= esc($b['title']) ?> &mdash; <?= esc($b['author']) ?> (<?= (int)$b['quantity'] ?> left)
                        </option>
                    <?php endforeach; ?>
                </select>
            </div>

            <div class="field">
                <label for="pickup_date">Pick-up date</label>
                <input type="date" id="pickup_date" name="pickup_date" data-label="Pick-up date"
                       min="<?= date('Y-m-d') ?>"
                       value="<?= esc($editing['pickup_date'] ?? date('Y-m-d')) ?>" required>
            </div>

            <div class="field">
                <label for="notes">Note for the librarian <span class="label-hint">(optional)</span></label>
                <input type="text" id="notes" name="notes" data-label="Note"
                       value="<?= esc($editing['notes'] ?? '') ?>"
                       placeholder="e.g. Needed for the CSE 3rd year project">
            </div>

            <div class="form-actions">
                <?php if ($isEdit): ?>
                    <a href="index.php?page=student" class="btn btn-ghost">Cancel</a>
                    <button type="submit" class="btn btn-primary">Save changes</button>
```

```
                <?php else: ?>
                    <button type="submit" class="btn btn-primary">Send request</button>
                <?php endif; ?>
            </div>
        </form>
    </div>

    <!-- ============ FEATURE 3: printable digital library card ============ -->
    <div class="card form-card">
        <h3 class="card-title">My library card</h3>

        <div class="library-card" id="libraryCard">
            <div class="lib-card-top">
                <span class="lib-card-brand"><?= esc(APP_NAME) ?></span>
                <span class="lib-card-type">Student</span>
            </div>
            <div class="lib-card-name"><?= esc($me['name']) ?></div>
            <div class="lib-card-rows">
                <span>Card no.</span><strong>STU-<?= str_pad((string)(int)$me['id'], 5, '0', STR_PAD_LEFT)
?></strong>
                <span>Username</span><strong><?= esc($me['username']) ?></strong>
                <span>Member since</span><strong><?= esc(nice_date($me['created_at'])) ?></strong>
                <span>Loan period</span><strong><?= (int)LOAN_DAYS ?> days</strong>
            </div>
            <div class="lib-card-foot">Fine after the due date: <?= esc(money(FINE_PER_DAY)) ?> per day</div>
        </div>

        <div class="form-actions">
            <button type="button" class="btn btn-ghost" onclick="window.print();">Print my card</button>
        </div>
    </div>
</div>

<!-- ============ READ + SEARCH: my requests ============ -->
<div class="card">
    <div class="card-toolbar">
        <div class="search-wrap">
            <span class="search-icon">&#128269;</span>
            <input type="text" id="requestSearch" class="search-input"
                   placeholder="Search my requests by book, author or status...">
        </div>
        <span class="badge" id="requestCount"><?= count($requests) ?> requests</span>
    </div>

    <div class="table-wrap">
        <table class="data-table">
            <thead>
                <tr>
                    <th>#</th><th>Book</th><th>Pick-up</th><th>Due date</th>
                    <th>Status</th><th>Fine</th><th class="text-right">Actions</th>
                </tr>
            </thead>
            <tbody id="requestTable">
                <?php if (empty($requests)): ?>
                    <tr><td colspan="7" class="empty">You have not requested a book yet. Use the form
above.</td></tr>
                <?php else: ?>
                    <?php foreach ($requests as $i => $r): ?>
                        <?php
                            $late = ($r['status'] === 'issued') ? days_late($r['due_date']) : 0;
                            $fine = ($r['status'] === 'issued') ? calculate_fine($r['due_date']) :
(float)$r['fine'];
                        ?>
                        <tr>
                            <td><?= $i + 1 ?></td>
                            <td><?= esc($r['title']) ?><br><span class="sub"><?= esc($r['author']) ?></span></td>
                            <td class="nowrap"><?= esc(nice_date($r['pickup_date'])) ?></td>
                            <td class="nowrap">
                                <?= esc(nice_date($r['due_date'])) ?>
                                <?php if ($r['status'] === 'issued'): ?>
                                    <br>
                                    <span class="sub <?= $late > 0 ? 'text-danger' : '' ?>">
                                        <?= $late > 0 ? $late . ' days late' : abs($late) . ' days left' ?>
                                    </span>
                                <?php endif; ?>
                            </td>
                            <td><span class="pill pill-<?= esc($r['status']) ?>"><?= esc($r['status'])
?></span></td>
                            <td><?= esc(money($fine)) ?></td>
                            <td class="text-right">
                                <?php if ($r['status'] === 'pending'): ?>
                                    <a class="btn-sm btn-edit"
                                       href="index.php?page=student&amp;action=edit&amp;id=<?= (int)$r['id']
?>">Edit</a>
                                    <a class="btn-sm btn-delete"
                                       href="<?= esc(csrf_url('index.php?page=student&action=delete&id=' .
(int)$r['id'])) ?>"
                                       onclick="return confirm('Cancel this request?');">Cancel</a>
                                <?php else: ?>
                                    <span class="sub">No action</span>
                                <?php endif; ?>
                            </td>
                        </tr>
                    <?php endforeach; ?>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
</div>

```

```
<!-- ============ FEATURE 1: catalogue browser ============ -->
<div class="card">
    <h3 class="card-title card-title-pad">Library catalogue</h3>

    <div class="card-toolbar">
        <div class="search-wrap">
            <span class="search-icon">&#128269;</span>
            <input type="text" id="catalogSearch" class="search-input"
                   placeholder="Search the whole catalogue...">
        </div>
        <span class="badge" id="catalogCount"><?= count($books) ?> titles</span>
    </div>

    <div class="table-wrap">
        <table class="data-table">
            <thead>
                <tr><th>Title</th><th>Author</th><th>Category</th><th>Availability</th><th class="text-
right">Action</th></tr>
            </thead>
            <tbody id="catalogTable">
                <?php if (empty($books)): ?>
                    <tr><td colspan="5" class="empty">The catalogue is empty right now.</td></tr>
                <?php else: ?>
                    <?php foreach ($books as $b): ?>
                        <tr>
                            <td><?= esc($b['title']) ?></td>
                            <td><?= esc($b['author']) ?></td>
                            <td><?= esc($b['category']) ?></td>
                            <td>
                                <?php if ((int)$b['quantity'] > 0): ?>
                                    <span class="pill pill-active"><?= (int)$b['quantity'] ?> on shelf</span>
                                <?php else: ?>
                                    <span class="pill pill-suspended">All copies out</span>
                                <?php endif; ?>
                            </td>
                            <td class="text-right">
                                <?php if ((int)$b['quantity'] > 0): ?>
                                    <button type="button" class="btn-sm btn-edit"
                                            onclick="pickBook(<?= (int)$b['id'] ?>)">Request</button>
                                <?php else: ?>
                                    <span class="sub">Unavailable</span>
                                <?php endif; ?>
                            </td>
                        </tr>
                    <?php endforeach; ?>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
</div>

<?php require __DIR__ . '/../partials/footer.php'; ?>
<script>
/* Clicking "Request" in the catalogue selects that book in the form above. */
function pickBook(id) {
    var select = document.getElementById('book_id');
    select.value = String(id);
    if (select.value === '') {
        alert('That book cannot be requested right now.');
        return;
    }
    select.scrollIntoView({ behavior: 'smooth', block: 'center' });
    select.focus();
}

/* ---------- AJAX 1: search my own requests ---------- */
liveSearch('requestSearch', function () {
    ajaxTable({
        url:     'index.php?page=ajax&action=search_requests&q=' +
                 encodeURIComponent(document.getElementById('requestSearch').value.trim()),
        tbody:   'requestTable',
        counter: 'requestCount',
        columns: 7,
        word:    'requests',
        row: function (r, i) {
            var actions = '<span class="sub">No action</span>';
            if (r.status === 'pending') {
                actions =
                    '<a class="btn-sm btn-edit" href="index.php?page=student&action=edit&id=' + r.id +
'">Edit</a>' +
                    '<a class="btn-sm btn-delete" href="index.php?page=student&action=delete&id=' + r.id +
                    '&csrf_token=<?= csrf_token() ?>" onclick="return confirm(\'Cancel this
request?\');">Cancel</a>';
            }
            return '<tr>' +
                '<td>' + (i + 1) + '</td>' +
                '<td>' + esc(r.title) + '<br><span class="sub">' + esc(r.author) + '</span></td>' +
                '<td class="nowrap">' + esc(r.pickup_date) + '</td>' +
                '<td class="nowrap">' + esc(r.due_date || '-') + '</td>' +
                '<td><span class="pill pill-' + esc(r.status) + '">' + esc(r.status) + '</span></td>' +
                '<td><?= esc(CURRENCY) ?>' + liveFine(r).toFixed(2) + '</td>' +
                '<td class="text-right">' + actions + '</td>' +
            '</tr>';
        }
    });
});

/* ---------- AJAX 2: search the catalogue ---------- */
liveSearch('catalogSearch', function () {
    ajaxTable({
```

```
        url:     'index.php?page=ajax&action=search_books&q=' +
                 encodeURIComponent(document.getElementById('catalogSearch').value.trim()),
        tbody:   'catalogTable',
        counter: 'catalogCount',
        columns: 5,
        word:    'titles',
        row: function (b) {
            var left = parseInt(b.quantity, 10);
            var availability = (left > 0)
                ? '<span class="pill pill-active">' + left + ' on shelf</span>'
                : '<span class="pill pill-suspended">All copies out</span>';
            var action = (left > 0)
                ? '<button type="button" class="btn-sm btn-edit" onclick="pickBook(' + b.id +
')">Request</button>'
                : '<span class="sub">Unavailable</span>';

            return '<tr>' +
                '<td>' + esc(b.title) + '</td>' +
                '<td>' + esc(b.author) + '</td>' +
                '<td>' + esc(b.category) + '</td>' +
                '<td>' + availability + '</td>' +
                '<td class="text-right">' + action + '</td>' +
            '</tr>';
        }
    });
});
</script>
</body>
</html>
```

Line	Code	What it does and why
16	`<div class="stat-card <?= $liveFine > 0 ? 'stat-danger' : '' ?>"><span class="stat-value"><?= esc(money($liveFine)) ?></span><span class="stat-label">Fine building up</span></div>`	The fine card turns red only when money is actually owed.
34	`<option value="">Choose a book...</option>`	An empty first option forces a deliberate choice instead of silently requesting whichever book happens to be first.
38	`<?= esc($b['title']) ?> &mdash; <?= esc($b['author']) ?> (<?= (int)$b['quantity'] ?> left)`	Each option shows how many copies remain, so the decision is informed before it is made.
47	`min="<?= date('Y-m-d') ?>"`	The date box refuses past dates in the browser. The server checks the same rule again.
73	`<div class="library-card" id="libraryCard">`	**Student feature 3.** The digital library card, built from data already on the page — no image, no download, nothing stored.
80	`<span>Card no.</span><strong>STU-<?= str_pad((string)(int)$me['id'], 5, '0', STR_PAD_LEFT) ?></strong>`	Turns user 42 into the card number `STU-00042`. Padding is cosmetic, but a card that looks like a card is more convincing.
89	`<button type="button" class="btn btn-ghost" onclick="window.print();">Print my card</button>`	Opens the browser print dialog. A `@media print` block in the stylesheet hides the navigation, tables and forms, so only the card reaches the paper.
131	`<?= $late > 0 ? $late . ' days late' : abs($late) . ' days left' ?>`	**Student feature 2 in one line.** The same number is phrased as a warning or as reassurance depending on its sign.
209	`function pickBook(id) {`	**Student feature 1.** Clicking Request in the catalogue selects that book in the form above and scrolls to it, turning two screens into one action.
216	`select.scrollIntoView({ behavior: 'smooth', block: 'center' });`	A gentle scroll rather than a jump, so it is obvious that the page moved and why.

8.7 The visitor dashboard

Full listing of **views/visitor/dashboard.php** (273 lines).

```
<?php
$pageTitle   = 'Visitor dashboard';
$pageHeading = 'My library visits';
$pageSub     = 'Book a day pass, apply for membership and suggest books to buy';
$isEdit      = !empty($editing);
$me          = current_user();

```

```
 8  // The newest pass that is still valid, shown as the printable pass below.
 9  $activePass = null;
10  foreach ($passes as $p) {
11      if ($p['status'] === 'booked' && strtotime($p['visit_date']) >= strtotime(date('Y-m-d'))) {
12          $activePass = $p;
13          break;
14      }
15  }
16  require __DIR__ . '/../partials/header.php';
17  ?>
18
19  <section class="stat-grid">
20      <div class="stat-card"><span class="stat-value"><?= (int)$stats['booked'] ?></span><span class="stat-
label">Passes booked</span></div>
21      <div class="stat-card"><span class="stat-value"><?= (int)$stats['upcoming'] ?></span><span class="stat-
label">Still upcoming</span></div>
22      <div class="stat-card"><span class="stat-value"><?= (int)$stats['suggestions'] ?></span><span class="stat-
label">Books suggested</span></div>
23      <div class="stat-card">
24          <span class="stat-value stat-text">
25              <?= $application ? esc(ucfirst($application['status'])) : 'None' ?>
26          </span>
27          <span class="stat-label">Membership</span>
28      </div>
29  </section>
30
31  <div class="two-col">
32      <!-- ============ CREATE / UPDATE: visit pass ============ -->
33      <div class="card form-card">
34          <h3 class="card-title">
35              <?= $isEdit ? 'Edit pass #' . (int)$editing['id'] : 'Book a visit' ?>
36          </h3>
37
38          <form method="POST" class="form" novalidate onsubmit="return validateForm(this);"
39                action="index.php?page=visitor&amp;action=<?= $isEdit ? 'update&amp;id=' . (int)$editing['id'] :
'add' ?>">
40              <?php csrf_field(); ?>
41
42              <div class="field">
43                  <label for="visit_date">Visit date</label>
44                  <input type="date" id="visit_date" name="visit_date" data-label="Visit date"
45                         min="<?= date('Y-m-d') ?>"
46                         value="<?= esc($editing['visit_date'] ?? date('Y-m-d')) ?>" required>
47              </div>
48
49              <div class="field">
50                  <label for="purpose">Purpose of the visit</label>
51                  <input type="text" id="purpose" name="purpose" data-label="Purpose" data-min="3"
52                         value="<?= esc($editing['purpose'] ?? '') ?>"
53                         placeholder="e.g. Reading room and newspaper archive" required>
54              </div>
55
56              <div class="field-row">
57                  <div class="field">
58                      <label for="guests">People coming</label>
59                      <input type="number" id="guests" name="guests" data-label="People coming"
60                             min="1" max="10" step="1"
61                             value="<?= esc($editing['guests'] ?? 1) ?>" required>
62                  </div>
63                  <?php if ($isEdit): ?>
64                      <div class="field">
65                          <label for="status">Status</label>
66                          <select id="status" name="status" data-label="Status" required>
67                              <option value="booked"    <?= (($editing['status'] ?? '') === 'booked')    ?
'selected' : '' ?>>Booked</option>
68                              <option value="cancelled" <?= (($editing['status'] ?? '') === 'cancelled') ?
'selected' : '' ?>>Cancelled</option>
69                          </select>
70                      </div>
71                  <?php endif; ?>
72              </div>
73
74              <div class="form-actions">
75                  <?php if ($isEdit): ?>
76                      <a href="index.php?page=visitor" class="btn btn-ghost">Cancel</a>
77                      <button type="submit" class="btn btn-primary">Save changes</button>
78                  <?php else: ?>
79                      <button type="submit" class="btn btn-primary">Book the pass</button>
80                  <?php endif; ?>
81              </div>
82          </form>
83      </div>
84
85      <!-- ============ FEATURE 3: printable day pass ============ -->
86      <div class="card form-card">
87          <h3 class="card-title">My day pass</h3>
88
89          <?php if ($activePass): ?>
90              <div class="library-card pass-card">
91                  <div class="lib-card-top">
92                      <span class="lib-card-brand"><?= esc(APP_NAME) ?></span>
93                      <span class="lib-card-type">Day pass</span>
94                  </div>
95                  <div class="pass-code"><?= esc($activePass['pass_code']) ?></div>
96                  <div class="lib-card-rows">
97                      <span>Name</span><strong><?= esc($me['name']) ?></strong>
98                      <span>Visit date</span><strong><?= esc(nice_date($activePass['visit_date'])) ?></strong>
99                      <span>People</span><strong><?= (int)$activePass['guests'] ?></strong>
100                     <span>Purpose</span><strong><?= esc($activePass['purpose']) ?></strong>
101                 </div>
```

```
                <div class="lib-card-foot">Show this code at the front desk.</div>
            </div>
            <div class="form-actions">
                <button type="button" class="btn btn-ghost" onclick="window.print();">Print this pass</button>
            </div>
        <?php else: ?>
            <p class="muted">Book a visit on the left and your pass code will appear here.</p>
        <?php endif; ?>
    </div>
</div>

<!-- ============ READ + SEARCH: my passes ============ -->
<div class="card">
    <div class="card-toolbar">
        <div class="search-wrap">
            <span class="search-icon">&#128269;</span>
            <input type="text" id="passSearch" class="search-input"
                   placeholder="Search my passes by date, code or purpose...">
        </div>
        <span class="badge" id="passCount"><?= count($passes) ?> passes</span>
    </div>

    <div class="table-wrap">
        <table class="data-table">
            <thead>
                <tr><th>#</th><th>Pass code</th><th>Visit date</th><th>Purpose</th>
                    <th>People</th><th>Status</th><th class="text-right">Actions</th></tr>
            </thead>
            <tbody id="passTable">
                <?php if (empty($passes)): ?>
                    <tr><td colspan="7" class="empty">You have not booked a visit yet.</td></tr>
                <?php else: ?>
                    <?php foreach ($passes as $i => $p): ?>
                        <tr>
                            <td><?= $i + 1 ?></td>
                            <td class="nowrap"><strong><?= esc($p['pass_code']) ?></strong></td>
                            <td class="nowrap"><?= esc(nice_date($p['visit_date'])) ?></td>
                            <td><?= esc($p['purpose']) ?></td>
                            <td><?= (int)$p['guests'] ?></td>
                            <td><span class="pill pill-<?= $p['status'] === 'booked' ? 'active' : 'rejected'
?>"><?= esc($p['status']) ?></span></td>
                            <td class="text-right">
                                <a class="btn-sm btn-edit"
                                   href="index.php?page=visitor&amp;action=edit&amp;id=<?= (int)$p['id']
?>">Edit</a>
                                <a class="btn-sm btn-delete"
                                   href="<?= esc(csrf_url('index.php?page=visitor&action=delete&id=' .
(int)$p['id'])) ?>"
                                   onclick="return confirm('Delete this pass?');">Delete</a>
                            </td>
                        </tr>
                    <?php endforeach; ?>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
</div>

<div class="two-col">
    <!-- ============ FEATURE 1: membership application ============ -->
    <div class="card form-card">
        <h3 class="card-title">Become a student member</h3>

        <?php if ($application && $application['status'] === 'pending'): ?>
            <p class="muted">Your application is with the administrator.</p>
            <div class="info-box">
                <span class="pill pill-pending">Pending</span>
                <p><?= esc($application['reason']) ?></p>
                <span class="sub">Sent <?= esc(nice_date($application['applied_at'])) ?></span>
            </div>
        <?php elseif ($application && $application['status'] === 'approved'): ?>
            <div class="info-box">
                <span class="pill pill-active">Approved</span>
                <p>Sign out and sign back in to open your student dashboard.</p>
            </div>
        <?php else: ?>
            <?php if ($application && $application['status'] === 'rejected'): ?>
                <div class="info-box">
                    <span class="pill pill-rejected">Rejected</span>
                    <p>Your last application was turned down. You may apply again.</p>
                </div>
            <?php endif; ?>
            <p class="muted">Members can borrow books and take them home.</p>
            <form method="POST" action="index.php?page=visitor&amp;action=apply" class="form"
                  novalidate onsubmit="return validateForm(this);">
                <?php csrf_field(); ?>
                <div class="field">
                    <label for="reason">Why do you want to join?</label>
                    <input type="text" id="reason" name="reason" data-label="Reason" data-min="10"
                           placeholder="At least 10 characters" required>
                </div>
                <button type="submit" class="btn btn-primary btn-block">Send application</button>
            </form>
        <?php endif; ?>
    </div>

    <!-- ============ FEATURE 2: book suggestion box ============ -->
    <div class="card form-card">
        <h3 class="card-title">Suggest a book to buy</h3>

```

```
        <form method="POST" action="index.php?page=visitor&amp;action=suggest" class="form"
              novalidate onsubmit="return validateForm(this);">
            <?php csrf_field(); ?>
            <div class="field-row">
                <div class="field">
                    <label for="s_title">Title</label>
                    <input type="text" id="s_title" name="s_title" data-label="Title" data-min="2"
                           placeholder="Book title" required>
                </div>
                <div class="field">
                    <label for="s_author">Author</label>
                    <input type="text" id="s_author" name="s_author" data-label="Author" data-min="2"
                           placeholder="Author name" required>
                </div>
            </div>
            <div class="field">
                <label for="s_reason">Why this book? <span class="label-hint">(optional)</span></label>
                <input type="text" id="s_reason" name="s_reason" data-label="Reason"
                       placeholder="e.g. Many students ask for it">
            </div>
            <button type="submit" class="btn btn-primary btn-block">Add suggestion</button>
        </form>

        <div class="suggestion-list">
            <?php if (empty($suggestions)): ?>
                <p class="sub">You have not suggested a book yet.</p>
            <?php else: ?>
                <?php foreach ($suggestions as $s): ?>
                    <div class="suggestion-item">
                        <div>
                            <strong><?= esc($s['title']) ?></strong>
                            <span class="sub">by <?= esc($s['author']) ?></span>
                        </div>
                        <a class="btn-sm btn-delete"
                           href="<?= esc(csrf_url('index.php?page=visitor&action=unsuggest&id=' . (int)$s['id'])) ?>"
                           onclick="return confirm('Remove this suggestion?');">Remove</a>
                    </div>
                <?php endforeach; ?>
            <?php endif; ?>
        </div>
    </div>
</div>

<?php require __DIR__ . '/../partials/footer.php'; ?>
<script>
/* ---------- AJAX: search my visit passes ---------- */
liveSearch('passSearch', function () {
    ajaxTable({
        url:     'index.php?page=ajax&action=search_passes&q=' +
                 encodeURIComponent(document.getElementById('passSearch').value.trim()),
        tbody:   'passTable',
        counter: 'passCount',
        columns: 7,
        word:    'passes',
        row: function (p, i) {
            var pill = (p.status === 'booked') ? 'pill-active' : 'pill-rejected';
            return '<tr>' +
                '<td>' + (i + 1) + '</td>' +
                '<td class="nowrap"><strong>' + esc(p.pass_code) + '</strong></td>' +
                '<td class="nowrap">' + esc(p.visit_date) + '</td>' +
                '<td>' + esc(p.purpose) + '</td>' +
                '<td>' + esc(p.guests) + '</td>' +
                '<td><span class="pill ' + pill + '">' + esc(p.status) + '</span></td>' +
                '<td class="text-right">' +
                    '<a class="btn-sm btn-edit" href="index.php?page=visitor&action=edit&id=' + p.id +
'">Edit</a>' +
                    '<a class="btn-sm btn-delete" href="index.php?page=visitor&action=delete&id=' + p.id +
                    '&csrf_token=<?= csrf_token() ?>" onclick="return confirm(\'Delete this
pass?\');">Delete</a>' +
                '</td>' +
            '</tr>';
        }
    });
});
</script>
</body>
</html>
```

Line	Code	What it does and why
9	`$activePass = null;`	Finds the next pass worth printing: still booked, and not in the past.
10	`foreach ($passes as $p) {`	The list is already sorted newest-first, so the first match is the right one and the loop stops there.
90	`<div class="library-card pass-card">`	**Visitor feature 3.** The printable day pass, reusing the student card's styling with one extra class for the large code.
95	`<div class="pass-code"><?= esc($activePass['pass_code']) ?></div>`	The code in large monospaced letters, sized to be read across a desk.

Line	Code	What it does and why
169	`<?php elseif ($application && $application['status'] === 'approved'): ?>`	**Visitor feature 1.** The application area shows one of four states: never applied, waiting, approved, or rejected — and only the states that make sense offer the form again.
172	`<p>Sign out and sign back in to open your student dashboard.</p>`	An approved visitor is told exactly what to do next. Their role has changed, but their current session still says visitor.
205	`<input type="text" id="s_title" name="s_title" data-label="Title" data-min="2"`	**Visitor feature 2.** The suggestion form. The `s_` prefix keeps its fields separate from the pass form on the same page.

Part 9 — The JavaScript and the stylesheet

The two files that run inside the browser rather than on the server.

9.1 assets/js/app.js

Purpose. Four shared tools used by every screen: form validation, escaping, the debounced search, and the AJAX table redraw. Every dashboard then adds a short script of its own that says what a row should look like.

Full listing of **assets/js/app.js** (172 lines).

```
/* ================================================================
   SHARED JAVASCRIPT
   1. validateForm()  - client side validation before a form is sent
   2. esc()           - escapes text before it is put into the page
   3. liveSearch()    - waits until typing stops, then runs a search
   4. ajaxTable()     - fetches JSON and redraws one table body
   Remember: the PHP side validates everything again. JavaScript is
   only here to give a fast answer to the user.
   ================================================================ */


/* ---------------- 1. Form validation ---------------- */

// Shows a red message under one input.
function showFieldError(input, message) {
    input.classList.add('is-invalid');
    var note = document.createElement('span');
    note.className = 'field-error';
    note.textContent = message;
    input.parentNode.appendChild(note);
}

function clearFieldErrors(form) {
    form.querySelectorAll('.field-error').forEach(function (el) { el.remove(); });
    form.querySelectorAll('.is-invalid').forEach(function (el) { el.classList.remove('is-invalid'); });
}

// Rules are written on the inputs themselves:
//   required            -> must not be empty
//   data-min="6"        -> minimum number of characters
//   data-match="pwd"    -> must equal the field called pwd
//   data-phone="1"      -> must look like a phone number
//   type="email"        -> must look like an email address
//   type="number" + min -> must be a number inside the range
//   data-label="Name"   -> friendly name used in the message
function validateForm(form) {
    clearFieldErrors(form);
    var valid = true;
    var firstBad = null;

    form.querySelectorAll('input, select, textarea').forEach(function (input) {
        if (input.type === 'hidden' || input.type === 'checkbox' || input.disabled) { return; }

        var value = (input.value || '').trim();
        var label = input.getAttribute('data-label') || input.name || 'This field';
        var message = '';

        if (input.hasAttribute('required') && value === '') {
            message = label + ' is required.';

        } else if (value !== '' && input.dataset.min && value.length < parseInt(input.dataset.min, 10)) {
            message = label + ' needs at least ' + input.dataset.min + ' characters.';

        } else if (value !== '' && input.type === 'email' &&
                   !/^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/.test(value)) {
            message = 'Enter a valid email address.';

        } else if (value !== '' && input.dataset.phone &&
                   !/^[0-9+\-\s()]{6,20}$/.test(value)) {
            message = 'Enter a valid contact number.';

        } else if (value !== '' && input.dataset.match) {
            var other = form.querySelector('[name="' + input.dataset.match + '"]');
            if (other && value !== other.value.trim()) {
                message = 'The two passwords do not match.';
            }

        } else if (value !== '' && input.type === 'number') {
            var num = parseFloat(value);
            if (isNaN(num)) {
                message = label + ' must be a number.';
            } else if (input.min !== '' && num < parseFloat(input.min)) {
                message = label + ' cannot be less than ' + input.min + '.';
            } else if (input.max !== '' && num > parseFloat(input.max)) {
                message = label + ' cannot be more than ' + input.max + '.';
            }

        } else if (value !== '' && input.type === 'date' && input.min && value < input.min) {
            message = label + ' cannot be in the past.';
        }
```

```
81
82          if (message) {
83              showFieldError(input, message);
84              valid = false;
85              if (!firstBad) { firstBad = input; }
86          }
87      });
88
89      if (firstBad) { firstBad.focus(); }
90      return valid;   // false stops the form from being submitted
91  }
92
93
94  /* ---------------- 2. Escaping ---------------- */
95
96  // SECURITY: any text coming back from the server is escaped before it
97  // is written into the page, so a title like <script> stays harmless.
98  function esc(text) {
99      return String(text === null || text === undefined ? '' : text)
100         .replace(/&/g, '&amp;')
101         .replace(/</g, '&lt;')
102         .replace(/>/g, '&gt;')
103         .replace(/"/g, '&quot;')
104         .replace(/'/g, '&#039;');
105 }
106
107
108 /* ---------------- 2b. Fine that is still growing ---------------- */
109
110 // A book that is out and past its due date keeps collecting a fine, so the
111 // number in an AJAX-refreshed table is worked out here instead of being read
112 // from the database.  FINE_RATE is printed by the page that loads this file.
113 function liveFine(request) {
114     if (request.status !== 'issued' || !request.due_date) {
115         return parseFloat(request.fine) || 0;
116     }
117     var due  = new Date(request.due_date + 'T00:00:00');
118     var days = Math.floor((Date.now() - due.getTime()) / 86400000);
119     return days > 0 ? days * (window.FINE_RATE || 0) : 0;
120 }
121
122
123 /* ---------------- 3. Live search ---------------- */
124
125 // Waits 250 ms after the last keystroke, then calls handler().
126 function liveSearch(inputId, handler) {
127     var input = document.getElementById(inputId);
128     if (!input) { return; }
129
130     var timer;
131     input.addEventListener('input', function () {
132         clearTimeout(timer);
133         timer = setTimeout(handler, 250);
134     });
135 }
136
137
138 /* ---------------- 4. AJAX table redraw ---------------- */
139
140 // options = { url, tbody, counter, columns, word, row }
141 //   row(item, index) must return one <tr> string.
142 function ajaxTable(options) {
143     var tbody   = document.getElementById(options.tbody);
144     var counter = document.getElementById(options.counter);
145     if (!tbody) { return; }
146
147     fetch(options.url, { credentials: 'same-origin' })
148         .then(function (response) { return response.json(); })
149         .then(function (rows) {
150
151             if (rows && rows.error) {
152                 tbody.innerHTML = '<tr><td colspan="' + options.columns + '" class="empty">' +
153                                   esc(rows.error) + '</td></tr>';
154                 return;
155             }
156             if (!rows.length) {
157                 tbody.innerHTML = '<tr><td colspan="' + options.columns + '" class="empty">' +
158                                   'Nothing matches your search.</td></tr>';
159                 if (counter) { counter.textContent = '0 ' + options.word; }
160                 return;
161             }
162
163             var html = '';
164             rows.forEach(function (item, index) { html += options.row(item, index); });
165             tbody.innerHTML = html;
166
167             if (counter) { counter.textContent = rows.length + ' ' + options.word; }
168         })
169         .catch(function (err) {
170             console.error('Search failed:', err);
171         });
172 }
```

Line by line

Line	Code	What it does and why
15	`function showFieldError(input, message) {`	Adds a red outline and a message directly under the offending box. Placing the message beside the field, rather than in one list at the top, means the user never has to hunt for what went wrong.
23	`function clearFieldErrors(form) {`	Wipes the previous attempt's messages so errors cannot pile up on repeated submissions.
36	`function validateForm(form) {`	**One function validates every form in the project.** It does not know about books or passes or users; it reads the rules from attributes on the inputs themselves.
41	`form.querySelectorAll('input, select, textarea').forEach(function (input) {`	Walks every field in the submitted form.
42	`if (input.type === 'hidden' || input.type === 'checkbox' || input.disabled) { return; }`	Skips hidden fields, tick boxes and disabled boxes, none of which the user can be asked to correct.
45	`var label = input.getAttribute('data-label') || input.name || 'This field';`	The friendly name used in messages, so the user reads "Full name is required" rather than a database column name.
48	`if (input.hasAttribute('required') && value === '') {`	Rule 1: required and empty.
51	`} else if (value !== '' && input.dataset.min && value.length < parseInt(input.dataset.min, 10)) {`	Rule 2: minimum length, read from `data-min`.
54	`} else if (value !== '' && input.type === 'email' &&`	Rule 3: a simple email shape. Deliberately simple, because the server uses the authoritative check.
58	`} else if (value !== '' && input.dataset.phone &&`	Rule 4: the same phone pattern the server uses, so the two never disagree.
63	`var other = form.querySelector('[name="' + input.dataset.match + '"]');`	Rule 5: this box must equal another named box — how the two password fields are compared.
68	`} else if (value !== '' && input.type === 'number') {`	Rule 6: numbers, including the `min` and `max` limits already written on the input.
78	`} else if (value !== '' && input.type === 'date' && input.min && value < input.min) {`	Rule 7: dates not before the allowed earliest. Dates in `YYYY-MM-DD` form compare correctly as plain text, which is one of the reasons that format is used everywhere.
89	`if (firstBad) { firstBad.focus(); }`	The cursor jumps to the first problem, so the fix can be typed straight away.
90	`return valid;   // false stops the form from being submitted`	Returning false from `onsubmit` cancels the submission. Returning true lets it through — to a server that will check everything again.
98	`function esc(text) {`	**The browser twin of PHP's `esc()`.** Every value that arrives as JSON passes through here before it is written into the page.
100	`.replace(/&/g, '&')`	The ampersand must be replaced first. Doing it later would corrupt the codes produced by the other replacements.
113	`function liveFine(request) {`	The fine calculation, mirrored in the browser so a searched table shows the same growing figure as the page did when it loaded.
119	`return days > 0 ? days * (window.FINE_RATE || 0) : 0;`	The rate is printed into the page by PHP rather than hard-coded here, so there is still only one place to change it.
126	`function liveSearch(inputId, handler) {`	**Debouncing, reusable.** Waits 250 milliseconds after the last keystroke before acting.
132	`clearTimeout(timer);`	Each keystroke cancels the pending search and starts the wait again, so a fast typist sends one request rather than one per letter.
142	`function ajaxTable(options) {`	The shared redraw. Given an address, a table body, a counter and a row-building function, it does the rest.

Line	Code	What it does and why
147	`fetch(options.url, { credentials: 'same-origin' })`	`credentials: same-origin` tells the browser to send the session cookie, without which the server would treat the request as a stranger and refuse it.
151	`if (rows && rows.error) {`	The server's refusal messages are shown inside the table instead of failing silently, so an expired session is visible rather than mysterious.
156	`if (!rows.length) {`	An empty result gets a plain "Nothing matches your search." and a zero counter.
164	`rows.forEach(function (item, index) { html += options.row(item, index); });`	The rows are assembled into one string and written once. Writing them one at a time would make the browser redraw the page repeatedly.

9.2 assets/css/style.css

Purpose. One stylesheet controls every screen. It is organised into labelled sections, and it opens with a block of variables so the whole colour scheme can be changed from one place.

CSS decides only how things look, never what they do, so it is explained by section rather than line by line.

Section	What it controls
`:root` variables	Every colour, shadow, corner radius and transition in the project, named once. Changing `--primary` restyles buttons, links, avatars and gradients together. This is the single most useful thing to experiment with.
Auth pages	The two-panel sign-in and sign-up screens: the purple gradient background, the blurred coloured circles behind the card, and the collapse to a single column on a narrow phone.
Forms	Inputs, labels, the two-across row that becomes one-across on small screens, and the red outline plus message that JavaScript adds when a field is wrong.
Buttons	The gradient primary button, the outlined secondary, and the small row buttons — edit, delete, approve, suspend — each in its own colour so an action can be recognised without reading it.
Alerts	The green and red message bars, with a short slide-down animation so a new message is noticed.
App layout	The sticky navigation bar, the user pill with its letter avatar, the centred content column, and the page heading block.
Statistic cards	The row of numbers at the top of each dashboard, including the amber and red variants used when a number needs attention.
Cards and toolbars	The white panels everything sits in, and the toolbar strip that holds a search box, a filter and a count.
Status pills	The small coloured badges for roles and statuses. The class name is built from the data, so the colour follows the value automatically.
Tables	Header styling, row hover, the horizontal scroll that keeps wide tables usable on a phone, and the italic "nothing here yet" row.
Library card and pass	The gradient card used for the student's library card and the visitor's day pass, plus the large monospaced pass code.
`@media print`	The print rules. When the user prints, the navigation, tables, forms and statistics are hidden so only the card or pass appears on the paper.

Section	What it controls
`@media` breakpoints	Three widths — 900px, 720px and 540px — where two-column layouts fold into one and non-essential text is hidden.

Full listing of **assets/css/style.css** (486 lines).

```
/* ============================================================
   Library Management System - Stylesheet
   Sections: variables, auth pages, forms, buttons, alerts,
             app layout, cards, stats, tables, cards to print
   ============================================================ */

:root {
    --primary: #6366f1;
    --primary-dark: #4f46e5;
    --accent: #8b5cf6;
    --bg-grad-start: #667eea;
    --bg-grad-end: #764ba2;

    --text: #1e293b;
    --text-muted: #64748b;
    --text-light: #94a3b8;

    --bg: #f8fafc;
    --card-bg: #ffffff;
    --border: #e2e8f0;
    --border-strong: #cbd5e1;

    --success: #10b981;
    --success-bg: #d1fae5;
    --warn: #f59e0b;
    --warn-bg: #fef3c7;
    --error: #ef4444;
    --error-bg: #fee2e2;
    --info-bg: #eef2ff;

    --shadow-sm: 0 1px 2px rgba(15,23,42,.06), 0 1px 3px rgba(15,23,42,.04);
    --shadow-md: 0 4px 6px -1px rgba(15,23,42,.08), 0 2px 4px -2px rgba(15,23,42,.05);
    --shadow-xl: 0 25px 50px -12px rgba(0,0,0,.25);

    --radius-sm: 8px;
    --radius: 12px;
    --radius-lg: 16px;
    --radius-xl: 24px;

    --transition: all .2s ease;
}

* { margin: 0; padding: 0; box-sizing: border-box; }
html, body { height: 100%; }
body {
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Oxygen, "Helvetica Neue", sans-serif;
    color: var(--text);
    line-height: 1.5;
    -webkit-font-smoothing: antialiased;
}
a { color: var(--primary); text-decoration: none; transition: var(--transition); }
a:hover { color: var(--primary-dark); }

/* ================== Auth pages ================== */
.auth-body {
    background: linear-gradient(135deg, var(--bg-grad-start) 0%, var(--bg-grad-end) 100%);
    min-height: 100vh;
    display: flex; align-items: center; justify-content: center;
    padding: 24px; position: relative; overflow: hidden;
}
.auth-body::before, .auth-body::after {
    content: ''; position: absolute; border-radius: 50%;
    filter: blur(80px); opacity: .4; pointer-events: none;
}
.auth-body::before { width: 400px; height: 400px; background: #f0abfc; top: -100px; left: -100px; }
.auth-body::after  { width: 500px; height: 500px; background: #60a5fa; bottom: -150px; right: -150px; }

.auth-shell {
    position: relative; z-index: 1; width: 100%; max-width: 1000px;
    background: var(--card-bg); border-radius: var(--radius-xl);
    box-shadow: var(--shadow-xl); overflow: hidden;
    display: grid; grid-template-columns: 1fr 1fr;
    min-height: 600px;
}

.auth-side {
    background: linear-gradient(135deg, var(--primary), var(--accent));
    color: #fff; padding: 48px;
    display: flex; flex-direction: column; justify-content: center;
    position: relative; overflow: hidden;
}
.auth-side::after {
    content: ''; position: absolute;
    width: 300px; height: 300px; border-radius: 50%;
    background: rgba(255,255,255,.1);
    bottom: -100px; right: -100px;
}
.auth-side > * { position: relative; z-index: 1; }
```

```
89
90  .logo-big {
91      width: 64px; height: 64px; border-radius: 16px;
92      background: rgba(255,255,255,.2);
93      display: flex; align-items: center; justify-content: center;
94      font-size: 32px; margin-bottom: 24px;
95  }
96  .auth-side h1 { font-size: 30px; font-weight: 700; margin-bottom: 12px; line-height: 1.2; }
97  .auth-side > p { opacity: .9; font-size: 15px; margin-bottom: 28px; }
98  .side-note { margin-top: 24px; font-size: 13px; opacity: .8; }
99
100 .feature-list { list-style: none; }
101 .feature-list li { padding: 7px 0; font-size: 14px; opacity: .95; }
102
103 .auth-form-wrap { padding: 48px; display: flex; align-items: center; justify-content: center; }
104 .auth-card { width: 100%; max-width: 400px; }
105 .auth-card h2 { font-size: 28px; font-weight: 700; margin-bottom: 6px; }
106 .muted { color: var(--text-muted); margin-bottom: 22px; font-size: 14px; }
107
108 .auth-foot { text-align: center; margin-top: 20px; color: var(--text-muted); font-size: 14px; }
109 .hint {
110     margin-top: 16px; padding: 12px; background: #f1f5f9;
111     border-radius: var(--radius-sm); font-size: 12px;
112     color: var(--text-muted); text-align: center;
113 }
114
115 @media (max-width: 768px) {
116     .auth-shell { grid-template-columns: 1fr; }
117     .auth-side { padding: 32px; }
118     .auth-side h1 { font-size: 24px; }
119     .auth-form-wrap { padding: 32px 24px; }
120 }
121
122 /* ================== Forms ================== */
123 .form { display: flex; flex-direction: column; gap: 16px; }
124 .field { display: flex; flex-direction: column; gap: 6px; }
125 .field label { font-size: 13px; font-weight: 600; color: var(--text); }
126 .label-hint { font-weight: 400; color: var(--text-light); font-size: 12px; }
127
128 .field input, .field select {
129     padding: 11px 14px;
130     border: 1.5px solid var(--border);
131     border-radius: var(--radius-sm);
132     font-size: 14px; font-family: inherit;
133     transition: var(--transition);
134     background: #fff; color: var(--text); width: 100%;
135 }
136 .field select { cursor: pointer; }
137 .field input:focus, .field select:focus {
138     outline: none; border-color: var(--primary);
139     box-shadow: 0 0 0 3px rgba(99,102,241,.15);
140 }
141
142 /* Validation feedback drawn by app.js */
143 .field input.is-invalid, .field select.is-invalid {
144     border-color: var(--error); background: #fff7f7;
145 }
146 .field-error { color: var(--error); font-size: 12px; font-weight: 500; }
147 .field-note  { font-size: 12px; color: var(--text-muted); min-height: 16px; }
148 .note-ok  { color: var(--success); }
149 .note-bad { color: var(--error); }
150
151 .field-row { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; }
152 @media (max-width: 540px) { .field-row { grid-template-columns: 1fr; } }
153
154 .checkbox {
155     display: flex; align-items: center; gap: 8px;
156     font-size: 13px; color: var(--text-muted);
157     cursor: pointer; user-select: none;
158 }
159 .checkbox input { accent-color: var(--primary); }
160
161 .form-actions { display: flex; justify-content: flex-end; gap: 10px; margin-top: 12px; }
162
163 /* ================== Buttons ================== */
164 .btn {
165     display: inline-flex; align-items: center; justify-content: center;
166     gap: 6px; padding: 11px 20px; border: none;
167     border-radius: var(--radius-sm);
168     font-size: 14px; font-weight: 600; font-family: inherit;
169     cursor: pointer; transition: var(--transition);
170     text-decoration: none; white-space: nowrap;
171 }
172 .btn-block { width: 100%; }
173 .btn-primary {
174     background: linear-gradient(135deg, var(--primary), var(--accent));
175     color: #fff; box-shadow: 0 4px 12px rgba(99,102,241,.3);
176 }
177 .btn-primary:hover {
178     transform: translateY(-1px);
179     box-shadow: 0 6px 16px rgba(99,102,241,.4);
180     color: #fff;
181 }
182 .btn-primary:active { transform: translateY(0); }
183
184 .btn-ghost { background: #fff; color: var(--text); border: 1.5px solid var(--border); }
185 .btn-ghost:hover { background: var(--bg); border-color: var(--border-strong); color: var(--text); }
186
187 .btn-sm {
188     display: inline-block; padding: 6px 12px; border-radius: 6px;
```

```
    font-size: 12px; font-weight: 600; font-family: inherit;
    text-decoration: none; margin-left: 4px; border: none;
    cursor: pointer; transition: var(--transition);
}
.btn-edit   { background: #eef2ff; color: var(--primary-dark); }
.btn-edit:hover   { background: var(--primary); color: #fff; }
.btn-delete { background: var(--error-bg); color: #b91c1c; }
.btn-delete:hover { background: var(--error); color: #fff; }
.btn-ok     { background: var(--success-bg); color: #047857; }
.btn-ok:hover     { background: var(--success); color: #fff; }
.btn-warn   { background: var(--warn-bg); color: #b45309; }
.btn-warn:hover   { background: var(--warn); color: #fff; }

.btn-logout {
    padding: 8px 16px; background: #f1f5f9;
    color: var(--text); border-radius: var(--radius-sm);
    font-size: 13px; font-weight: 600; text-decoration: none;
    transition: var(--transition);
}
.btn-logout:hover { background: var(--error); color: #fff; }

/* ================== Alerts ================== */
.alert {
    padding: 12px 16px; border-radius: var(--radius-sm);
    margin-bottom: 18px; font-size: 14px; font-weight: 500;
    border-left: 4px solid;
    animation: slideDown .3s ease;
}
@keyframes slideDown {
    from { opacity: 0; transform: translateY(-8px); }
    to   { opacity: 1; transform: translateY(0); }
}
.alert-error   { background: var(--error-bg);   color: #991b1b; border-color: var(--error); }
.alert-success { background: var(--success-bg); color: #065f46; border-color: var(--success); }

/* ================== App layout ================== */
.app-body { background: var(--bg); min-height: 100vh; display: flex; flex-direction: column; }

.navbar {
    background: #fff;
    border-bottom: 1px solid var(--border);
    box-shadow: var(--shadow-sm);
    position: sticky; top: 0; z-index: 100;
}
.navbar-inner {
    max-width: 1200px; margin: 0 auto;
    padding: 0 24px; height: 64px;
    display: flex; align-items: center; gap: 32px;
}
.brand {
    display: flex; align-items: center; gap: 10px;
    font-weight: 700; font-size: 18px;
    color: var(--text); text-decoration: none; flex: 1;
}
.brand:hover { color: var(--primary); }
.brand-icon {
    width: 36px; height: 36px; border-radius: 10px;
    background: linear-gradient(135deg, var(--primary), var(--accent));
    color: #fff;
    display: flex; align-items: center; justify-content: center;
    font-size: 18px;
}

.nav-user { display: flex; align-items: center; gap: 14px; }
.user-pill {
    display: flex; align-items: center; gap: 10px;
    padding: 4px 14px 4px 4px;
    background: #f8fafc; border-radius: 999px;
    border: 1px solid var(--border);
}
.user-avatar {
    width: 32px; height: 32px; border-radius: 50%;
    background: linear-gradient(135deg, var(--primary), var(--accent));
    color: #fff;
    display: flex; align-items: center; justify-content: center;
    font-weight: 700; font-size: 14px;
}
.user-meta { display: flex; flex-direction: column; line-height: 1.1; }
.user-name { font-size: 13px; font-weight: 600; color: var(--text); }
.user-role { font-size: 11px; color: var(--text-muted); }

@media (max-width: 720px) {
    .navbar-inner { gap: 12px; padding: 0 16px; }
    .user-meta { display: none; }
}

.main-content { flex: 1; max-width: 1200px; width: 100%; margin: 0 auto; padding: 32px 24px; }

.page-header {
    display: flex; justify-content: space-between;
    align-items: flex-end; margin-bottom: 24px;
    gap: 16px; flex-wrap: wrap;
}
.page-title { font-size: 26px; font-weight: 700; color: var(--text); margin-bottom: 4px; }
.page-sub { color: var(--text-muted); font-size: 14px; }

/* ================== Statistic cards ================== */
.stat-grid {
    display: grid; gap: 14px; margin-bottom: 8px;
    grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
```

```
}
.stat-card {
    background: var(--card-bg);
    border: 1px solid var(--border);
    border-radius: var(--radius);
    box-shadow: var(--shadow-sm);
    padding: 18px 20px;
    display: flex; flex-direction: column; gap: 4px;
}
.stat-value { font-size: 26px; font-weight: 700; color: var(--primary-dark); line-height: 1.1; }
.stat-value.stat-text { font-size: 20px; }
.stat-label { font-size: 12px; color: var(--text-muted); font-weight: 600; }
.stat-warn   { border-color: var(--warn);  background: #fffbeb; }
.stat-warn   .stat-value { color: #b45309; }
.stat-danger { border-color: var(--error); background: #fff5f5; }
.stat-danger .stat-value { color: #b91c1c; }
.stat-note { font-size: 12px; color: var(--text-light); margin: 8px 0 20px; }

/* ================== Card ================== */
.card {
    background: var(--card-bg);
    border-radius: var(--radius-lg);
    box-shadow: var(--shadow-md);
    border: 1px solid var(--border);
    overflow: hidden;
    margin-bottom: 24px;
}
.form-card { padding: 24px 28px; }

.two-col { display: grid; grid-template-columns: 3fr 2fr; gap: 24px; align-items: start; }
@media (max-width: 900px) { .two-col { grid-template-columns: 1fr; } }

.card-title {
    font-size: 18px; font-weight: 600;
    color: var(--text); margin-bottom: 18px;
    padding-bottom: 12px;
    border-bottom: 1px solid var(--border);
}
.card-title-pad { padding: 18px 20px 12px; margin-bottom: 0; }

.card-toolbar {
    padding: 16px 20px;
    border-bottom: 1px solid var(--border);
    display: flex; align-items: center; gap: 12px;
    background: #fafbff; flex-wrap: wrap;
}

.search-wrap { position: relative; flex: 1; max-width: 420px; min-width: 200px; }
.search-icon {
    position: absolute; left: 14px; top: 50%;
    transform: translateY(-50%);
    font-size: 14px; color: var(--text-light);
    pointer-events: none;
}
.search-input {
    width: 100%; padding: 10px 14px 10px 40px;
    border: 1.5px solid var(--border);
    border-radius: var(--radius-sm);
    font-size: 14px; background: #fff;
    transition: var(--transition); font-family: inherit;
}
.search-input:focus {
    outline: none; border-color: var(--primary);
    box-shadow: 0 0 0 3px rgba(99,102,241,.15);
}
.filter-select {
    padding: 10px 12px; border: 1.5px solid var(--border);
    border-radius: var(--radius-sm); background: #fff;
    font-size: 14px; font-family: inherit; cursor: pointer;
}
.filter-select:focus { outline: none; border-color: var(--primary); }

.badge {
    background: #eef2ff; color: var(--primary-dark);
    padding: 6px 12px; border-radius: 999px;
    font-size: 12px; font-weight: 600; margin-left: auto;
}

/* ================== Status pills ================== */
.pill {
    display: inline-block; padding: 3px 10px;
    border-radius: 999px; font-size: 11px;
    font-weight: 700; text-transform: capitalize;
    background: #f1f5f9; color: var(--text-muted);
}
.pill-admin     { background: #ede9fe; color: #5b21b6; }
.pill-librarian { background: #dbeafe; color: #1d4ed8; }
.pill-student   { background: #dcfce7; color: #15803d; }
.pill-visitor   { background: #fef3c7; color: #b45309; }
.pill-active    { background: var(--success-bg); color: #047857; }
.pill-suspended { background: var(--error-bg);   color: #b91c1c; }
.pill-pending   { background: var(--warn-bg);    color: #b45309; }
.pill-issued    { background: #dbeafe; color: #1d4ed8; }
.pill-returned  { background: var(--success-bg); color: #047857; }
.pill-rejected  { background: var(--error-bg);   color: #b91c1c; }

/* ================== Table ================== */
.table-wrap { overflow-x: auto; }
.data-table { width: 100%; border-collapse: collapse; font-size: 14px; }
.data-table thead th {
```

```
    background: #fafbff; text-align: left;
    padding: 13px 20px; font-weight: 600;
    font-size: 12px; letter-spacing: .3px;
    color: var(--text-muted);
    border-bottom: 1px solid var(--border);
}
.data-table tbody td {
    padding: 14px 20px;
    border-bottom: 1px solid #f1f5f9;
    color: var(--text); vertical-align: middle;
}
.data-table tbody tr { transition: background .15s ease; }
.data-table tbody tr:hover { background: #fafbff; }
.data-table tbody tr:last-child td { border-bottom: none; }
.text-right { text-align: right; }
.nowrap { white-space: nowrap; }
.sub { font-size: 12px; color: var(--text-muted); }
.text-danger { color: var(--error); font-weight: 600; }

.empty {
    text-align: center !important;
    padding: 44px 20px !important;
    color: var(--text-light) !important;
    font-style: italic;
}

/* ================== Info box + suggestions ================== */
.info-box {
    background: var(--info-bg); border-radius: var(--radius-sm);
    padding: 14px 16px; margin-bottom: 16px;
    display: flex; flex-direction: column; gap: 6px;
    align-items: flex-start;
}
.info-box p { font-size: 14px; }

.suggestion-list { margin-top: 20px; display: flex; flex-direction: column; gap: 8px; }
.suggestion-item {
    display: flex; align-items: center; justify-content: space-between;
    gap: 12px; padding: 10px 12px;
    background: var(--bg); border: 1px solid var(--border);
    border-radius: var(--radius-sm); font-size: 14px;
}
.suggestion-item .sub { display: block; }

/* ================== Library card / day pass ================== */
.library-card {
    background: linear-gradient(135deg, var(--primary), var(--accent));
    color: #fff; border-radius: var(--radius);
    padding: 20px; box-shadow: var(--shadow-md);
}
.lib-card-top {
    display: flex; justify-content: space-between; align-items: center;
    font-size: 12px; opacity: .9; margin-bottom: 14px;
}
.lib-card-brand { font-weight: 700; letter-spacing: .5px; }
.lib-card-type {
    background: rgba(255,255,255,.22); padding: 3px 10px;
    border-radius: 999px; font-weight: 600;
}
.lib-card-name { font-size: 20px; font-weight: 700; margin-bottom: 14px; }
.lib-card-rows {
    display: grid; grid-template-columns: auto 1fr;
    gap: 6px 14px; font-size: 13px;
}
.lib-card-rows span   { opacity: .85; }
.lib-card-rows strong { font-weight: 600; text-align: right; }
.lib-card-foot {
    margin-top: 14px; padding-top: 12px;
    border-top: 1px solid rgba(255,255,255,.25);
    font-size: 12px; opacity: .9;
}
.pass-code {
    font-size: 26px; font-weight: 700; letter-spacing: 2px;
    margin-bottom: 14px; font-family: "Courier New", monospace;
}

/* Printing: only the card or the pass goes on paper. */
@media print {
    body { background: #fff; }
    .navbar, .footer, .card-toolbar, .page-header,
    .stat-grid, .stat-note, .table-wrap, .form, .form-actions { display: none !important; }
    .card { box-shadow: none; border: none; margin: 0; }
    .library-card { box-shadow: none; }
}

/* ================== Footer ================== */
.footer {
    text-align: center; padding: 24px;
    color: var(--text-muted); font-size: 13px;
    border-top: 1px solid var(--border);
    background: #fff;
}

/* ================== Scrollbar ================== */
::-webkit-scrollbar { width: 10px; height: 10px; }
::-webkit-scrollbar-track { background: var(--bg); }
::-webkit-scrollbar-thumb { background: var(--border-strong); border-radius: 5px; }
::-webkit-scrollbar-thumb:hover { background: var(--text-light); }
```

Part 10 — The twelve features, role by role

Each role owns one kind of record and three features that no other role has. This chapter shows where each one lives in the code.

10.1 The feature map

Role	CRUD on	Feature 1	Feature 2	Feature 3
Admin	User accounts (all roles)	Suspend / activate accounts and approve membership upgrades	System-wide activity log with search	Live statistics panel, refreshed by AJAX
Librarian	Books	Issue and return desk with automatic stock and fines	Low stock alert list	Catalogue export to a CSV file
Student	My borrow requests	Catalogue browser showing live availability	Due-date and fine tracker	Printable digital library card
Visitor	My visit passes	Membership upgrade application	Book suggestion box	Printable day pass with a unique code

No feature appears on two dashboards. Where two roles touch the same information they do so from opposite sides: the student **creates** a borrow request, the librarian **decides** it; the visitor **submits** a membership application, the admin **approves** it.

10.2 Admin features in detail

Feature 1 — account control and membership approval

Suspending is the middle ground between doing nothing and deleting. A suspended account keeps its history and its borrowing record but cannot sign in; the sign-in check refuses it after the password has been verified. One click puts it back.

The same feature handles the membership queue. Approving an application writes the decision **and** changes that person's role from visitor to student, which is the only place in the project where a role changes after registration.

Where	What is there
`admin_controller.php`, action `status`	Validates the target status against the white list, blocks self-suspension, logs the change.
`admin_controller.php`, action `membership`	Records the decision and, when approved, calls `set_user_role`.
`user_model.php`	`set_user_status()` and `set_user_role()`.
`visitor_model.php`	`get_pending_applications()` and `decide_application()`.

Feature 2 — the activity log

Every meaningful action calls `log_activity()`, which records the username, the role, a description, the network address and the time. The dashboard shows the twelve most recent entries and a search box that queries all four columns.

The username and role are stored as text rather than only as a link to the users table, so the log stays readable after an account is deleted — which is exactly when it is most needed.

Feature 3 — the live statistics panel

Ten numbers gathered by `get_system_stats()`. The browser asks for them again every fifteen seconds and writes them into the page, so an admin watching the dashboard sees new registrations appear without touching anything. Each card carries a `data-stat` attribute naming its value, so adding a new number needs no new JavaScript.

10.3 Librarian features in detail

Feature 1 — the issue and return desk

The busiest feature in the project, and the only place where two tables must change together.

Click	What happens
Issue	The shelf count is lowered first and the result checked. If no copy was available, nothing else happens and the librarian is told which title is out. Otherwise the request becomes issued and a due date fourteen days ahead is stored. If that second step were to fail, the copy is put straight back.
Return	The fine is calculated from the due date, stored against the request, the status becomes returned, and the copy goes back on the shelf.
Reject	The request is closed without ever being issued, so no stock moves at all.

Every one of these actions is also guarded in SQL: the issue query ends `AND status = 'pending'` and the return query ends `AND status = 'issued'`. Two librarians clicking the same button at the same moment cannot both succeed.

Feature 2 — low stock alerts

Any title with `LOW_STOCK` copies or fewer appears in its own table, emptiest first, and the statistic card at the top of the page turns amber while any exist. The same threshold colours the copies column in the main catalogue, so the warning is visible in two places without being calculated twice.

Feature 3 — CSV export

One link produces a spreadsheet of the whole catalogue, named with today's date. It is written straight to the browser with `php://output`, so nothing is stored on the server, and `fputcsv` handles the escaping so a title containing a comma cannot break the file. The export is recorded in the activity log like any other action.

10.4 Student features in detail

Feature 1 — the catalogue browser

The whole catalogue with a live search and an availability badge on every row. Titles with copies on the shelf show a **Request** button that selects that book in the form at the top of the page and scrolls to it; titles that are entirely out show "Unavailable" and no button.

Feature 2 — the due-date and fine tracker

Five counters across the top of the dashboard, and a due-date column that phrases the same number two ways: "6 days left" in grey, or "4 days late" in red. The fine card shows what is accruing **right now** across every book still out, which is deliberately more useful than a total that only appears after the book is returned.

The figure comes from `calculate_fine()`, the same function the librarian's return button uses, so what the student sees is exactly what they will be asked to pay.

Feature 3 — the digital library card

A card showing the name, a padded card number such as `STU-00042`, the username, the member-since date, the loan period and the fine rate. **Print my card** opens the browser print dialog, and the `@media print` block in the stylesheet hides everything except the card itself.

10.5 Visitor features in detail

Feature 1 — membership application

A visitor explains in at least ten characters why they want to join. The panel then shows one of four states — never applied, waiting, approved or rejected — and only offers the form again when it makes sense. A second application cannot be sent while one is still waiting.

Feature 2 — the book suggestion box

A visitor records books the library should buy, and can remove their own entries. Kept as a table on the visitor's own dashboard, so the record belongs to the person who made it.

Feature 3 — the printable day pass

Every booking generates a code such as `LIB-4821-KQ`. The dashboard shows the next upcoming pass as a printable card with the code in large monospaced type. The alphabet used to build the code leaves out I and O so that it cannot be misread as 1 and 0 at the desk.

Part 11 — Security explained

Every defence in the project, what it stops, and where to find it. Written so a non-technical reader can follow the reasoning.

11.1 The one rule behind everything

Everything the browser sends can be faked. Every box, every hidden field, every value in the address bar, every cookie, and every rule enforced by JavaScript can be edited by anybody with the developer tools that ship with every browser.

So the browser is treated as a helpful assistant, never as a source of truth. It is asked to check forms because that gives a fast answer, but nothing it says is believed. The server checks everything again, on its own, every time.

11.2 The defences, one by one

The attack	How it works, in plain words	What stops it here
SQL injection	A user types database commands into an ordinary form box, hoping the site glues them into its own query. `'; DROP TABLE books;--` in a search box could delete everything.	Prepared statements in every model. The query shape is sent to MySQL first, values second, so a value can never become a command. Tested with that exact input: it was stored as a book title and the table survived.
Cross-site scripting (XSS)	A user saves a script as their name or a book title. Every visitor who later sees that page runs the attacker's script in their own browser.	`esc()` on the server before printing, and `esc()` again in JavaScript before inserting anything from a background request. Tested: `<script>alert(1)</script>` as a title appears on screen as text.
Cross-site request forgery (CSRF)	While you are signed in, a different website makes your browser send a request here. Your cookie goes with it and the server thinks you meant it.	A secret token in every form and every action link, checked by `csrf_check()`. A foreign site cannot read the token. `samesite=Lax` on the cookie is a second, independent barrier.
Password theft	Somebody obtains a copy of the database and reads everyone's password.	`password_hash()` scrambles passwords one way. Nobody — not the admin, not the thief — can turn a hash back into a password. Checking is done by hashing the attempt and comparing.
Session hijacking	An attacker steals the session cookie and becomes you.	`httponly` stops JavaScript reading the cookie at all, and the thirty-minute idle timeout limits how long a stolen one would be worth anything.
Session fixation	An attacker plants a session id they already know on a victim, then waits for them to sign in with it.	`session_regenerate_id(true)` issues a brand-new id at the exact moment of signing in, so the planted id becomes worthless.
Privilege escalation	A student types the admin address, or signs up choosing the admin role.	`require_role()` runs before every dashboard, every AJAX endpoint checks the role again, and registration accepts only three roles from a white list.

The attack	How it works, in plain words	What stops it here
URL tampering (IDOR)	A student changes `id=7` to `id=8` in the address to open somebody else's record.	Every student and visitor query carries the id from the **session** in its `WHERE` clause. Another person's row simply comes back empty.
Username enumeration	An attacker learns which usernames exist by watching for a different error message.	A wrong username and a wrong password produce the same sentence. The "account suspended" message is only shown after the password has been proved correct.
Timing attacks	Guessing a secret one character at a time by measuring which attempt takes fractionally longer.	`hash_equals()` compares the CSRF token in constant time.
Locking yourself out	An administrator suspends, demotes or deletes their own account and nobody can get back in.	Three explicit guards in `admin_controller.php` that refuse all three actions on your own id.

11.3 Where each defence lives

Defence	Function	File
Escape output (server)	`esc()`	`helpers/helpers.php`, used in every view
Escape output (browser)	`esc()`	`assets/js/app.js`
CSRF token	`csrf_token()`, `csrf_field()`, `csrf_url()`, `csrf_check()`	`helpers/helpers.php`
Password hashing	`password_hash()`, `password_verify()`	`user_model.php`, `auth_controller.php`
Role gate	`require_role()`	`helpers/helpers.php`, called from `index.php`
AJAX role checks	`if ($role !== …) break;`	`controllers/ajax_controller.php`
Idle timeout	`check_session_timeout()`	`helpers/helpers.php`
Fresh session id	`session_regenerate_id(true)`	`auth_controller.php`
Cookie hardening	`session_set_cookie_params()`	`config/config.php`
Prepared statements	`mysqli_prepare` + `bind_param`	every file in `models/`
Ownership filters	`WHERE … AND student_id = ?`	`borrow_model.php`, `visitor_model.php`

11.4 What a real deployment would still need

Being clear about what a teaching project does not do is part of documenting it honestly. Before this system carried real personal data, it would need:

- **HTTPS**, and the `secure` flag added to the session cookie. Without encryption in transit, everything above protects the server but not the wire.
- **Rate limiting on sign-in**, so an attacker cannot try thousands of passwords. There is no lockout in this version.

- **A password reset flow.** Today only an administrator can set a new password.
- **A stronger password policy** than six characters.
- **Database backups**, and a MySQL account with only the permissions the application actually needs rather than `root`.
- **Removal of the demo credentials** printed on the sign-in page, and a change of the default admin password.

Part 12 — Validation and the AJAX layer

12.1 Why every rule is written twice

Each rule exists in two places: once in JavaScript for speed, once in PHP for truth. This is not duplication by accident — it is the standard arrangement, and each copy has a different job.

	JavaScript (browser)	PHP (server)
Runs where	On the user's own computer	On the server
Purpose	Instant feedback, no waiting for the network	The rule that actually decides
Can be bypassed?	Yes — trivially	No
If it is missing	The form feels slow and clumsy	The system is insecure
Where it is	`validateForm()` in `app.js`	The `if / elseif` chain at the top of each controller

NOTE
A good way to demonstrate this in a viva: switch JavaScript off in the browser, submit an empty form, and show that the server still refuses it with a proper message.

12.2 The complete rule table

Field	Rule	Where it is checked
Any required field	Must not be empty or only spaces	`is_blank()` — both sides
Full name	At least 3 characters	Both
Email	A valid address	`filter_var` on the server; a simple pattern in the browser
Contact number	6–20 characters, digits and `+ - ( )` only	Both, using the same pattern
Username	4–20 letters, digits or underscore, and unique	Both; uniqueness on the server only
Password	At least 6 characters	Both
Repeat password	Must equal the first	Both
Role	One of the four permitted words	Server only — a drop-down can be edited
Book quantity	A whole number, 0 or more	`ctype_digit` on the server; `type="number"` in the browser
Book price	A number, 0 or more	`is_numeric` on the server
ISBN	6–20 characters, digits and dashes	Server pattern
Pick-up / visit date	A real date, not in the past	`checkdate` on the server; `min` on the input
Guests	A whole number from 1 to 10	Both
Membership reason	At least 10 characters	Both
Any field on update	None may be left empty	Server — the explicit "no field can be NULL" branch

12.3 The AJAX endpoints

Every background request goes to `index.php?page=ajax&action=…` and comes back as JSON. Each endpoint checks the role for itself, and anything unrecognised is refused.

Action	Who may call it	Parameters	What comes back
`check_username`	Anybody	`username`	`{ok, message}` — whether the name is free. No user details, ever.
`search_users`	Admin	`q`, `role`	Matching accounts, without password hashes
`stats`	Admin	—	The ten numbers for the statistics panel
`search_logs`	Admin	`q`	Matching activity log entries
`search_books`	Librarian, Student	`q`	Matching books
`low_stock`	Librarian	—	Books at or below the threshold
`search_issues`	Librarian	`q`	Borrow requests with student names attached
`search_requests`	Student	`q`	That student's own requests only
`search_passes`	Visitor	`q`	That visitor's own passes only

The flow is the same every time: the user types, `liveSearch()` waits 250 milliseconds, `ajaxTable()` fetches the JSON, and a small per-page function turns each object into a table row — escaping every value on the way in.

Part 13 — Testing and evidence

What was tested, how, and what the results were.

13.1 How the project was tested

The system was installed on a real PHP 8.3 and MariaDB server, the schema was imported exactly as a student would import it through phpMyAdmin, and an automated script then drove the application through a browser-like client, checking both the pages returned and the contents of the database afterwards.

Checking the database as well as the page matters. A page that says "Book issued" proves nothing on its own; the test also confirms that the status column changed, that the due date was set, and that the shelf count went down by exactly one.

13.2 Results

Group	Checks	Result
A — Registration for all three self-service roles	3	All passed
B — Validation and CSRF rejection	5	All passed
C — Sign-in and role routing	5	All passed
D — Role gates on pages and AJAX	5	All passed
E — Admin CRUD, suspend, statistics, log	10	All passed
F — Librarian CRUD and CSV export	7	All passed
G — Student CRUD and the full borrow cycle	13	All passed
H — Visitor CRUD, membership, suggestions	8	All passed
I — XSS and SQL injection attempts	2	All passed
J — Sign-out and session invalidation	2	All passed
Total	**60**	**60 passed, 0 failed**

The server error log was also inspected after every run: **no warnings, no notices and no deprecation messages** were produced. The HTML of all six screens was parsed and every tag was found to be correctly closed.

13.3 The checks worth repeating in a demonstration

What to show	What it proves
Sign in as a student, then type the admin address by hand	The role gate works on the server, not just by hiding buttons.
Add a book titled `<script>alert(1)</script>`	It appears as ordinary text. XSS is escaped, not executed.
Add a book titled `Robert'); DROP TABLE books;--`	It is stored as a title and the catalogue is untouched. Prepared statements work.
Issue a book and watch the copies column	Two becomes one. The stock and the request change together.
Change a due date in the database to three days ago, then return the book	The fine is exactly 15 — three days at five per day.

What to show	What it proves
Switch JavaScript off and submit an empty form	The server refuses it anyway.
Suspend an account, then try to sign in as it	A suspended user is blocked after the password check, not before.
Approve a visitor’s membership, then sign in as them	The role changed and the student dashboard appears.

Part 14 — Changing and extending the project

14.1 Settings you can change safely

All of these live in `config/config.php` and take effect everywhere immediately.

Constant	Default	Effect of changing it
`LOAN_DAYS`	14	The due date moves. The librarian's confirmation message updates itself, because it reads the same constant.
`FINE_PER_DAY`	5	Fines change on the server **and** in the browser, because the value is passed into JavaScript rather than written there twice.
`LOW_STOCK`	3	How many copies count as low, in the alert table, the statistic card and the coloured badge.
`CURRENCY`	`$`	The symbol printed before every amount.
`SESSION_TIMEOUT`	1800	Seconds of inactivity before automatic sign-out.
`APP_NAME`	`LibSys`	The name in the navigation bar, page titles, library card and day pass.

14.2 Adding a field, in four steps

Suppose books need a **publisher**. Because the layers are separate, the change is four small edits in four predictable places — and that predictability is the whole point of MVC.

1. **Database.** `ALTER TABLE books ADD publisher VARCHAR(120) NOT NULL DEFAULT '';`
2. **Model.** Add `publisher` to the column lists in `get_books`, `get_book`, `add_book` and `update_book`, and add one more `s` to each `bind_param` type string.
3. **Controller.** Read `$_POST['publisher']`, add it to the validation chain, pass it to the model.
4. **View.** Add the input to the form, a column to the table, and the same column to the JavaScript row builder so search results match.

> **NOTE**
> If you ever find yourself unsure which file to edit, ask what kind of change it is. Wrong data saved → model. Wrong rule applied → controller. Wrong appearance → view. If a change needs edits in all three, that is normal and expected; what matters is that it is never ambiguous.

14.3 Sensible next projects

- **Email reminders** three days before a book is due.
- **Reservations**, so a student can join a queue for a title that is fully out.
- **A categories table**, replacing the free-text category with a proper relationship.
- **Pagination**, so a catalogue of ten thousand books does not load in one page.
- **Password reset by email**, removing the administrator from the loop.
- **A librarian view of visitor suggestions**, closing the loop between what readers want and what gets bought.

Part 15 — Glossary

Every technical term used in this document, in one alphabetical list.

Term	Meaning
AJAX	Fetching data from the server in the background so part of a page can change without reloading the whole thing.
Bind / binding	Attaching a value to a `?` placeholder in a prepared statement, declaring its type as you do so.
Constant	A named value that cannot be changed once set. Created with `define()`.
Controller	The code that receives a request, checks it, calls the model and chooses a view.
Cookie	A small piece of text the server asks the browser to store and send back on later requests.
CRUD	Create, Read, Update, Delete — the four basic things you can do to a record.
CSRF	Cross-Site Request Forgery: another site making your browser send a request here while you are signed in.
CSV	Comma-Separated Values — a plain text table that spreadsheet programs can open.
Debounce	Waiting for a short pause in typing before acting, so ten keystrokes cause one request instead of ten.
Escaping	Converting characters that have a special meaning into harmless codes before printing them.
Flash message	A message stored in the session, shown once on the next page, then deleted.
Foreign key	A rule the database enforces: this column must point at a row that really exists in another table.
Front controller	A single entry point that every request passes through. Here, `index.php`.
Hash	The scrambled, one-way result of running a password through a hashing function. It cannot be reversed.
HTTP status code	A number describing how a request went: 200 fine, 302 redirect, 403 forbidden, 404 missing.
JOIN	Combining rows from two tables in one query, for example a request with its book's title.
JSON	A simple text format for data, easy for both PHP and JavaScript to read.
Model	The code that talks to the database. It never prints HTML.
MVC	Model, View, Controller — the rule that keeps data access, presentation and decisions apart.
Prepared statement	A query sent to the database in two parts: the shape first, the values second. Prevents SQL injection.
Post/Redirect/Get	Redirecting after a change so that refreshing the page cannot repeat it.
Query	A question or instruction sent to the database.
Role	Which kind of user somebody is. Decides which screen they see and what they may do.
Session	The server's memory of one visitor, kept between page loads and identified by a cookie.
SQL	The language used to talk to the database.
SQL injection	Typing database commands into a form box hoping the site will run them.
Token	A secret random value used to prove a request came from a legitimate page.
UTF-8	The text encoding that supports every alphabet, so names in any language are stored correctly.
Validation	Checking that submitted data makes sense before acting on it.

Term	Meaning
View	The code that produces HTML. It never runs a query.
White list	A list of what is allowed, refusing everything else. Safer than listing what is forbidden.
XAMPP	A free bundle that installs the Apache web server, PHP and MySQL together.
XSS	Cross-Site Scripting: saving a script inside data so it runs in other people’s browsers.

Part 16 — Viva preparation

Twenty-five questions an examiner is likely to ask, with short answers you can say out loud.

16.1 Architecture

Question	Answer
What is MVC and why use it?	Model holds the SQL, View holds the HTML, Controller makes the decisions. It means every kind of change has exactly one home, so bugs are easy to locate.
What is a front controller?	One entry point, `index.php`, that every request passes through. The session, the database connection and the role check happen once and cannot be forgotten.
Why one users table instead of four?	All four roles share the same fields and the same sign-in form. One table means one sign-in query and one list query instead of four of each.
Where would you add a new role?	Add it to the `ENUM` in the schema, the `$roles` white list in the admin controller, a `case` in `index.php`, and create a controller and a view. Sign-in needs no change, because the role word is the page name.
Why does the model never print HTML?	So the same function can serve a normal page and an AJAX request. `get_books()` feeds both the table and the JSON endpoint.

16.2 Security

Question	Answer
How do you prevent SQL injection?	Prepared statements everywhere. The query shape goes to MySQL before the values do, so a value can never become a command.
How do you prevent XSS?	Escape on output, in two places: `esc()` in PHP before printing, and `esc()` in JavaScript before inserting anything fetched in the background.
What is CSRF and how do you stop it?	Another site making my browser send a request here while I am signed in. A secret token in every form and action link, checked by `csrf_check()`, plus `samesite=Lax` on the cookie.
Why `hash_equals` instead of `==`?	It always takes the same amount of time, so an attacker cannot work out the token character by character from response times.
How are passwords stored?	As `password_hash()` output, which is one-way. Verification hashes the attempt and compares. Nobody can read the original, including the administrator.
Why regenerate the session id at login?	To defeat session fixation, where an attacker plants a session id and waits for the victim to sign in with it.
A student changes `id=7` to `id=8`. What happens?	Nothing. Every student query includes `AND student_id = ?` with the id from the session, so another person's row comes back empty.
Is JavaScript validation enough?	No. It can be switched off in seconds. It is for speed; the PHP checks are the real gate.
Why the same message for wrong username and wrong password?	Different messages would tell an attacker which usernames exist.

16.3 Database and code

Question	Answer
Why DECIMAL for money and not FLOAT?	Floating point cannot store values like 0.10 exactly, and the small errors accumulate. DECIMAL is exact.
Why is a phone number stored as text?	It may start with `+` or `0`, and a numeric column would destroy both.
What is CASCADE versus SET NULL?	CASCADE deletes the children with the parent, used where a child is meaningless alone. SET NULL just forgets the link, used for `books.added_by` so a book survives the librarian leaving.
Why `quantity = quantity + ?` instead of reading then writing?	The database does the arithmetic, so two people clicking at the same instant cannot both read the same number and overwrite each other.
What does `affected_rows` add?	It tells you whether a row really changed. A query that matched nothing still "succeeds", so this is the useful question.
Why `AND status = 'pending'` in the issue query?	It makes issuing the same book twice impossible. The second attempt matches no rows and changes nothing.
What does `ctype_digit` catch that `intval` misses?	`intval('2abc')` silently gives 2. `ctype_digit('2abc')` is false, so the bad input is rejected rather than quietly accepted.
Why `LEFT`-anchored patterns like `/^…$/`?	Without the anchors the pattern would match anything that merely contains a valid section, so `bad!!name` would pass.

16.4 Features and behaviour

Question	Answer
How is a fine calculated?	Days past the due date times `FINE_PER_DAY`, in `calculate_fine()`. While the book is out it is measured to today, so it grows; on return it is frozen into the record.
Why redirect after saving?	Post/Redirect/Get. Without it, pressing F5 would repeat the save and the browser would ask about resending the form.
What is a flash message?	A message stored in the session, printed once on the next page and deleted as it is read, so a refresh does not show it again.
Why does AJAX search wait 250 milliseconds?	Debouncing. Without it every keystroke would be a separate request; with it, one search runs when typing stops.

16.5 Two questions worth asking yourself

"What would you do differently next time?" An honest answer scores better than a defensive one. Reasonable answers: split `visitor_model.php` into three files as it grows; add pagination before the catalogue gets large; introduce a proper categories table; add rate limiting on the sign-in form.

"What is the weakest part of the project?" The lack of HTTPS and of any limit on repeated sign-in attempts. Both are listed in section 11.4, and knowing your own weak points is a sign that you understand the system rather than merely finished it.

Part 17 — Closing summary

This project is a complete, working library management system for four kinds of user, written in plain PHP and MySQL with no framework, and organised so that a beginner can read it.

It contains 26 files and roughly 4,000 lines. Every role can create, read, update, delete and search its own records, and each has three features of its own. The security work — prepared statements, output escaping, CSRF tokens, hashed passwords, role gates, ownership filters and session hardening — is present in every file rather than bolted on at the end.

The system was tested against a live server with sixty automated checks covering registration, sign-in, role separation, every CRUD path, the complete borrow-issue-return cycle including fine calculation, and deliberate XSS and SQL injection attempts. All sixty passed, and the server produced no warnings.

Most of all, it was written to be explained. Every design choice in this document has a reason attached, because a project you can defend is worth more than a project that merely runs.